

Модуль 1. Основы Claude Code: как настроить инструмент и научиться им управлять

УРОК 1. CLn01 Claude Code: настройка инфраструктуры, подключение и основные команды

Тезисы урока:

- что такое Claude Code и чем он отличается от обычных чат-ботов и AI-помощников в редакторе кода;
- в каких задачах Claude Code действительно полезен;
- как установить Claude Code и подключить модель;
- как запустить рабочую среду и начать работу через CLI;
- как устроен интерфейс Claude Code;
- какие базовые команды нужны для старта;
- Что такое действия и ограничения агента (permissions). Как ими управлять.
- как агент читает проект и работает с файлами в папке.

Практика: студенты настроят CLI под свои задачи.

УРОК 2. CLn02 Промптинг в Claude Code: как составлять запросы и настраивать CLAUDE.md

Тезисы урока:

- как формулировать запросы для терминального агента, чтобы он понимал задачу, контекст и ожидаемый результат;
- чем хороший запрос отличается от расплывчатого;
- как ставить задачи на уровне действия, файла, папки и целого сценария;
- что такое CLAUDE.md и зачем он нужен в проекте;
- как через CLAUDE.md задавать агенту постоянные правила работы;
- как указывать, с какими файлами и папками агенту можно работать;
- как задавать ограничения, доступы и правила поведения агента;
- как сделать работу Claude Code более предсказуемой и управляемой.

Практика: студенты создадут проект «Умный реорганизатор» — ИИ-агента, который по заданным инструкциям, промптам и правилам в CLAUDE.md приводит в порядок рабочие документы и папки.

УРОК 3. CLn03 Архитектура ИИ-агентов. Работа с файлами, таблицами и данными

Тезисы урока:

- как Claude Code работает с локальными файлами, таблицами и структурированными данными;
- как использовать Claude Code для CSV, Excel, JSON и других форматов;
- как агент получает доступ к данным проекта;
- как Claude Code применять в прикладных задачах бизнеса и аналитики;
- что такое базовая архитектура ИИ-агента;
- как агент получает контекст и превращает текстовую задачу в рабочий результат.

Практика: студенты создадут мини-проект «Панель продаж» — скрипт, который читает Excel-файлы, собирает сводную таблицу и выгружает готовый PDF-отчёт.

Модуль 2. Сайты, автоматизация и усиление возможностей Claude Code

УРОК 4. CLs01 Глубокая настройка CLI-агентов в Claude Code

Тезисы урока:

- как настраивать CLI-агентов в Claude Code на разных уровнях;
- чем отличаются глобальные настройки, настройки проекта и локальные правила агента;
- как конфигурация влияет на поведение CLI-агента;
- где задаются основные правила работы агента;
- как управлять доступами, ограничениями и логикой выполнения команд;
- как подстроить CLI-агента под свой рабочий процесс и тип задач.

Практика: студенты создадут персонально настроенного CLI-агента для своего проекта: зададут правила работы, настройт доступы, ограничения и логику поведения агента под свой сценарий использования.

УРОК 5. CLs02 Создание конверсионных сайтов с помощью Claude Code

Тезисы урока:

- как использовать Claude Code для создания веб-страниц и веб-сайтов;
- как строится структура сайта;
- как сделать понятный и современный интерфейс;
- как добавить форму заявки;
- как быстро собирать рабочие страницы для бизнеса;
- что такое локальный сервер;
- как сайт запускается на компьютере;
- что потребуется, чтобы потом опубликовать сайт в интернете.

Практика: студенты создадут бизнес-визитку или простой конверсионный сайт с понятной структурой, современным интерфейсом и формой заявки.

УРОК 6. CLs03 Скилы в Claude Code: как усиливать агента под свои задачи

Тезисы урока:

- что такое skills в Claude Code;
- зачем выносить отдельные навыки агента в самостоятельные блоки;
- как skills помогают стандартизировать работу;
- как усиливать агента под конкретные типы задач;
- как сделать Claude Code полезнее для своей профессии или ниши;
- в каких случаях достаточно промпта, а когда лучше создавать отдельный навык для многократного использования.

Практика: студенты соберут свой набор skills под выбранную задачу — для работы с контентом, анализом, продажами, документами, исследованиями или другой прикладной областью

УРОК 7. CLs04 MCP: как подключать Claude Code к внешним инструментам

Тезисы урока:

- что такое MCP (Model Context Protocol);
- почему MCP — один из ключевых способов расширения возможностей Claude Code;
- как подключать готовые MCP-инструменты;
- как дать агенту доступ к браузеру, Figma, базе данных и внешним сервисам;
- как использовать внешние источники контекста в прикладной работе;
- почему на этом этапе важнее научиться подключать готовые решения, а не писать свой MCP-сервер с нуля.

Практика: студенты создадут проект «SEO-аудит конкурента», в котором Claude Code откроет сайт, соберёт нужные данные и сформирует аналитический отчёт.

Модуль 3. Цепочки задач и прикладные сценарии для бизнеса

УРОК 8. CLb01 Локальные модели и автоматизация рутинных задач

Тезисы урока:

- зачем нужны локальные модели;
- в каких случаях их имеет смысл использовать;
- чем локальные модели отличаются от облачных;
- какие у локальных моделей ограничения;
- на каком железе они работают;
- в каких задачах локальные модели дают реальную пользу;
- как применять локальную модель для работы с приватными данными, текстами и внутренними документами;
- как использовать локальные модели, чтобы не отправлять данные во внешние сервисы и не тратить токены на облачные запросы.

Практика: студенты соберут рабочий сценарий автоматизации рутинной задачи с использованием локальной модели — например, для обработки клиентской базы, анализа текстов или подготовки типовых действий по данным.

УРОК 9. CLb02 Контент-завод для фриланса и бизнеса: реальные цепочки процессов

Тезисы урока:

- как с помощью Claude Code выстраивать не одну отдельную команду, а рабочую цепочку действий;
- как на основе одного источника получать сразу несколько полезных результатов;
- как использовать сайт, текст, документ или бриф как основу для серии материалов;
- как Claude Code участвует в создании повторяемых процессов;
- как собирать, перерабатывать и структурировать информацию;
- как превращать исходные материалы в набор готовых контентных единиц.

Практика: студенты создадут проект «Контент-завод», который парсит информацию с сайта клиента или использует исходный материала, структурирует данные в контент-план, генерирует серию постов, заголовков и заготовок для контент-плана, публикует посты по расписанию.

Модуль 4. Запуск проектов, мультиагентность и монетизация навыка

УРОК 10. Мультиагентные системы и Claude Agents Team

Тезисы урока:

- как строится работа не с одним агентом, а с несколькими;
- чем отличаются параллельные агенты от последовательных;
- как делить роли между агентами;
- как выстраивать систему, в которой разные агенты выполняют разные части одной задачи;
- что такое Claude Agents Team;
- когда мультиагентная система действительно нужна;
- в каких случаях задачу проще решить одним агентом.

Практика: студенты создадут проект «Команда агентов», где несколько агентов совместно выполняют одну большую задачу и работают как единая система.

УРОК 11. Деплой проекта: сервер, SSH, SSL и базовая безопасность

Тезисы урока:

- что происходит после того, как проект готов локально;
- как перевести проект в формат работающего онлайн-сервиса;
- чем отличается локальный запуск от деплоя;
- какие бывают серверы, и как выбирать между российскими и зарубежными серверами для проектов с данными;
- как подключаться к серверу по SSH;
- как Claude Code помогает с настройкой окружения и запуском проекта;
- что такое переменные окружения и зачем они нужны;
- как работают доступы и SSL-сертификаты.

Практика: студенты задеплойт мультиагентную систему на облачный сервер.

УРОК 12. Монетизация навыка и финальный проект под личную задачу

Тезисы урока:

- что сегодня формируется отдельный рынок вокруг ролей промпт-инженера и вайб-кодера;
- как зарабатывать на навыке работы с Claude Code и вайб-кодингом;
- какие задачи можно брать на фрилансе;
- какие мини-сервисы, сайты и автоматизации реально делать под заказ;
- как упаковывать навык в понятное предложение для клиента;
- в какую сторону можно развиваться дальше после курса;
- как выбрать тему финального проекта под свою личную или рабочую задачу.

Практика: каждый студент соберёт и представит финальный проект под свою личную задачу — ИИ-агента, автоматизацию или мини-сервис, который можно использовать в работе, положить в портфолио или доработать под коммерческий заказ.

7. Условия для получения сертификата

CLs01 Глубокая настройка CLI-агентов в Claude Code

Есть задание

[Следующий урок](#)

Дорогой студент! Интерфейсы современных приложений могут очень быстро меняться. Если вдруг то, что видишь ты, отличается от того, что демонстрирует эксперт, настолько, что тебе непонятно, куда нажимать — напиши об этом в учебный чат, указав номер урока и видео. Наш куратор обязательно поможет тебе разобраться, а мы постараемся как можно скорее актуализировать материал :)



Как проходить уроки эффективно: доступы, оплата и лимиты

1 Доступ к мировым технологиям. Мы показываем лучшие мировые нейросети, но доступ к некоторым из них из РФ и/или других стран может требовать средств защищенного доступа. Мы действуем в рамках закона и НЕ консультируем по настройке соединения, но показываем эти сервисы, потому что они — лидеры рынка. Разобраться с доступом к ним — значит получить конкурентное преимущество.

2 Весь курс можно пройти на бесплатных версиях инструментов. Мы показываем платные PRO-функции для ознакомления, чтобы ты мог(ла) оценить их потенциал и осознанно решить, нужны ли они тебе в будущем.

3 Умное управление лимитами. Бесплатные попытки ограничены, поэтому регистрируйся в сервисе только перед выполнением ДЗ. Если лимиты закончились, а попрактиковаться еще хочется — часто помогает регистрация нового аккаунта на другую почту.

4 Сначала смотри, потом делай. Сначала посмотри урок полностью, чтобы понять логику, и только потом трать генерации. Это сэкономит твоё время и попытки.

Так ты освоишь самые мощные инструменты бесплатно и без лишних нервов! 🚀

Результат дня

- ✅ Выберем проектную задачу, которую будем развивать с помощью Claude Code до конца курса
- ✅ Разберёмся в архитектуре конфигов: глобальные и проектные настройки, в чём их отличие и как они взаимодействуют
- ✅ Поймём систему разрешений агента и научимся управлять тем, какие действия он совершает самостоятельно
- ✅ Узнаем, что такое **MCP**, **субагенты**, **скиллы** и **хуки** — ключевые архитектурные элементы Claude Code

▶ Время чтения и просмотра: ~ 70 минут

🕒 Время выполнения задания: ~60 минут



Под каждым видео в уроке есть текстовая расшифровка, чтобы у тебя была возможность лучше запомнить новые термины, прочитать пояснения и инструкции.

Чтобы прочитать текст, нажимай на него. Текст скрыт в строках подчёркнутых пунктирной линией со значком "⌵" в конце строки.

1. Глубокая настройка CLI-агентов: обзор урока и выбор проекта

Без конечной цели и понимания того, во что мы хотим превратить инструмент, весь потенциал Claude Code не раскрыть. Поэтому первый шаг сегодняшнего урока — зафиксировать направление и выбрать проект, который мы будем развивать до конца курса.



Нужно ответить на вопрос: **зачем я использую агентов и Claude Code?** Именно от ответа на этот вопрос зависит, какие настройки, инструменты и архитектурные решения нам понадобятся.

Что разберём в этом уроке ☞

На занятии мы:

- поговорим о **глобальных и проектных конфигах** — это важно понимать и разделять
- рассмотрим **систему разрешений** агента и как её настраивать
- разберём, что такое **скиллы, субагенты и MCP** — ключевые архитектурные особенности CLI-агентов

Идеи для проекта ☞

Вот несколько направлений, из которых можно выбрать — или взять их за основу для своей идеи:

- **Исследовательский агент** — собирает информацию из сети, делает подробные исследования и сохраняет результаты в нужных форматах: Word, PDF, Markdown. Полезен для создания контента, самообразования, мониторинга конкурентов.
- **Контент-помощник** — автоматизирует создание контента: описания, переводы, вычитка, генерация текста, изображений, видео.
- **Помощник по данным** — работает с таблицами, подключается к удалённым сервисам (например, бухгалтерии), помогает с финансами и аналитикой по клиентам.
- **Кодинговый агент** — помощь в разработке, дебаг, поиск ошибок, поддержка CI/CD-процессов, проверка pull-реквестов, поддержка кодовой базы.
- **Внутренний рабочий инструмент** — личный администратор: планирование дня, структурирование задач, заметки, напоминания, отложенная отправка сообщений.

Подумайте, что вам сейчас наиболее актуально. Запишите своё направление — сегодня у вас должен появиться чёткий ответ, зачем вы используете агентов.



Можно выбрать несколько направлений или сделать микс — главное, зафиксировать это в голове и записать прямо сейчас.



Лайфхак для тех, кто затрудняется с выбором проекта

Начни выбор проекта с рабочих болей.

Подумай, что в твоей работе раздражает, отнимает много времени или регулярно повторяется. Это могут быть задачи, которые ты хочешь делать реже, быстрее или полностью автоматизировать.

Например:

- сбор информации из разных источников;
- подготовка однотипных документов;
- обработка заявок или сообщений;
- составление отчётов;

- перенос данных между сервисами;
- проверка файлов, таблиц или текстов;
- рутинная переписка.

Сначала выпиши такой список вручную. Не пытайся сразу придумать «идеальный проект». На этом этапе важно просто собрать реальные задачи из своей практики.

Дальше передай этот список в Claude Code.

Перед этим создай файл **Claude.md**, как мы разбирали во втором уроке. В нём опиши:

- кто ты;
- чем занимаешься;
- с какими сервисами работаешь;
- какие задачи обычно получаешь;
- какие процессы хочешь упростить.

После этого попроси Claude Code проанализировать твои рабочие боли и предложить идеи проектов для автоматизации.

Так у тебя появится не абстрактная идея, а проект, который решает твою реальную задачу и может быть полезен в работе.

2. Конфиги: глобальные и проектные настройки

Что такое конфиги и зачем их разделять ↴



Конфиг (от англ. *configuration* — конфигурация) — это **сокращенное название файла, содержащего настройки программы, игры или операционной системы.**

Claude Code — достаточно сложный и гибкий инструмент. У него есть разные типы настроек, которые влияют на поведение агента по-разному в зависимости от уровня, на котором они заданы.

Важно понимать три слоя:

- **Глобальные настройки** — применяются ко всем проектам на компьютере. В какой бы папке мы ни открыли агента, он будет действовать так, как настроено здесь.
- **Проектные настройки** — применяются только к конкретной папке и проекту. Всё, что туда записано, влияет исключительно на агента, запущенного из этой папки.
- **Контекст проекта** — системные инструкции в файле `claude.md`, при помощи которых мы программируем поведение самой модели.

Где находятся глобальные настройки ↴

Глобальные настройки Claude Code хранятся в домашней папке пользователя. Путь зависит от операционной системы:

- **Mac:** `/Users/[имя пользователя]/.claude`
- **Windows:** `C:\Users\[имя пользователя]\.claude`

Или универсально:

- **Mac / Linux:** `~/.claude`
- **Windows:** `%USERPROFILE%\claude`



На Windows папку можно открыть быстро: нажать **Win + R**, вставить %USERPROFILE%\.claude и нажать **Enter**.

На Mac папку можно открыть быстро: нажать Shift + Command + G, вставить ~/.claude и нажать **Enter**.

Чтобы увидеть скрытые папки:

- **Mac:** нажмите **Shift + Command + точка** в Finder.
- **Windows:** откройте Проводник → Вид → Показать → Скрытые элементы.

Внутри папки .claude находятся:

- settings.json — основной файл глобальных настроек. Здесь можно задать язык, глубину размышлений модели (high / medium), разрешения и другие параметры. Руками туда лезть почти не нужно — разве что при углублённой настройке.
- .claude.json — файл, где настраиваются глобальные **MCP-серверы**, тема интерфейса, автообновление и другие параметры программы. Он обновляется автоматически по мере работы с агентом.
- **Папка agents** — место, где настраиваются субагенты.
- **Папка skills** — место, где настраиваются навыки, то есть скиллы агента.
- claude.md — файл с глобальными системными инструкциями. Если он создан, эти инструкции распространяются на все проекты.



Не изменяйте файлы в скрытых папках вручную без необходимости — можно случайно сломать настройки приложений.

Где находятся проектные настройки ↘

Внутри рабочей папки проекта тоже можно создать скрытую папку .claude. Всё, что в ней находится, применяется **только** к агенту, запущенному из этой папки.

Структура аналогична глобальной:

- settings.local.json — проектный аналог глобального settings.json. Форматы идентичны, но файл работает только на уровне конкретного проекта.
- Папки agents и skills — проектные субагенты и скиллы.

Ключевое правило: вложенные настройки перезаписывают внешние. Настройки в папке проекта имеют приоритет над глобальными.

Как решить, куда класть настройку ↘



Простая логика: если настройка полезна в 8 проектах из 10 — скорее всего, она глобальная. Если важна только для одного специфичного проекта — кладем в проектную папку.

Задайте себе несколько вопросов:

- Это правило нужно мне почти везде или только в одном проекте?
- Это настройка поведения инструмента или просто контекст для конкретной задачи?
- Если поменяю это глобально — не сломаются ли другие проекты?
- Нужна ли мне системная настройка или достаточно локального файла рядом с проектом?

3. Разрешения агента: как настроить права

Что такое разрешения и зачем они нужны ↘

Самое важное в файлах `settings.json` и `settings.local.json` — это **permissions**, разрешения агента. Именно они определяют, насколько свободно агент может действовать без нашего подтверждения.

Агент может совершать разные категории действий:

- **Безопасные команды** — например, посмотреть список файлов в директории или прочитать файл. Выполняются без вопросов.

`ls -la`, `read`

- **Bash-команды** — вводятся в терминал. Могут быть безобидными, а могут удалить все папки или выполнить вредоносный скрипт. Важно контролировать.

`git status`, `curl` скрипт | `sh`, `rm -...`

- **Редактирование файлов** (Edit и Patch) — агент открывает и изменяет файлы. Некритично, но потерять данные обидно.
- **Создание файлов** (Write и Create) — запись новых файлов.
- **Браузер и MCP** — использование браузера и подключённых сервисов.
- Прочие.

Три режима работы агента ☝

Переходим в **Claude Code**: запускаем терминал, открываем нужную папку проекта и запускаем Claude Code той же командой, которую использовали ранее.

Переключаться между режимами можно горячими клавишами **Shift + Tab**:

- **Дефолтный режим** — агент спрашивает разрешение перед каждым потенциально опасным действием. Рекомендуется для большинства задач.
- **Accept Edits** — агент может создавать и редактировать файлы без запроса. Активируется, когда мы впервые выбираем «да, разрешить во время этой сессии». Внизу интерфейса появляется пометка *accept edits on*.
- **Plan Mode** — агент полностью лишён возможности совершать действия. Только планирует, задаёт вопросы и описывает шаги. Полезен для проверки логики перед запуском.



Режим **bypass permissions** (когда в настройках прописан `"defaultMode": "bypassPermissions"`) разрешает агенту делать абсолютно всё без единого вопроса, включая удаление файлов. Не рекомендуется для постоянного использования.

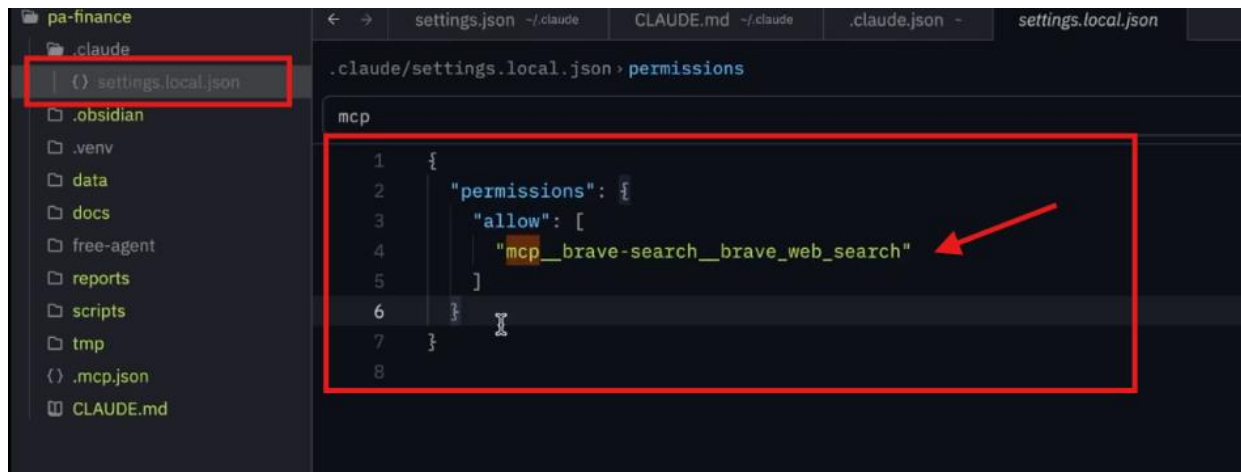
Как постепенно настраивать разрешения ☝

Рекомендуемый подход — **начать с дефолтного режима** и постепенно накапливать разрешения:

1. Работаем в дефолтном режиме — агент спрашивает разрешение на каждое новое действие.
2. Если команда безопасна и мы хотим, чтобы агент делал её всегда — выбираем «Yes, allow all edits during this session (Shift + Tab)» («да, разрешить всегда для этого типа команд»).

```
Do you want to create 2027-finance-forecast-alternative.md?
> 1. Yes
   2. Yes, allow all edits during this session (shift+tab)
   3. No
```

3. Агент записывает это разрешение в файл `settings.local.json` в папке `.claude` нашего проекта.



4. Если в какой-то момент кажется, что агенту выдано слишком много прав — удаляем конкретное разрешение из файла настроек.

Например: когда агент впервые использует git-команды, он спросит разрешения. Если ответить «да, всегда» — в settings.local.json появится запись "git *": "allow". Агент запомнил и больше не спросит.

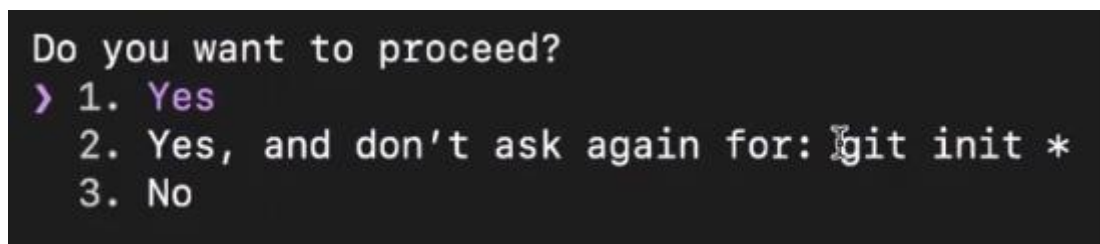


Также можно добавить разрешение на конкретный домен — например, разрешить webfetch только для techcrunch.com. Тогда агент будет ходить туда без вопросов, но на другие домены всё равно будет спрашивать.

Введите:

Иницилируй **Git** — программу для контроля изменений в папке проекта.

Запусти инициализацию Git внутри этого репозитория и пришли мне результат команды git status.



Вложенность настроек: приоритет $\searrow$

1. Скопируйте папку .claude и вставьте её внутрь нужной папки проекта.
2. Внутри папки .claude создайте файл settings.local.json.
3. Измените содержимое файла на:
4. {
5. "permissions": {
6. "defaultMode": "bypassPermissions"
7. }

}

4. Открываем новую сессию в той же папке, где сделали эту настройку, и видим, что агент больше ни о чём не спрашивает и выполняет действия без дополнительных разрешений.

Опасно давать агенту все разрешения: он может случайно удалить важные файлы, а откатить изменения назад получится не всегда.

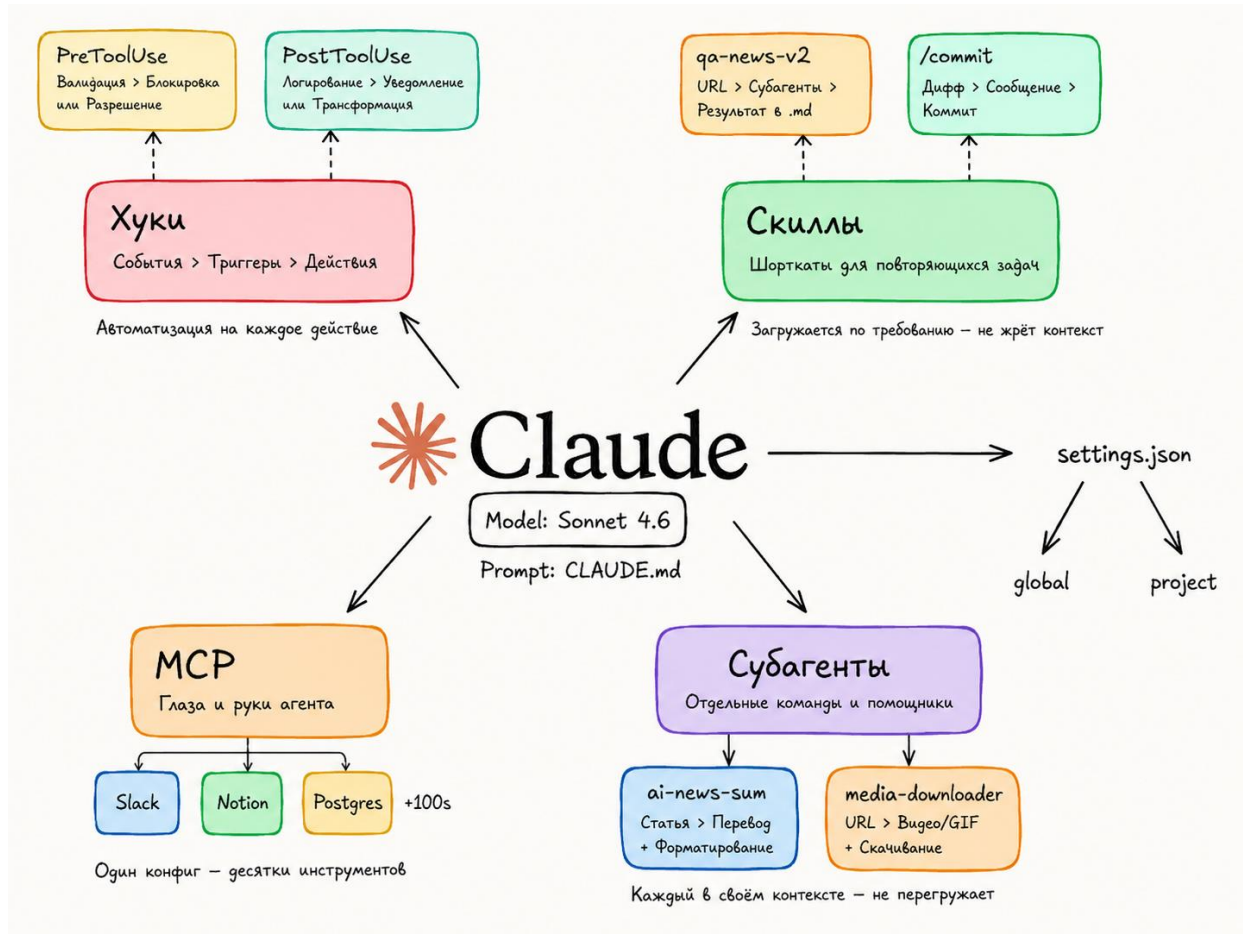
Если внутри проекта есть **вложенная папка** (например, forecasts/) со своим файлом .claude/settings.local.json, то её настройки **перезаписывают** настройки родительской папки.

То есть иерархия такая:

- настройки глобальные (~/.claude/settings.json)
- настройки проекта (.claude/settings.local.json в корне проекта)
- настройки вложенной папки (.claude/settings.local.json внутри подпапки) — **наивысший приоритет**

Это важно учитывать, чтобы случайно не выдать агенту избыточные права в отдельной части проекта.

4. Архитектура Claude Code: навыки, субагенты, MCP и хуки



Общая архитектура: из чего состоит агент ↘

Важно разделять два понятия, которые часто путают:

- **Модель** (например, Claude Sonnet или Opus) — это мозг, нейросеть. Она живёт внутри Claude Code.
- **Claude Code** — это не модель. Это **CLI-агент**, агентный харнес, программа-оболочка, внутри которой живёт модель.

У модели есть **системный промпт** — набор инструкций, который мы кладём в **claude.md**.

У программы Claude Code есть **настройки** — **settings.json** (глобальные) и **settings.local.json** (проектные). Именно там записываются разрешения.

Конфиги (**settings.json**) настраивают программу. Инструкции (**claude.md**) настраивают то, как модель с нами коммуницирует. Это разные слои — важно их не путать.

MCP — глаза и руки агента ↘



MCP (Model Context Protocol) — это протокол, который позволяет связать Claude Code с любым внешним сервисом.

Примеры сервисов, которые можно подключить через MCP:

- **Notion, Slack, Obsidian** — работа с контентом и заметками

- **Postgres** — базы данных
- **Figma** — дизайн
- И вообще **любой сервис** с API

Через MCP агент может:

- **отправлять запросы** в подключённый сервис
- **совершать действия** внутри него
- **получать данные** обратно

Это коммуникация в обе стороны — и чтение, и запись. Именно поэтому MCP называют «**глазами и руками**» агента.

MCP можно настраивать глобально (в `.claude.json`) или проектно (в файле `mcp.json` в папке проекта).

Субагенты — помощники главного агента ↴



До сих пор мы работали в рамках одного агента: открыли чат — работаем. Но Claude Code может **выдавать задачи другим агентам** — это и есть **субагенты**.

Субагенты можно сравнить с отделами компании или помощниками. Это очень полезно, потому что:

- помогает **сохранять контекст** главного агента (не перегружать его)
- позволяет **параллельно** решать разные задачи
- делает систему более управляемой

Субагенты настраиваются в папке `agents` — глобально (`~/.claude/agents/`) или проектно (`.claude/agents/` внутри проекта).

Скиллы — переиспользуемые промпты ↴



Скиллы — это переиспользуемые промпты, шорткаты для повторяющихся задач.

Как это работает: агент подгружает нужный скилл в тот момент, когда понимает, что задача требует определённого контекста, правил или инструкций. Сначала загружает нужный скилл — потом выполняет задачу.

Скиллы настраиваются в папке `skills` — глобально (`~/.claude/skills/`) или проектно (`.claude/skills/` внутри проекта).

Хуки — детерминированные автоматизации ↴



Хуки — это действия, которые происходят автоматически по заданному **триггеру**. Это более сложный архитектурный элемент, который позволяет создавать комплексные автоматизации.

Мы не будем углублённо разбирать хуки в этом курсе, но важно знать, что они существуют.

Где что настраивается: итоговая схема ↴

Элемент	Глобально	Проектно
Настройки программы	<code>~/.claude/settings.json</code>	<code>.claude/settings.local.json</code>
MCP-серверы	<code>~/.claude.json</code>	<code>.mcp.json</code> в корне проекта

Элемент

Глобально

Проектно

Субагенты	~/claude/agents/	.claude/agents/
Скиллы	~/claude/skills/	.claude/skills/
Системные инструкции	~/claude/claude.md	claude.md в папке проекта



Всё, что лежит в проектной папке .claude, работает **только** для агента, запущенного из этого проекта. Это главный принцип проектной изоляции.

5. Итоги урока

Что мы разобрали сегодня ☹

Сегодняшний урок был насыщенным — мы разобрали несколько важных архитектурных слоёв Claude Code:

Выбор проекта. Зафиксировали направление, в котором будем развивать агента до конца курса. Без цели сложно понять, какие настройки и инструменты использовать.

Конфиги. Разобрались, что есть **глобальные** и **проектные** конфиги, и поняли принципиальное отличие между ними.

Конфиги vs инструкции. Это разные слои:

- settings.json — настраивает **программу** Claude Code
- claude.md— настраивает **модель**, то есть то, как она с нами общается

Разрешения. Научились управлять степенью свободы агента: дефолтный режим, Accept Edits, Plan Mode, bypass permissions.

Архитектура. Разобрали, что такое **МСП, субагенты, скиллы и хуки** — и как всё это настраивается глобально и проектно.

В следующих уроках подробно разберём МСП, субагентов и скиллы — с конкретными примерами и настройками.

Дополнительные материалы

Словарь занятия ☹

- **CLI-агент** — программа-оболочка командной строки (**Command Line Interface**), внутри которой работает языковая модель. Claude Code — это CLI-агент.
- **Агентный харнес** — другое название CLI-агента; программа, которая обеспечивает инфраструктуру для работы модели.
- **Конфиг / config** — файл настроек программы. В Claude Code основные конфиги: settings.json и settings.local.json.
- settings.json — глобальный файл настроек Claude Code. Хранится в папке ~/claude/.
- settings.local.json — проектный файл настроек Claude Code. Хранится в папке .claude/ внутри проекта.
- .mcp.json — файл для подключения МСП-серверов на уровне проекта.
- claude.md — файл системных инструкций для модели. Определяет, как агент общается, отвечает и работает с проектом.
- **Permissions / разрешения** — настройки, которые определяют, какие действия агент может выполнять без дополнительного подтверждения пользователя.

- **Bypass permissions** — режим, при котором агент может выполнять любые действия без подтверждения пользователя. Не рекомендуется для постоянного использования.
- **Accept Edits** — режим, при котором агент может создавать и редактировать файлы без дополнительного запроса.
- **Plan Mode** — режим, при котором агент не выполняет действия, а только планирует и отвечает текстом.
- **MCP / Model Context Protocol** — протокол, который позволяет связать Claude Code с внешними сервисами: Notion, Slack, Figma, базами данных и другими инструментами.
- **MCP-сервер** — конкретная реализация MCP для отдельного сервиса. Например, MCP-сервер для поиска, Notion или базы данных.

Домашнее задание

Предлагаем два варианта заданий. Первый поможет развить ключевое умение по теме занятия, а второй даст свободу для **проектирования алгоритма для упрощения или даже автоматизации одной из твоих рутинных рабочих задач**.



Выбери только один вариант домашнего задания и выполни его сегодня.

Другой вариант, если захочешь, можешь проработать позже самостоятельно для закрепления материала.

Вариант 1 (Базовый — повтори за экспертом) ☞

Задача: настроить проектного CLI-агента с базовой структурой, инструкциями и MCP-подключением.

Что нужно сделать:

1. Создай папку проекта, например research-agent.
2. Открой её в редакторе кода.
3. Создай файл проектных настроек:

.claude/settings.local.json

4. Добавь базовую структуру разрешений:

```
5. {  
6.   "permissions": {  
7.     "allow": []  
8.   }  
}
```

5. Добавь в claude.md:

- роль агента;
- правила безопасности;
- правило нейминга файлов.

6. Запусти Claude Code из папки проекта и попроси агента:

- проверить структуру проекта;
- объяснить, где лежат настройки;
- создать тестовый отчёт в папке reports/.

Ожидаемый результат: готовая папка проекта с настройками, инструкциями, MCP-файлом и тестовым отчётом.

Что приложить:

1. Скриншот структуры папок.
2. Скриншот .claude/settings.local.json.
3. Скриншот claude.md.
4. Скриншот тестового отчёта из reports/.

Вариант 2 (Продвинутый — адаптируй под свою задачу) ☹

Задача: спроектировать CLI-агента под реальную рабочую задачу.

Что нужно сделать:

1. Выбери задачу для агента.

Например:

- исследование рынка;
- подготовка контента;
- работа с документами;
- анализ данных;
- ведение базы знаний;
- личная продуктивность.

2. Создай папку проекта с понятным названием.

Например:

content-agent

market-research-agent

project-knowledge-base

3. Создай файл:

.claude/settings.local.json

4. Добавь базовую структуру разрешений:

5. {
6. "permissions": {
7. "allow": []
8. }

}

5. Опиши в claude.md:

- роль агента;
- какие задачи он решает;
- структуру папок;
- правило нейминга;
- правила работы с источниками;
- правила безопасности.

6. Запусти Claude Code и попроси агента:
 - проверить архитектуру проекта;
 - предложить улучшения;
 - выполнить мини-задачу по твоему направлению;
 - сохранить результат в reports/.

Ожидаемый результат: настроенный проектный агент под твою рабочую задачу и первый тестовый результат.

Что приложить:

1. Скриншот структуры проекта.
2. Скриншот .claude/settings.local.json.
3. Скриншот claude.md.
4. Скриншот результата из reports/.
5. Короткий комментарий: какую задачу выбрал(а) и чем агент может помочь в работе.

Анкета обратной связи по уроку

Эл. адрес

Курс*

Модуль*

Урок*

Понравился ли урок?*

Понравился, все было понятноНеплохо, но не все получилось/не все понятноНе понравился

Обратная связь по уроку (не обязательно)

Задание

Предлагаем два варианта заданий. Первый поможет развить ключевое умение по теме занятия, а второй даст свободу для **проектирования алгоритма для упрощения или даже автоматизации одной из твоих рутинных рабочих задач.**



Выбери только один вариант домашнего задания и выполни его сегодня.

Другой вариант, если захочешь, можешь проработать позже самостоятельно для закрепления материала.

Вариант 1 (Базовый — повтори за экспертом) ▾

Задача: настроить проектного CLI-агента с базовой структурой, инструкциями и MCP-подключением.

Что нужно сделать:

1. Создай папку проекта, например research-agent.
2. Открой её в редакторе кода.
3. Создай файл проектных настроек:

.claude/settings.local.json

4. Добавь базовую структуру разрешений:
5. {

- ```
6. "permissions": {
7. "allow": []
8. }
}
```
5. Добавь в `claude.md`:
- роль агента;
  - правила безопасности;
  - правило нейминга файлов.
6. Запусти Claude Code из папки проекта и попроси агента:
- проверить структуру проекта;
  - объяснить, где лежат настройки;
  - создать тестовый отчёт в папке `reports/`.

**Ожидаемый результат:** готовая папка проекта с настройками, инструкциями, MCP-файлом и тестовым отчётом.

**Что приложить:**

1. Скриншот структуры папок.
2. Скриншот `.claude/settings.local.json`.
3. Скриншот `claude.md`.
4. Скриншот тестового отчёта из `reports/`.

Вариант 2 (Продвинутый — адаптируй под свою задачу) ☞

**Задача:** спроектировать CLI-агента под реальную рабочую задачу.

**Что нужно сделать:**

1. Выбери задачу для агента.

Например:

- исследование рынка;
- подготовка контента;
- работа с документами;
- анализ данных;
- ведение базы знаний;
- личная продуктивность.

2. Создай папку проекта с понятным названием.

Например:

`content-agent`

`market-research-agent`

`project-knowledge-base`

3. Создай файл:

`.claude/settings.local.json`

```
4. Добавь базовую структуру разрешений:
5. {
6. "permissions": {
7. "allow": []
8. }
9. }
```

5. Опиши в `claude.md`:

- роль агента;
- какие задачи он решает;
- структуру папок;
- правило нейминга;
- правила работы с источниками;
- правила безопасности.

6. Запусти Claude Code и попроси агента:

- проверить архитектуру проекта;
- предложить улучшения;
- выполнить мини-задачу по твоему направлению;
- сохранить результат в `reports/`.

**Ожидаемый результат:** настроенный проектный агент под твою рабочую задачу и первый тестовый результат.

**Что приложить:**

1. Скриншот структуры проекта.
2. Скриншот `.claude/settings.local.json`.
3. Скриншот `claude.md`.
4. Скриншот результата из `reports/`.
5. Короткий комментарий: какую задачу выбрал(а) и чем агент может помочь в работе.

[Список тренингов](#)[Практический курс по вайб-кодингу на Claude Code](#)[Модуль 2: Сайты, автоматизация и усиление возможностей Claude Code](#)

CLs02 Создание конверсионных сайтов с помощью Claude Code

[Предыдущий урок](#)

Есть задание

[Следующий урок](#)

Дорогой студент! Интерфейсы современных приложений могут очень быстро меняться. Если вдруг то, что видишь ты, отличается от того, что демонстрирует эксперт, настолько, что тебе непонятно, куда нажимать — напиши об этом в учебный чат, указав номер урока и видео. Наш куратор обязательно поможет тебе разобраться, а мы постараемся как можно скорее актуализировать материал :)



Как проходить уроки эффективно: доступы, оплата и лимиты

**1** Доступ к мировым технологиям. Мы показываем лучшие мировые нейросети, но доступ к некоторым из них из РФ и/или других стран может требовать средств защищенного доступа. Мы действуем в рамках закона и НЕ

консультируем по настройке соединения, но показываем эти сервисы, потому что они — лидеры рынка. Разобраться с доступом к ним — значит получить конкурентное преимущество.

2. Весь курс можно пройти на бесплатных версиях инструментов. Мы показываем платные PRO-функции для ознакомления, чтобы ты мог(ла) оценить их потенциал и осознанно решить, нужны ли они тебе в будущем.
3. Умное управление лимитами. Бесплатные попытки ограничены, поэтому регистрируйся в сервисе только перед выполнением ДЗ. Если лимиты закончились, а попрактиковаться еще хочется — часто помогает регистрация нового аккаунта на другую почту.
4. Сначала смотри, потом делай. Сначала посмотри урок полностью, чтобы понять логику, и только потом трать генерации. Это сэкономит твоё время и попытки.

Так ты освоишь самые мощные инструменты бесплатно и без лишних нервов! 🚀

---

## Результат дня

- ✓ Поймёшь, из каких секций состоит конверсионный сайт-лендинг
- ✓ Научишься создавать технический документ (PRD/SPEC) для агента в Claude Code
- ✓ Создашь готовую веб-страницу с нуля при помощи Claude Code
- ✓ Освоишь итерационный подход: как дорабатывать сайт через скриншоты и текстовые правки
- ✓ Поймёшь разницу между локальной и удалённой (продакшн) версией сайта

▶ Время чтения и просмотра: ~ 60 минут

🕒 Время выполнения задания: ~50 минут



Под каждым видео в уроке есть текстовая расшифровка, чтобы у тебя была возможность лучше запомнить новые термины, прочитать пояснения и инструкции.

Чтобы прочитать текст, нажимай на него. Текст скрыт в строках подчёркнутых пунктирной линией со значком "⌵" в конце строки.

---

## Теория на сегодня

### 1. Из чего состоит конверсионный сайт



Пример сайтов, который можно сделать с помощью Claude Code:

<https://yoghermeisters.com/>

<https://prodadvice.com/>

Что такое фронтенд и зачем он нужен ⌵



**Фронтенд** — это всё, что пользователь видит на экране: сайт, веб-страница, интерфейс на ноутбуке или телефоне. В отличие от **бэкенда** — логики, которая обрабатывается на серверах в облаке, — фронтенд является визуальным слоем продукта.

Сегодня мы сфокусируемся именно на фронтенде: создадим готовую рабочую страницу, которую можно показать клиентам, заказчикам или коллегам.

Наша задача — не дизайнерский шедевр, а понятная, удобная, осмысленная страница, которая работает на нас.

Навык создания фронтендов при помощи **Claude Code** полезен практически всем, кто хочет присутствовать в диджитал-среде и презентовать свои услуги.

Примеры сайтов, созданных в Claude Code ↴

**Claude Code** позволяет создавать полноценные сайты с анимацией, встроенным видео, несколькими вложенными страницами и даже с базой данных. Примеры того, что реально сделать:

- Сайт проекта с кастомными анимациями и визуальными элементами
- Лендинг для записи на ретрит с **Hero-секцией**, видео-отзывами, фотографиями и кнопками СТА



Большую часть времени при создании таких сайтов занимает не написание кода, а подготовка визуальных элементов — картинок, иллюстраций, фото.

Структура конверсионной страницы ↴

Стандартная структура лендинга, с которой мы работаем:

- **Hero-секция** — первый экран, главный заголовок, попадание в боль целевой аудитории
- **Польза** — несколько секций с объяснением, почему это важно для клиента
- **Доверие** — почему стоит доверять: кейсы, отзывы, логотипы партнёров
- **СТА (Call to Action)** — призыв к действию: форма заявки, кнопка покупки
- **Контакты / Футер** — служебная информация, ссылки, контакты

Начинаем с Hero → польза → доверие → продажа → контакты.



Структура может быть индивидуальной, но логика остаётся неизменной: сначала зацепить, потом убедить, затем продать.

## 2. Создание PRD и первого сайта в Claude Code

Что такое PRD и почему простого промпта недостаточно ↴

Просто написать агенту «сделай красивый сайт» или «сделай сайт-визитку для фрилансера» — недостаточно для предсказуемого и качественного результата.

Мы используем более зрелый подход: создаём **PRD (Product Requirements Document)** — технический документ, по которому агент идёт и создаёт страницу. Его также называют **SPEC.md**.

Это документ, в котором прописаны:

- цель сайта и целевая аудитория
- структура секций
- технический стек
- дизайн и стилистика
- файловая структура
- текстовые заглушки для каждой секции



Вместо того чтобы всё придумывать самому, предлагаем агенту **побрейнштормить вместе** и составить этот документ в режиме **Plan Mode**.

Работа в Plan Mode: составляем техническое задание ↴

Алгоритм работы:

1. Создаём новую папку для проекта (например, finance-card) и запускаем **Claude Code** из терминала
2. В **Plan Mode** пишем агенту свободный запрос: описываем, какой сайт хотим, для кого, какие секции должны быть
3. Агент задаёт уточняющие вопросы и предлагает варианты — отвечаем на них прямо в интерфейсе

Агент предложит выбрать **технический стек**. Варианты от простого к сложному:

- **Vanilla HTML + CSS + JavaScript** — самый простой, рекомендуется для начала
- **Vanilla JS + Tailwind CDN** — чуть более профессиональная стилизация
- **Next.js / React** — для более сложных проектов



Если вы не очень разбираетесь в технологиях — выбирайте самый простой вариант. В нём меньше всего мест, где что-то может пойти не так.

Агент также спросит про дизайн. Можно попросить его предложить варианты:

«Предложи мне интересные варианты дизайна — минимум 7 различных вариантов, которые могут подойти под мой проект»

Примеры вариантов, которые может предложить агент: *классический минимализм, тёмный AI-стартап, корпоративный голубой, брутализм швейцарский, хайтек моноспейс* и другие.

Итерация плана и запуск генерации ↴

После того как агент составил план, его нужно **обязательно прочитать** перед тем, как давать разрешение на выполнение.



Если вы не прочитаете план и не согласуете его, агент может начать делать то, что вам не подходит — и вы потратите время впустую.

Что можно скорректировать на этапе плана:

- выбрать конкретный стиль дизайна
- указать, что нужна фотография в Hero-секции (добавьте файл в папку проекта)
- уточнить, что хостинг пока не нужен — разрабатываем локально

После согласования включаем режим **Accept Edits** — агент пишет код без остановок. Внутри Claude Code есть инструмент **Task Tool**: агент сам составляет список задач (написать index.html, style.css, main.js) и выполняет их по порядку.

Локальный сервер: как открыть сайт в браузере ↴

Когда агент закончит писать код, он запустит **локальный сервер**. Это значит, что ваш компьютер одновременно является и сервером (запускает сайт), и клиентом (открывает страницу в браузере).

Адрес сайта выглядит так: `http://localhost:8080`

Чтобы открыть сайт — скопируйте эту ссылку и вставьте в браузер.

Если сервер не запущен — сайт не откроется. Чтобы запустить сервер вручную:

```
cd [папка-проекта]
```

```
python3 -m http.server 8080
```



Попросите агента запустить сервер — он это умеет делать автоматически. Но полезно знать команду и для самостоятельного запуска.

### 3. Итерации и доработка сайта

Как дорабатывать сайт через итерации ↴

Если результат не нравится — **не нужно начинать заново**. Модели обладают большим контекстным окном и способны продолжить работу с того места, где остановились.

Алгоритм доработки:

1. Смотрим на сайт и находим, что нужно изменить
2. Делаем **скриншот** нужной секции
3. Открываем агента в терминале и закидываем скриншот
4. Описываем, что именно хотим изменить

Пример запроса:

> «Измени, пожалуйста, эту секцию в футере таким образом, чтобы все мои контакты шли не в столбик, а в одну строчку в десктопной версии»

После подтверждения изменений — просто обновляем страницу в браузере.



Большинство агентов сейчас умеют читать картинки и понимать, что на них изображено. Это удобно: можно показать скриншот и объяснить правки визуально.



Уточняйте, для какой версии вносите правки — **десктопной** или **мобильной**. Агент адаптирует сайт под оба варианта, и правки в одном могут не затронуть другой.

### 4. Разработка сайта на основе визуального референса. Скиллы

Создание сайта по визуальному референсу ↴

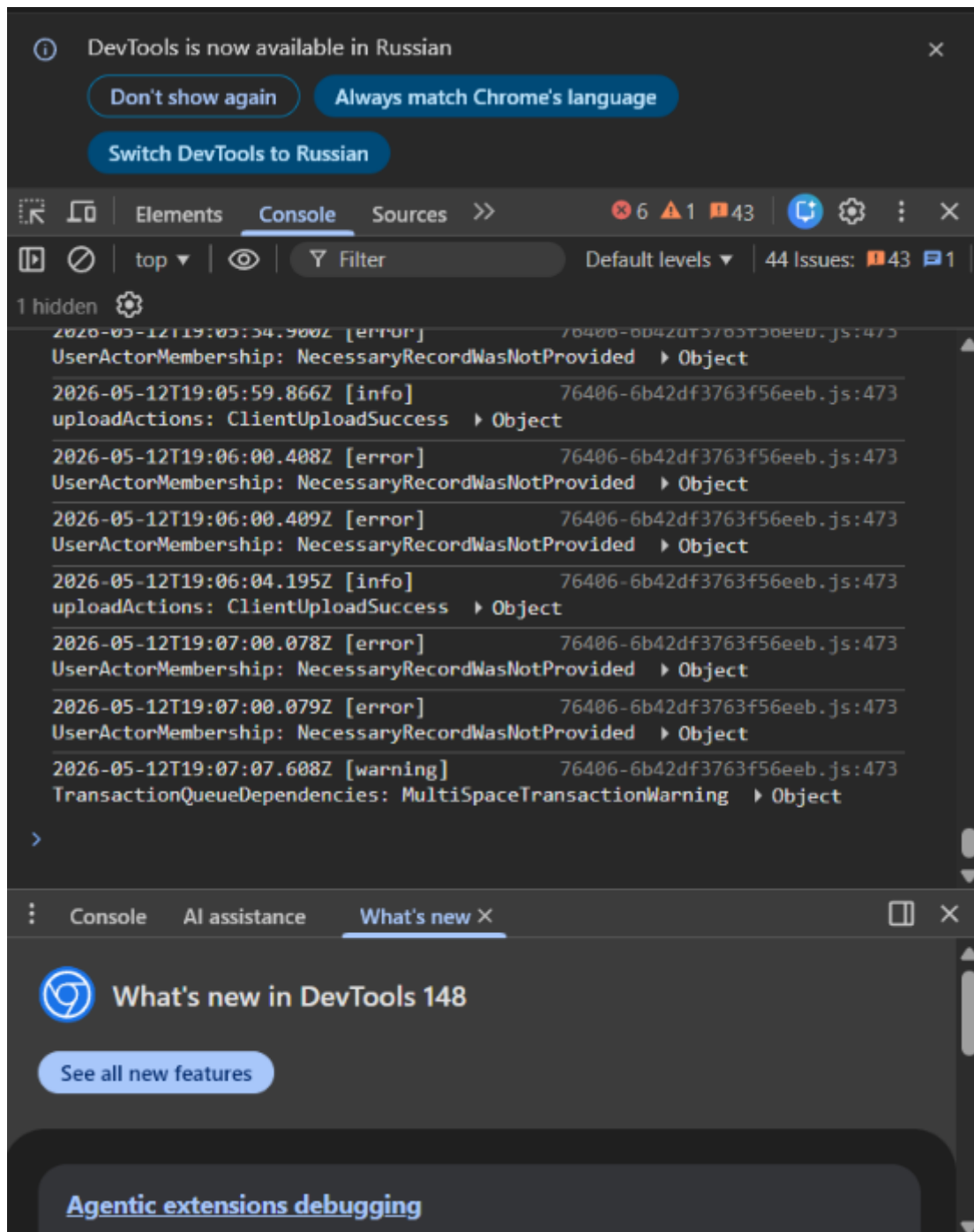
Второй подход — создание сайта на основе **референс-картинки**. Это удобно, когда есть пример стиля, который нравится.



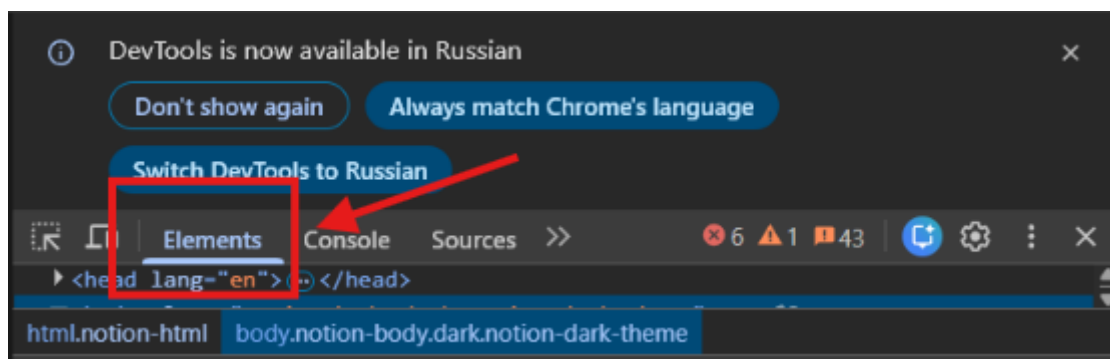
Пример сайта из урока: <https://take-your-time.ru/>

Как получить скриншот всего сайта через DevTools ↴

1. Открываем панель разработчика:
  - на Mac — Command + Option + I;
  - на Windows — Ctrl + Shift + I или F12.

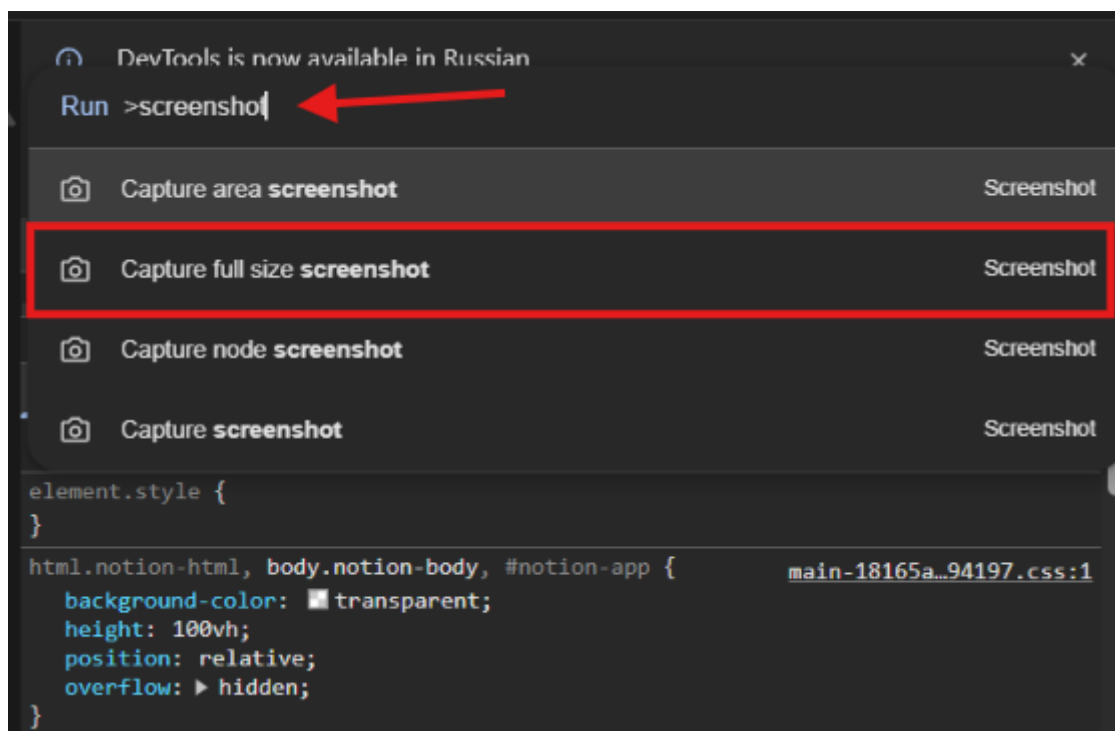


2. Переходим в раздел **Elements**.



3. Нажимаем Command + Shift + P и вводим команду screenshot.

4. Выбираем Capture full-size screenshot.



После этого браузер сохранит изображение всей страницы сайта.

5. Полученный скриншот кладем в папку, в которой создаем сайт.
6. Кладем в папку с проектом картинки, соответствующие тематике сайта (можно сгенерировать их с помощью нейросети).







Прежде чем начать генерацию, мы собираем **ассеты** — исходные материалы проекта: скриншоты, изображения, тексты, логотипы, цвета и другие файлы, которые агент использует как референсы для работы.

Использование скиллов в Claude Code 

У нас будет отдельный урок, посвящённый скилам, а сейчас мы делаем первое касание с этим понятием.



**Скиллы** — это дополнительные инструкции для агента, которые загружаются в его контекст перед выполнением задачи. Они делают агента более профессиональным в конкретной области.

1. Создаем внутри нашего проекта папку `.claude`.
2. Внутри этой папки переносим файлы со скиллом:

[SKILL.md](#)

Правильная структура файлов для скилла:

[папка-проекта]/  
└─ .claude/  
└─ skills/  
└─ frontend-skill/  
└─ skill.md



Важна именно такая иерархия: .claude → skills → [название скилла] → skill.md. Без промежуточной папки skills скилл не будет найден.

После добавления скилла — перезапустите Claude Code, чтобы он подтянул новые настройки. Проверить, что скилл подключился, можно командой /skills.

Если агент не использует скилл автоматически — явно попросите его:  
«Используй скилл frontend-skill»



Файл skill.md для фронтенд-разработки прикреплён к уроку — скачай и положи в нужную папку.

🔗 Пример промпта:

Привет, создай вебсайт body&mind.

Драфт текста страницы:

(вставь сюда свой текст)

Возьми за референс стиль и лейаут с сайта Take your time. Скриншот сайта лежит в рабочей директории.

Используй в хиро-секции одно из изображений, которые ты найдёшь в папке проекта.

5. Локальная версия vs. продакшн: как разместить сайт в интернете

Разница между локальной и удалённой версией ↗



Сегодня наша главная задача — научиться создавать сайт. Публикацию на виртуальный сервер мы будем разбирать в следующих уроках.

Сайт, созданный в Claude Code, изначально работает **только локально** — на вашем компьютере. Другие люди не смогут открыть его по ссылке.

Чтобы сайт стал доступен всем в интернете, его нужно **задеплоить** — то есть разместить на хостинге.

Вариант публикации зависит от сложности сайта:

| Тип сайта                  | Что нужно сделать                                                   |
|----------------------------|---------------------------------------------------------------------|
| Чистый HTML / CSS / JS     | Перетащить файлы на хостинг: например, Netlify, GitHub Pages, TimeW |
| Фреймворк: Next.js / React | Собрать проект одной командой, а затем загрузить готовую сборку на  |
| Сайт с базой данных        | Настроить серверную логику, базу данных и полноценный хостинг.      |

Чем проще сайт — тем проще его публиковать. Лендинг на чистом HTML — это один-три файла: взяли, перекинули, заработало.



Подробно про деплой — домены, хостинги, настройку — мы разберём в финальных уроках курса.

## 6. Итоги урока

Что мы разобрали сегодня ☞

На этом уроке мы:

- Разобрали структуру конверсионного сайта: **Hero → Польза → Доверие → CTA → Контакты**
- Создали **PRD/SPEC** — технический документ для агента — в режиме **Plan Mode**
- Написали готовую веб-страницу при помощи **Claude Code**
- Попробовали два подхода: через текстовый промпт и через **визуальный референс**
- Освоили **итерационный подход**: как дорабатывать сайт через скриншоты и текстовые правки
- Подключили **скилл для фронтенд-разработки** в настройки проекта
- Поняли разницу между **локальной** и **удалённой (продакшн)** версиями сайта

## Дополнительные материалы

Ссылки на сервисы из урока ☞

- **Claude Code** — агентная среда для разработки от Anthropic
- **Netlify** — хостинг для статических сайтов, [netlify.com](https://netlify.com)
- **GitHub Pages** — бесплатный хостинг для HTML/CSS/JS сайтов, [pages.github.com](https://pages.github.com)
- **TimeWeb** — российский хостинг-провайдер, [timeweb.com](https://timeweb.com)
- **Tailwind CSS CDN** — утилитарный CSS-фреймворк для быстрой стилизации, [tailwindcss.com](https://tailwindcss.com)

Словарь занятия ☞

- **Фронтенд** — визуальная часть сайта, всё, что пользователь видит на экране
- **Бэкенд** — серверная логика, обработка данных на сервере; скрыта от пользователя
- **PRD (Product Requirements Document)** — технический документ с описанием требований к продукту, по которому агент разрабатывает сайт
- **SPEC.md** — файл со спецификацией проекта, аналог PRD
- **Hero-секция** — первый экран сайта с главным заголовком и призывом к действию
- **CTA (Call to Action)** — призыв к действию: кнопка, форма заявки, ссылка
- **Футер** — нижняя часть страницы со служебной информацией и контактами
- **Технический стек (tech stack)** — набор технологий, на которых написан сайт
- **Vanilla HTML/CSS/JS** — базовые веб-технологии без фреймворков
- **Локальный сервер** — сервер, запущенный на вашем компьютере; сайт доступен только вам
- **localhost** — адрес локального сервера в браузере (например, `http://localhost:8080`)
- **Деплой** — размещение сайта на внешнем сервере, чтобы он стал доступен в интернете
- **Plan Mode** — режим планирования в Claude Code, в котором агент задаёт вопросы и составляет ТЗ перед выполнением задачи
- **Task Tool** — инструмент внутри Claude Code для составления списка задач перед выполнением
- **Скилл (skill)** — файл с дополнительными инструкциями для агента, расширяющий его компетенции в конкретной области

- **Итерация** — последовательное улучшение результата через серию правок
- **Дебаг** — исправление ошибок в коде или в отображении элементов
- **Адаптив (responsive design)** — адаптация сайта под разные размеры экранов (десктоп, мобильный)
- **DevTools (панель разработчика)** — встроенный инструмент браузера для анализа и отладки сайтов

#### Домашнее задание

Предлагаем два варианта заданий. Первый поможет развить ключевое умение по теме занятия, а второй даст свободу для **проектирования алгоритма для упрощения или даже автоматизации одной из твоих рутинных рабочих задач.**



Выбери только один вариант домашнего задания и выполни его сегодня.

Другой вариант, если захочешь, можешь проработать позже самостоятельно для закрепления материала.

Вариант 1 (Базовый — повтори за экспертом) ▾

Создай лендинг-визитку для себя или выдуманного персонажа в **Claude Code**.

#### Что сделать:

1. Создай новую папку для проекта и запусти Claude Code
2. В **Plan Mode** опиши агенту: кто ты, для кого сайт, какие услуги предлагаешь
3. Попроси агента составить **PRD** с пятью секциями: Hero, Польза, Доверие, СТА, Контакты
4. Согласуй план, выбери стиль дизайна из предложенных вариантов
5. Запусти генерацию, открой сайт через `localhost`
6. Внеси минимум **одну итерационную правку**: измени любую секцию через скриншот или текстовое описание

**Результат:** готовая локальная страница, которую можно открыть в браузере.

**Сделай скриншот своего сайта и прикрепи его в поле для сдачи домашнего задания как изображение.**

*На скриншоте должно быть видно, как выглядит готовая страница сайта.*

Вариант 2 (Продвинутый — адаптируй под свою задачу) ▾

Создай лендинг для реального проекта, продукта или услуги — своей или клиентской.

#### Что сделать:

1. Подбери **визуальный референс**: найди сайт в нужном стиле, сделай его полный скриншот через DevTools (`Capture full-size screenshot`)
2. Подготовь **ассеты**: фотографии, логотип, реальные тексты для секций
3. Добавь в проект **скилл для фронтенда** (файл `skill.md` из материалов урока)
4. Запусти агента, передай ему референс и ассеты, опиши задачу
5. Доработай результат через итерации до состояния, которое не стыдно показать клиенту

**Результат:** локальная версия сайта, готовая к дальнейшему деплою.

**Сделай скриншот своего сайта и прикрепи его в поле для сдачи домашнего задания как изображение.**

*На скриншоте должно быть видно, как выглядит готовая страница сайта.*

Анкета обратной связи по уроку

Эл. адрес

Курс\*

Модуль\*

Урок\*

Понравился ли урок?\*

Понравился, все было понятноНеплохо, но не все получилось/не все понятноНе понравился

Обратная связь по уроку (не обязательно)

### Задание

Предлагаем два варианта заданий. Первый поможет развить ключевое умение по теме занятия, а второй даст свободу для **проектирования алгоритма для упрощения или даже автоматизации одной из твоих рутинных рабочих задач**.



Выбери только один вариант домашнего задания и выполни его сегодня.

Другой вариант, если захочешь, можешь проработать позже самостоятельно для закрепления материала.

Вариант 1 (Базовый — повтори за экспертом) ☑

Создай лендинг-визитку для себя или выдуманного персонажа в **Claude Code**.

#### Что сделать:

1. Создай новую папку для проекта и запусти Claude Code
2. В **Plan Mode** опиши агенту: кто ты, для кого сайт, какие услуги предлагаешь
3. Попроси агента составить **PRD** с пятью секциями: Hero, Польза, Доверие, СТА, Контакты
4. Согласуй план, выбери стиль дизайна из предложенных вариантов
5. Запусти генерацию, открой сайт через `localhost`
6. Внеси минимум **одну итерационную правку**: измени любую секцию через скриншот или текстовое описание

**Результат:** готовая локальная страница, которую можно открыть в браузере.

**Сделай скриншот своего сайта и прикрепи его в поле для сдачи домашнего задания как изображение.**

*На скриншоте должно быть видно, как выглядит готовая страница сайта.*

Вариант 2 (Продвинутый — адаптируй под свою задачу) ☑

Создай лендинг для реального проекта, продукта или услуги — своей или клиентской.

#### Что сделать:

1. Подбери **визуальный референс**: найди сайт в нужном стиле, сделай его полный скриншот через DevTools (`Capture full-size screenshot`)
2. Подготовь **ассеты**: фотографии, логотип, реальные тексты для секций
3. Добавь в проект **скилл для фронтенда** (файл `skill.md` из материалов урока)
4. Запусти агента, передай ему референс и ассеты, опиши задачу
5. Доработай результат через итерации до состояния, которое не стыдно показать клиенту

**Результат:** локальная версия сайта, готовая к дальнейшему деплою.

**Сделай скриншот своего сайта и прикрепи его в поле для сдачи домашнего задания как изображение.**

*На скриншоте должно быть видно, как выглядит готовая страница сайта.*

CLs03 Силлы (Skills) в Claude Code: как усилить ИИ-агента под свои задачи

[Предыдущий урок](#)

Есть задание

[Следующий урок](#)

Дорогой студент! Интерфейсы современных приложений могут очень быстро меняться. Если вдруг то, что видишь ты, отличается от того, что демонстрирует эксперт, настолько, что тебе непонятно, куда нажимать — напиши об этом в учебный чат, указав номер урока и видео. Наш куратор обязательно поможет тебе разобраться, а мы постараемся как можно скорее актуализировать материал :)



Как проходить уроки эффективно: доступы, оплата и лимиты

**1** Доступ к мировым технологиям. Мы показываем лучшие мировые нейросети, но доступ к некоторым из них из РФ и/или других стран может требовать средств защищенного доступа. Мы действуем в рамках закона и НЕ консультируем по настройке соединения, но показываем эти сервисы, потому что они — лидеры рынка. Разобраться с доступом к ним — значит получить конкурентное преимущество.

**2** Весь курс можно пройти на бесплатных версиях инструментов. Мы показываем платные PRO-функции для ознакомления, чтобы ты мог(ла) оценить их потенциал и осознанно решить, нужны ли они тебе в будущем.

**3** Умное управление лимитами. Бесплатные попытки ограничены, поэтому регистрируйся в сервисе только перед выполнением ДЗ. Если лимиты закончились, а попрактиковаться еще хочется — часто помогает регистрация нового аккаунта на другую почту.

**4** Сначала смотри, потом делай. Сначала посмотри урок полностью, чтобы понять логику, и только потом трать генерации. Это сэкономит твоё время и попытки.

Так ты освоишь самые мощные инструменты бесплатно и без лишних нервов! 🚀

---

Результат дня

- ✅ Поймём, почему одного хорошего промпта недостаточно для регулярной работы
- ✅ Узнаем, что такое силлы (Skills) и как они экономят время на повторяющихся задачах
- ✅ Разберём архитектуру скиллов: сессионный, проектный, глобальный и встроенный уровни
- ✅ Создадим свой первый скилл и протестируем его в Claude Code
- ✅ Научимся использовать JSON-промптинг для структурированных результатов
- ✅ Увидим примеры комплексных скиллов со скриптами, API и референсами
- ✅ Узнаем, где искать готовые силлы и как безопасно их устанавливать

▶ Время чтения и просмотра: ~ 60 минут

🕒 Время выполнения задания: ~50 минут



Под каждым видео в уроке есть текстовая расшифровка, чтобы у тебя была возможность лучше запомнить новые термины, прочитать пояснения и инструкции.

Чтобы прочитать текст, нажимай на него. Текст скрыт в строках подчёркнутых пунктирной линией со значком "⌵" в конце строки.

---

Теория на сегодня

## 1. Почему одного промпта недостаточно для регулярной работы

Что такое скилл и зачем он нужен 📄

В реальной работе есть повторяющиеся задачи: контент-мейкер каждую неделю собирает выпуски новостей, продакт-менеджер проверяет документацию, аналитик регулярно собирает сводки, преподаватель готовит уроки по одной структуре. Каждый раз писать агенту новый промпт и объяснять, как работать — неэффективно и, честно говоря, лень.



Скилл (Skill) — это сохранённая рабочая методика выполнения задачи. Это пакет инструкций для агента: когда применять навык, какие шаги пройти, какие файлы открыть, какие скрипты использовать.

В самом упрощённом формате скилл — это один Markdown-файл (skill.md), который лежит в нужной папке и который агент видит и может использовать.

Когда скилл нужен:

- Задача повторяется регулярно (каждую неделю/месяц)
- У задачи есть понятные правила и структура
- Вы замечаете, что на этом можно экономить время

Примеры задач под скиллы:

- Написание постов для разных соцсетей в нужном формате
- Проверка документа/спецификации по чек-листу
- Сбор и форматирование регулярного отчёта
- Создание урока по заданной структуре
- Написание коммитов, пушей, git-операций
- Обработка и категоризация входящих данных

Архитектура скиллов: 4 уровня размещения 📄

В Claude Code существует несколько уровней размещения скиллов:

1. **Сессионный уровень** — вы даёте агенту инструкцию прямо в чате («начиная отвечать, делай так-то»). Это ещё не полноценный скилл, а просто промпт-шаблон для текущей сессии.
2. **Проектный уровень** — внутри папки проекта создаётся директория `.claude/skills/`, в которую добавляются скиллы. Работает только в этом проекте.
3. **Глобальный (пользовательский) уровень** — скиллы кладутся в домашнюю корневую директорию `~/claude/skills/`. Доступны в Claude Code из любой папки на компьютере.
4. **Встроенный уровень** — скиллы, которые идут вместе с плагинами или установлены по умолчанию разработчиками Claude Code.



Рекомендуем начинать с проектного уровня: создаёте скилл внутри рабочей папки, тестируете, а затем при необходимости переносите на глобальный уровень.

Простые vs Комплексные скиллы 📄

Скиллы можно разделить на два типа:

### Простой скилл

- Один-единственный файл `skill.md` с промптом
- Достаточно для: постов, писем, отчётов, проверки по шаблону

- Это просто сохранённый промпт в нужной папке

### Комплексный скилл

- Содержит: скрипты (Python, bash), шаблоны, вызовы к разным инструментам
- Может использовать: MCP-серверы, субагентов, внешние API
- Нужен, когда: более сложная логика, агент должен использовать инструменты, проверять результат, вести себя проактивно

Простого скилла хватит для большинства повседневных задач. Комплексные нужны, когда мы хотим, чтобы агент не просто форматировал текст, а совершал действия: запускал скрипты, отправлял запросы, сохранял файлы.

## 2. Создаём первый скилл: от структуры до результата

Из чего состоит скилл ↴

Каждый скилл состоит из двух частей:

1. **Название и описание в YAML-формате (frontmatter)** — обязательная часть. Именно этот кусочек попадает в контекст агента. Если название и описание написаны правильно, агент будет знать, КОГДА вызывать скилл.
2. **Тело скилла — инструкции (промт)** — что агенту делать, какие есть ограничения, формат ответа, доступные инструменты.

---

### Формат YAML frontmatter:

---

name: AI News to Social

description: Превращает одну новость об ИИ в посты для Telegram, Instagram, X (Twitter). Использовать, когда нужно быстро привести одну новость в несколько готовых публикаций для разных площадок.

---

### # Инструкция

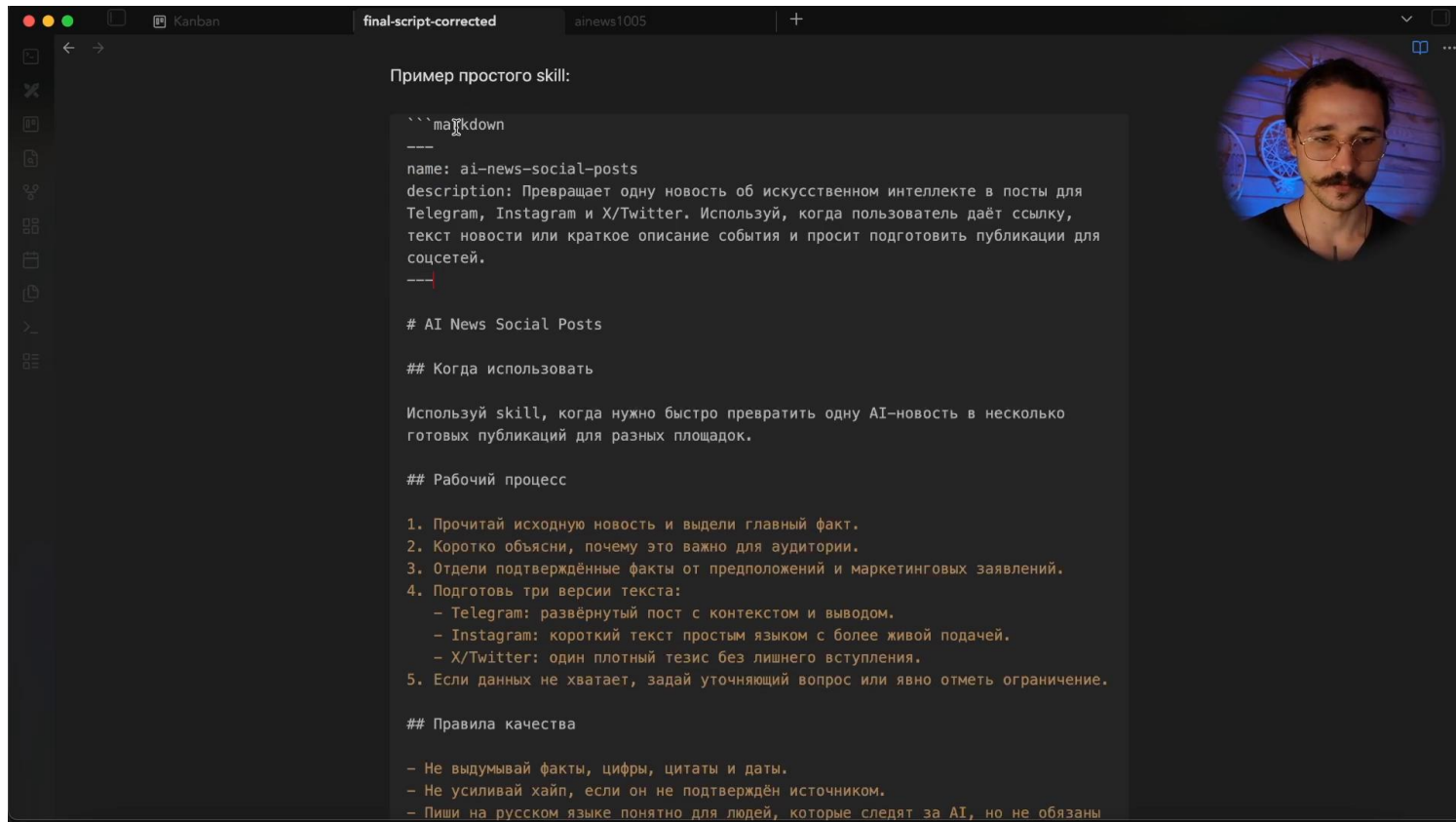
Ниже идёт сам промпт с правилами работы, форматами ответа и примерами.



Агент в начале сессии видит ТОЛЬКО содержимое YAML-блока. Всё, что ниже трёх дефисов, он прочитает только когда решит использовать скилл. Поэтому название и описание должны чётко описывать, КОГДА и ЗАЧЕМ применять скилл.

Визуально в редакторе (ZED, VSCode) YAML frontmatter подсвечивается, и агент его корректно распознаёт.





YAML frontmatter в редакторе ZED — три дефиса, name, description

Структура папок ↘

Файловая структура для скиллов:

папка-проекта/

└─ .claude/

| └─ skills/

| └─ ai-news-to-social/

| | └─ skill.md

| └─ yt-annotation/

| | └─ skill.md

| | └─ extract\_news.py

| | └─ generate\_annotation.sh

| └─ pa-copy/

| └─ skill.md

| └─ telegram-post.md

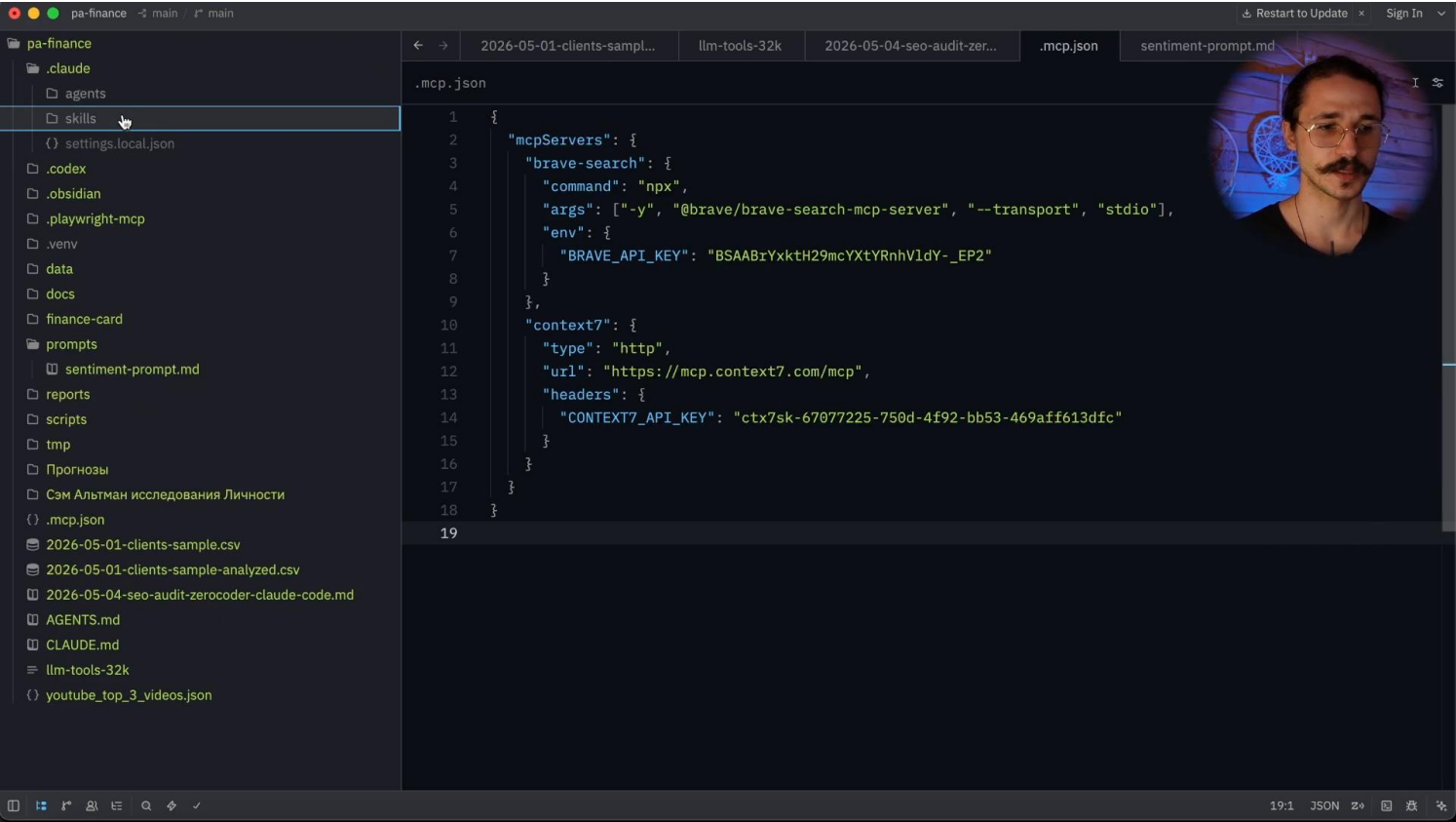
| └─ telegram-post.examples.md

Важные правила:

- Папка skills находится внутри .claude (проектный уровень) или ~/.claude (глобальный)
- Каждый скилл — ОТДЕЛЬНАЯ папка с названием скилла
- Внутри папки обязательно должен быть файл skill.md
- Дополнительно: скрипты, референсы, примеры — всё, что нужно скиллу для работы



Если папки skills нет — создайте её. Без неё ничего не заработает.



Структура папок .claude/skills/ в файловом менеджере

JSON-промптинг: как задать агенту чёткий формат ответа



**JSON-промптинг** — это техника, при которой мы даём агенту JSON-схему и внутри неё описываем, как заполнять каждый ключ. Это даёт чёткий, машиночитаемый формат ответа, который можно дальше переиспользовать.

Пример JSON-схемы в скилле:

```
{
 "news_summary": {
 "short_title": "Короткое название новости (до 100 символов)",
 "source_url": "Ссылка на источник",
 "main_idea": "Главная мысль: почему это важно для аудитории",
 "facts": ["Факт 1", "Факт 2"]
 },
 "telegram_post": {
 "headline": "Заголовок для Telegram (до 150 символов)",
 "body": "Текст поста, 100-200 слов короткими абзацами",
 "hashtags": ["#ter1", "#ter2"],
 "cta": "Призыв к действию в конце"
 },
 "x_post": {
```

```
"hook": "Сильная первая строка",

"body": "Текст до 280 символов"

}

}
```

Преимущества JSON-промптоинга:

- Чёткая структура ответа — не нужно гадать, в каком формате агент вернёт результат
- Результат можно передать дальше по пайплайну: другому агенту, скрипту, в базу данных
- Легче проверять: все ключи на месте или нет



Важно: не забудьте прописать в скилле, куда сохранять результат. Без этого агент выдаст JSON прямо в чате и вам придётся копировать его вручную.

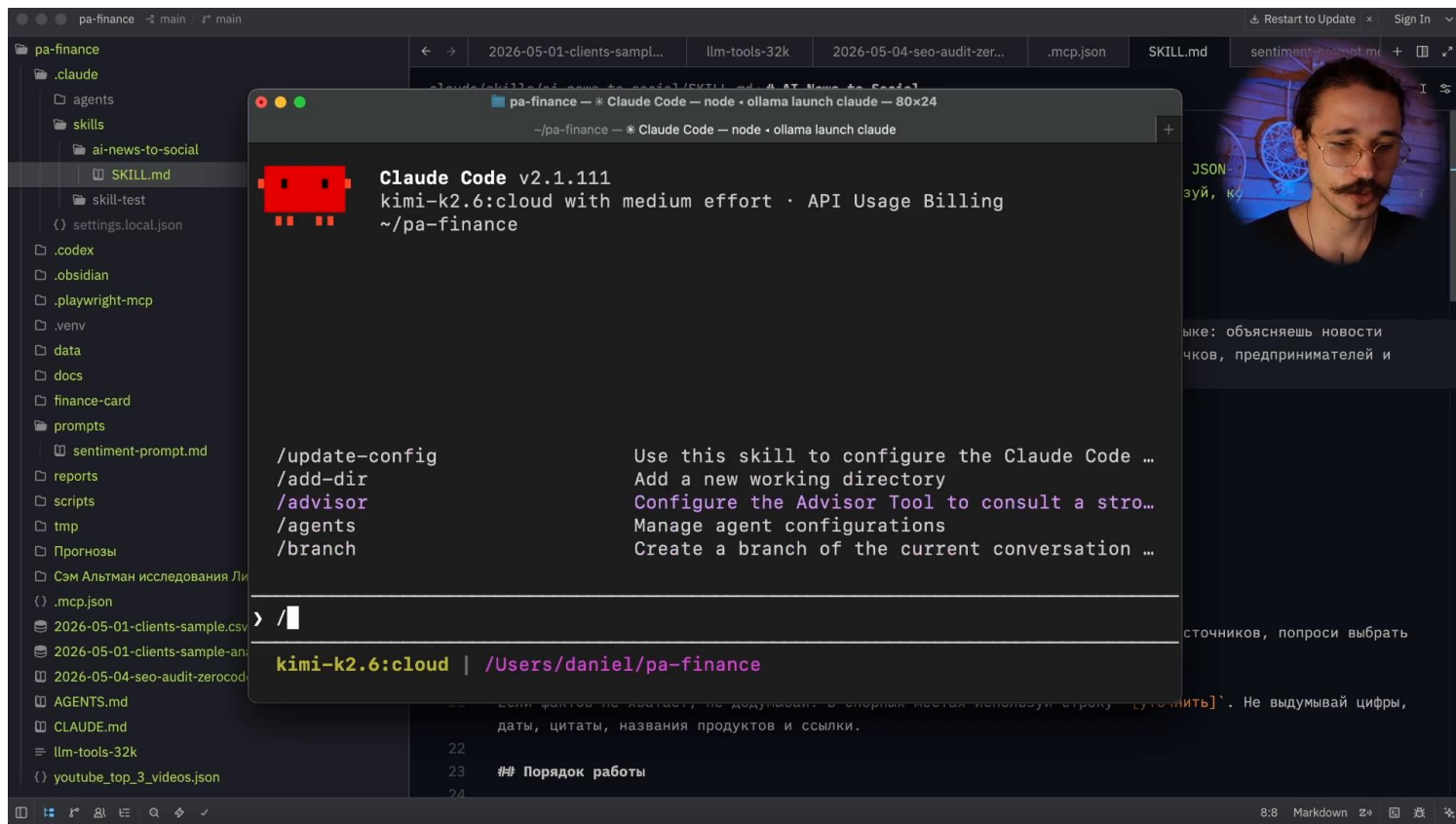
Тестируем скилл: запуск, отладка и команда ↵

После создания скилла проверяем его работу:

1. Запускаем Claude Code в рабочей папке (там, где лежит `.claude/skills/`)
2. Нажимаем / (слэш) — открывается список доступных команд и скиллов
3. Находим наш скилл по названию — он должен быть в списке
4. Даём агенту задачу, которая предполагает использование скилла

Важные нюансы:

- Агент может автоматически определить, что нужно использовать скилл — просто дайте ему релевантную задачу
- Если агент не догадался — явно говорим: «Используем скилл [название]»
- Обращайте внимание на индикатор вызова скилла: в интерфейсе Claude Code появляется уведомление о том, что скилл загружен
- Ctrl+O показывает детальный лог действий агента — полезно для отладки
- Если скилл не появляется в списке — проверьте путь и имя файла (должен быть `skill.md` в папке `.claude/skills/название/`)
- Иногда нужно перезапустить Claude Code, чтобы новый скилл подхватился



### 3. Комплексные скиллы: реальные примеры из ежедневной работы

YT Annotation Generator: Скилл для генерации тайм-кодов и аннотаций ↘

Пример комплексного скилла из ежедневной работы эксперта. Задача: после записи ролика быстро сделать аннотацию и тайм-коды.

Как это работает:

1. Пользователь кидает агенту mp3-файл (единственный input)
2. Скилл находит соответствующий Markdown-файл с новостями в рабочей директории
3. Python-скрипт извлекает заголовки всех новостей из Markdown (по хештегам #)
4. Другой скрипт (curl) отправляет mp3 + список заголовков в Google Gemini API
5. Gemini обрабатывает аудио, сопоставляет с заголовками и возвращает тайм-коды + аннотацию
6. Агент записывает результат обратно в Markdown-файл

Структура скилла:

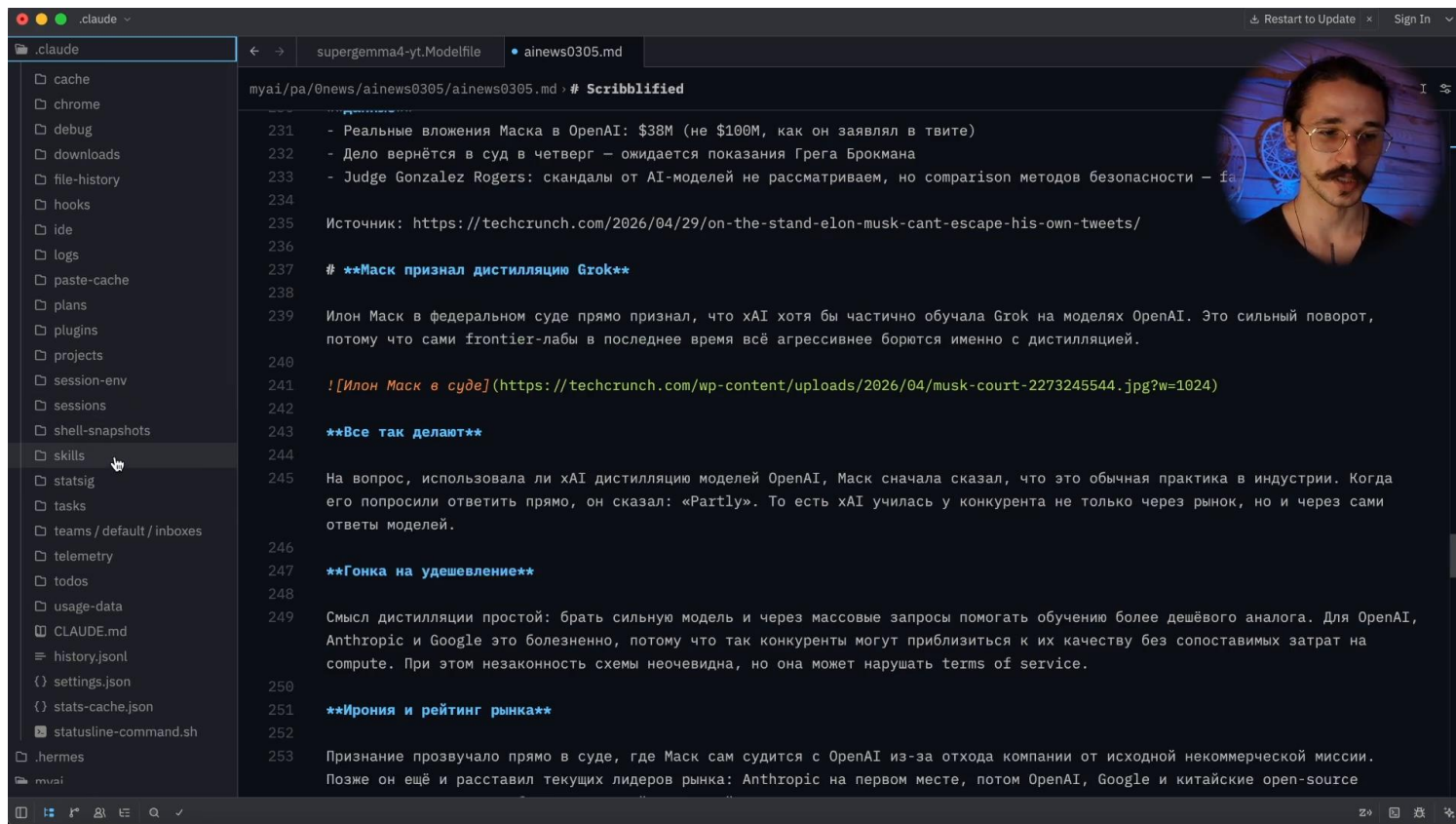
yt-annotation-generator/

- └─ skill.md ← основной файл с YAML и инструкциями
- └─ resolve\_news.sh ← найти нужный Markdown по mp3
- └─ extract\_news.py ← вытащить заголовки из Markdown
- └─ generate\_annotation.sh ← отправить в Gemini API
- └─ .env ← API-ключи (не показываем никому!)



Эксперт не писал эти скрипты самостоятельно — он попросил агента создать их. Мы тоже так можем: описываем задачу агенту, и он пишет нужные bash-команды и Python-скрипты.

Раньше на аннотацию и тайм-коды уходило 30+ минут. Теперь — один mp3-файл на вход, и всё готово.



РА Сору: скилл с референсами и маршрутизацией по площадкам ↘

Ещё один пример комплексного скилла — РА Сору для написания контента на разные площадки.

### Как устроен:

1. В skill.md описана маршрутизация: агент читает задачу и понимает, для какой площадки пишем
2. Затем идёт в файл-референс для этой площадки (например, telegram-post.md) — там описаны правила и Tone of Voice
3. Затем открывает файл с примерами (telegram-post.examples.md) — чтобы понять стиль
4. Только после этого пишет контент, опираясь на референсы

Структура скилла:

ра-сору/

- └─ skill.md ← маршрутизация и общие правила
- └─ telegram-post.md ← как писать для Telegram
- └─ telegram-post.examples.md ← примеры постов для Telegram
- └─ youtube-ad.md ← как писать рекламные ролики
- └─ course-sell.md ← как писать для продажи курсов
- └─ site-article.md ← как писать статьи на сайт



Принцип маршрутизации: агент сначала читает skill.md → понимает, какая площадка нужна → идёт в соответствующий референс → затем в примеры. Так он последовательно собирает весь контекст для конкретной задачи.

Главный принцип: один скилл = одна рутина (до 30 минут) ↘



Главный принцип: упаковывайте в скилл одну конкретную рутину, которая занимает у вас до 30 минут.

Почему не стоит делать один мега-скилл:

- Разные задачи = разные последовательности действий
- Разные задачи = разные инструменты
- Проще поддерживать и обновлять маленькие скиллы
- Агенту легче понять, какой скилл использовать, когда их зоны ответственности чётко разделены

Пример правильного разделения:

- Скилл для постов в Telegram — отдельно
- Скилл для рекламных роликов YouTube — отдельно
- Потом, если начальная логика одинаковая, их можно объединить

Примерная рамка: если задача занимает у вас до 15–20 минут руками — её можно упаковать в скилл. Если дольше 30 минут — возможно, лучше разбить на несколько скиллов.

#### 4. Где искать готовые скиллы и как их безопасно устанавливать



**GERMS Agent** — проект от Naos Research на GitHub.

Ссылка: <https://github.com/NousResearch/hermes-agent/tree/main/skills> (папка Skills)



**ClawHub** — агрегатор скиллов для AI-агентов.

Ссылка: <https://clawhub.ai/> — агрегатор скиллов

GERMS Agent от Naos Research (GitHub) ↘



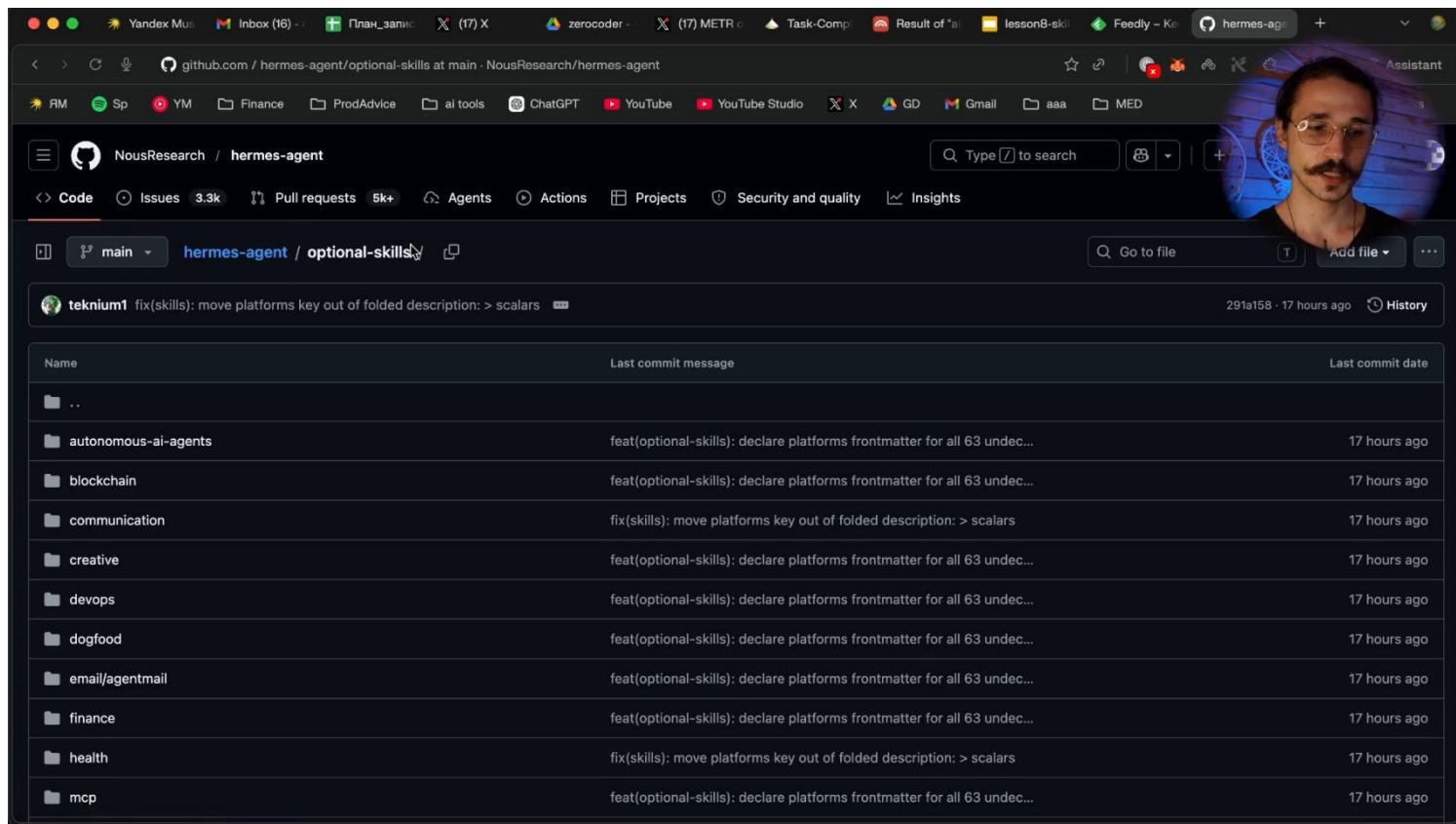
**GERMS Agent** — проект от Naos Research на GitHub, набирающий огромную популярность. В репозитории есть папка Skills с большим количеством готовых скиллов, разбитых по категориям.

Категории скиллов:

- DevOps — Jupyter, LiveKernel, работа с библиотеками
- Data Science — анализ данных, работа с датасетами
- Media — Spotify, работа с аудио и видео
- Creative — генерация мемов, HyperFrames (создание роликов для соцсетей)
- MCP — скиллы для работы с MCP-серверами

Как использовать: заходим в репозиторий → выбираем категорию → смотрим скилл → копируем папку в свой .claude/skills/.





ClawHub — агрегатор скиллов для AI-агентов ↘



**ClawHub** — агрегатор скиллов для AI-агентов. Огромное количество скиллов на все случаи жизни: от создания презентаций до сложных DevOps-пайплайнов.

На что обращать внимание при выборе скилла:

- Количество скачиваний — чем больше, тем провереннее скилл
- Рейтинг и отзывы
- Security scans — наличие проверок безопасности
- Количество звёзд на GitHub (если скилл оттуда)

Интересные скиллы на ClawHub:

- Self-improving agent — агент, который улучшает сам себя
- Создание PowerPoint-презентаций — очень частая задача
- HyperFrames — создание коротких вертикальных роликов для соцсетей

Безопасность: VirusTotal, CloScan и другие проверки ↘



Когда мы загружаем скилл из интернета, мы добавляем агенту инструкции, которые, скорее всего, не читали полностью. Внутри могут быть скрипты, которые отправляют наши данные во внешние сервисы. Поэтому перед установкой скиллы нужно внимательно исследовать и использовать с осторожностью.

Как исследуем скиллы перед установкой:

1. Смотрим на количество скачиваний и звёзд — популярные скиллы обычно уже проверены сообществом.

- Ищем security scans: VirusTotal, CloScan и другие сервисы помогают проверить скилл на подозрительную активность.
- Проверяем рейтинг и отзывы.
- Открываем и читаем skill.md и скрипты перед использованием — хотя бы бегло.
- Не используем скиллы, которые запрашивают доступ к API-ключам без явной необходимости.
- Храним API-ключи в .env файле и не добавляем его в публичные репозитории.

Хороший признак безопасного скилла: несколько сканов безопасности с зелёными галочками, много скачиваний и открытый исходный код.

Как попросить агента устанавливать скилл за нас ↴

Самый простой способ устанавливать скилл — попросить об этом агента:

Устанавливаем этот скилл в проектную директорию Claude Code, в которой ты сейчас находишься: [ссылка на репозиторий/папку со скиллом]

Что делает агент:

- Копирует все файлы скилла в .claude/skills/.
- Проверяет структуру (есть ли skill.md в корне папки скилла).
- Адаптирует под Claude Code, если скилл был написан для другого агента (Germes, OpenCode).
- Сообщает о результате.

Альтернативный способ — вручную:

- Скачать папку скилла из репозитория.
- Добавить её в .claude/skills/.
- Перезапускаем Claude Code.
- Проверить через / (слэш), что скилл появился.

## 5. Итого: как выстраивать систему скиллов

Что мы разобрали на уроке ↴

На этом уроке мы рассмотрели:

- Что такое **skill** и из чего он состоит (`YAML Front Matter` + промпт)
- Почему **YAML-кусочек** (`name` + `description`) — обязательная и критически важная часть каждого скилла
- Уровни хранения скиллов: сессионный, проектный, глобальный, встроенный
- Отличие скиллов от **МСП** и почему скиллы экономичнее по контексту
- Как создать скилл с нуля и подключить его к проекту
- Как работают комплексные скиллы с bash-командами, скриптами и внешними API
- Где искать готовые скиллы и как их безопасно устанавливать

Что делать дальше ↴

**Шаг 1.** Выберите одну простую задачу, которую вы делаете руками и которая занимает до 15–20 минут. Упакуйте её в простой скилл: папка с `skill.md` и текстовым промптом.

**Шаг 2.** Если есть более сложная задача, которую можно автоматизировать с помощью скриптов, инструментов или МСП — попробуйте упаковать её в комплексный скилл. Попросите агента помочь написать нужные команды.



**Шаг 3.** Если пока не знаете, с чего начать — попробуйте готовый скилл из **ClawHub**. Например, скилл для создания презентаций — частая и понятная задача для большинства.

Ваша задача — начать выстраивать систему скиллов вокруг своего проекта. Это работа с большим потенциалом для экономии времени.

## Дополнительные материалы

Словарь занятия ☞

- **Skill (скилл)** — сохранённая рабочая методика выполнения задачи; файл или папка с инструкциями для агента
- **YAML Front Matter** — блок метаданных в начале файла, ограниченный `---`; содержит ``name`` и ``description`` скилла
- **MCP (Model Context Protocol)** — протокол подключения агента к внешнему серверу с инструментами и ресурсами
- **skill.md** — основной файл скилла в формате Markdown
- **JSON-схема / JSON-промтинг** — структурированный формат промта, при котором агент заполняет заданную схему с ключами
- **YAML** — формат разметки данных, используемый для Front Matter в скиллах
- **Pipeline** — цепочка последовательных операций обработки данных
- **Bash-команда** — команда для выполнения в командной строке; используется в комплексных скиллах для автоматизации действий
- **`.env``** — файл с переменными окружения (например, API-ключами), который подтягивается агентом при работе с внешними сервисами
- **Контекстное окно** — ограниченный объём памяти агента в рамках одной сессии; MCP уменьшает его сильнее, чем скиллы

---

## Домашнее задание

Предлагаем два варианта заданий. Первый поможет развить ключевое умение по теме занятия, а второй даст свободу для **проектирования алгоритма для упрощения или даже автоматизации одной из твоих рутинных рабочих задач**.



Выбери только один вариант домашнего задания и выполни его сегодня.

Другой вариант, если захочешь, можешь проработать позже самостоятельно для закрепления материала.

Вариант 1 (Базовый — повтори за экспертом) ☞

Создай простой скилл для написания постов в Telegram (или в любую другую соцсеть, которую ты ведёшь).

### Что сделать:

1. В папке своего проекта открой ``.claude/skills/``
2. Создай папку с названием скилла, например ``telegram-post``
3. Внутри создай файл ``skill.md``
4. Добавь YAML Front Matter: ``name`` и ``description`` (опиши, когда использовать скилл)
5. Добавь промпт: тон, структуру поста, правила форматирования, длину
6. Запусти **Claude Code** из папки проекта, скинь тему или ссылку и проверь, что агент автоматически вызвал скилл (должен появиться маркер вызова в интерфейсе)

**Ожидаемый результат:** агент по одной теме или ссылке генерирует готовый пост нужного формата, используя скилл.

Вариант 2 (Продвинутый — адаптируй под свою задачу) ☑

Выбери одну из своих реальных рутинных задач, которая занимает у тебя до 20–30 минут, и упакуй её в скилл.

**Примеры задач:**

- Написание резюме встречи по заметкам
- Составление технического задания по описанию задачи
- Подготовка еженедельного отчёта по шаблону
- Адаптация одного текста под несколько форматов или аудиторий

**Что сделать:**

1. Опиши агенту свою задачу и попроси его помочь создать скилл: написать `skill.md` с YAML Front Matter и промптом
2. Разместите файл в правильной структуре папок `.claude/skills/`
3. Добавь в скилл инструкцию, куда сохранять результат
4. Протестируй скилл: убедись, что агент его вызывает и результат сохраняется в нужную папку

**Ожидаемый результат:** рабочий скилл, который закрывает одну конкретную рутину и экономит твоё время при следующем выполнении задачи.

Анкета обратной связи по уроку

Эл. адрес

Курс\*

Модуль\*

Урок\*

Понравился ли урок?\*

Понравился, все было понятноНеплохо, но не все получилось/не все понятноНе понравился

Обратная связь по уроку (не обязательно)

**Задание**

Предлагаем два варианта заданий. Первый поможет развить ключевое умение по теме занятия, а второй даст свободу для **проектирования алгоритма для упрощения или даже автоматизации одной из твоих рутинных рабочих задач.**



Выбери только один вариант домашнего задания и выполни его сегодня.

Другой вариант, если захочешь, можешь проработать позже самостоятельно для закрепления материала.

Вариант 1 (Базовый — повтори за экспертом) ☑

Создай простой скилл для написания постов в Telegram (или в любую другую соцсеть, которую ты ведёшь).

**Что сделать:**

1. В папке своего проекта открой `.claude/skills/`
2. Создай папку с названием скилла, например `telegram-post`
3. Внутри создай файл `skill.md`

4. Добавь YAML Front Matter: `name` и `description` (опиши, когда использовать скилл)
5. Добавь промпт: тон, структуру поста, правила форматирования, длину
6. Запусти **Claude Code** из папки проекта, скинь тему или ссылку и проверь, что агент автоматически вызвал скилл (должен появиться маркер вызова в интерфейсе)

**Ожидаемый результат:** агент по одной теме или ссылке генерирует готовый пост нужного формата, используя скилл.

Вариант 2 (Продвинутый — адаптируй под свою задачу) ☞

Выбери одну из своих реальных рутинных задач, которая занимает у тебя до 20–30 минут, и упакуй её в скилл.

#### Примеры задач:

- Написание резюме встречи по заметкам
- Составление технического задания по описанию задачи
- Подготовка еженедельного отчёта по шаблону
- Адаптация одного текста под несколько форматов или аудиторий

#### Что сделать:

1. Опиши агенту свою задачу и попроси его помочь создать скилл: написать `skill.md` с YAML Front Matter и промптом
2. Разместите файл в правильной структуре папок `.claude/skills/`
3. Добавь в скилл инструкцию, куда сохранять результат
4. Протестируй скилл: убедись, что агент его вызывает и результат сохраняется в нужную папку

**Ожидаемый результат:** рабочий скилл, который закрывает одну конкретную рутину и экономит твоё время при следующем выполнении задачи.

[Список тренингов](#)[Практический курс по вайб-кодингу на Claude Code](#)[Модуль 2: Сайты, автоматизация и усиление возможностей Claude Code](#)

CLs04. MCP: как подключить Claude Code к внешним инструментам

[Предыдущий урок](#)

Есть задание

[Следующий урок](#)

Дорогой студент! Интерфейсы современных приложений могут очень быстро меняться. Если вдруг то, что видишь ты, отличается от того, что демонстрирует эксперт, настолько, что тебе непонятно, куда нажимать — напиши об этом в учебный чат, указав номер урока и видео. Наш куратор обязательно поможет тебе разобраться, а мы постараемся как можно скорее актуализировать материал :)



Как проходить уроки эффективно: доступы, оплата и лимиты

**1** Доступ к мировым технологиям. Мы показываем лучшие мировые нейросети, но доступ к некоторым из них из РФ и/или других стран может требовать средств защищенного доступа. Мы действуем в рамках закона и НЕ консультируем по настройке соединения, но показываем эти сервисы, потому что они — лидеры рынка. Разобраться с доступом к ним — значит получить конкурентное преимущество.

**2** Весь курс можно пройти на бесплатных версиях инструментов. Мы показываем платные PRO-функции для ознакомления, чтобы ты мог(ла) оценить их потенциал и осознанно решить, нужны ли они тебе в будущем.

**3** Умное управление лимитами. Бесплатные попытки ограничены, поэтому регистрируйся в сервисе только перед выполнением ДЗ. Если лимиты закончились, а попрактиковаться еще хочется — часто помогает регистрация нового аккаунта на другую почту.

4 Сначала смотри, потом делай. Сначала посмотри урок полностью, чтобы понять логику, и только потом трать генерации. Это сэкономит твоё время и попытки.

Так ты освоишь самые мощные инструменты бесплатно и без лишних нервов! 🚀

---

## Результат дня

- ✓ Поймёшь, что такое MCP (Model Context Protocol) и зачем он нужен агенту
- ✓ Разберёшься в схеме «агент → MCP-сервер → внешний сервис»
- ✓ Узнаешь, как подключить Playwright MCP и провести SEO-аудит сайта
- ✓ Освоишь Context7 MCP для получения актуальной документации по библиотекам
- ✓ Научишься выбирать MCP-серверы безопасно и осознанно, не перегружая контекст агента

▶ Время чтения и просмотра: ~ 60 минут

🕒 Время выполнения задания: ~60 минут



Под каждым видео в уроке есть текстовая расшифровка, чтобы у тебя была возможность лучше запомнить новые термины, прочитать пояснения и инструкции.

Чтобы прочитать текст, нажимай на него. Текст скрыт в строках подчёркнутых пунктирной линией со значком "⌵" в конце строки.

---

## Теория на сегодня

### 1. Зачем агенту MCP

Разбираем, какую проблему решает Model Context Protocol и почему Claude Code не может работать с внешним миром «из коробки».

Ограничения Claude Code без MCP ⌵

Claude Code умеет многое: анализирует файлы, работает с кодом внутри проекта, выполняет команды в терминале. Но если задача требует выхода во внешний мир — обращения к сайту конкурента, актуальной документации, базе данных или стороннему сервису — из коробки он этого сделать не может.

Именно эту проблему решает **MCP (Model Context Protocol)** — открытый стандарт для подключения внешних инструментов к агенту. Разработан компанией **Anthropic**.

**MCP может добавить:**

- доступ к браузеру и веб-сайтам
- актуальную документацию библиотек
- данные из внешних сервисов через API
- подключение к базам данных и CRM

План урока ⌵

В этом уроке мы:

- разберём, зачем агенту нужен MCP и как работает схема «агент → MCP-сервер → инструмент»
- подключим **Playwright MCP** и проведём SEO-аудит реального лендинга

- подключим **Context7 MCP** для работы с актуальной документацией
- научимся выбирать MCP-серверы осознанно
- добавим нужный MCP-сервер в свой рабочий проект

## 2. Как работает MCP: схема и аналогия

Схема: агент → MCP-сервер → сервис ↘

Упрощённая схема выглядит так:

**Агент (Claude Code) → MCP-сервер → Внешний сервис**

Хорошая аналогия: **MCP-сервер — это как USB-порт**. Так же, как USB-C позволяет подключать к ноутбуку разные устройства, MCP позволяет подключать к агенту разные внешние сервисы.

- **Claude Code** — место, где мы ставим задачу агенту. У него есть LLM-мозг и агентная среда с инструментами.
- **MCP-сервер** — конкретное подключение к инструменту, универсальный «переходник».
- **Инструмент** — реальный внешний сервис или источник данных.

Что внутри MCP-сервера ↘

Важно понимать: **MCP-сервер — это не один инструмент**, а набор инструментов, объединённых для работы с одним сервисом. Можно подключить сколько угодно MCP-серверов к Claude Code.

Внутри MCP-сервера бывает несколько типов возможностей:

1. **Инструменты и действия** — агент может и получать данные, и совершать действия во внешнем сервисе.
2. **Ресурсы** — MCP-сервер только забирает контекст из внешнего сервиса (только чтение).
3. **Готовые промпты и шаблоны** — часть экосистемы, которая на практике почти не прижилась; вместо неё сейчас используются **skills** (разберём в следующих уроках).



Один MCP-сервер может уметь и забирать данные, и отправлять информацию, и вытаскивать метаданные — всё для работы с одним внешним источником.

## 3. Подключение Playwright MCP

Набор команд для этой темы:

```
node --version
```

```
npx --version
```

```
claude --version
```

```
claude mcp add playwright -- npx -y
```

```
@playwright/mcp@latest
```

```
claude mcp list
```

```
claude mcp get playwright
```

```
/mcp
```

Проверка зависимостей ↘

Перед установкой убеждаемся, что на компьютере установлено всё необходимое. Вводим в терминале:

```
node --version
```

npx --version

claude --version

- node --version — проверяем наличие **Node.js**;
- npx --version — проверяем наличие **NPX**. Он устанавливается вместе с **Node.js**;
- claude --version — проверяем наличие **Claude Code**.

Если **Node.js** не установлен, скачиваем его на [nodejs.org](https://nodejs.org).

**NPX** установится автоматически вместе с **Node.js**.

Установка Playwright MCP 📄

**Playwright** — это инструмент, который позволяет агенту открывать сайты, делать скриншоты, анализировать визуальный контент и вытаскивать метаданные страниц.



Документация Playwright — <https://playwright.dev/mcp/clients/claude-code>.

Чтобы найти команду установки:

1. Переходим на сайт **playwright.dev**
2. Находим раздел **Installation → Client Setup**
3. Выбираем клиент **Claude Code**
4. Копируем предложенную команду

Переходим в терминале в корень рабочей директории проекта и вводим команду. После выполнения терминал сообщит, что MCP-сервер добавлен, и укажет, в какой файл внесены изменения.



Команда устанавливает Playwright MCP **проектно** — он будет работать только внутри той папки, из которой была запущена команда. Если нужно использовать его в другом проекте, установите его туда отдельно.

Где хранятся настройки MCP 📄

После выполнения команды мы увидим все подключённые **MCP-серверы** и слои, на которых они настроены.

Существует два места хранения настроек **MCP** внутри проекта:

- .claude.json в корневой директории пользователя — например, Users/имя/.claude.json. Здесь хранятся глобальные и проектные настройки;
- .mcp.json внутри папки проекта — альтернативное место для проектных настроек.

*Разницы между этими двумя вариантами нет. Главное, чтобы настройки были добавлены и корректно работали.*

Если хочется держать конфигурацию аккуратно, можно перенести все **MCP-серверы** в один файл.

После установки проверяем конфигурацию внутри **Claude Code**. Для этого вводим команду:

/mcp



MCP-серверы можно подключать **глобально** (доступны из любого проекта) или **проектно** (только в конкретной папке). Глобальная установка занимает токены контекста во всех ваших проектах — об этом важно помнить.

#### 4. SEO-аудит через Playwright MCP



**Playwright** — это инструмент, который позволяет агенту открывать сайты, делать скриншоты, анализировать визуальный контент и вытаскивать метаданные страниц.

Промпт для SEO-аудита ↴

После подключения Playwright мы даём агенту такой промпт:

Проведи SEO-аудит этого лендинга. Составь план SEO-аудита, определи, что ты можешь вытащить с этого сайта и какую статистику собрать при помощи MCP-сервера Playwright, и используй этот инструмент для того, чтобы провести полноценный аудит. Результат аудита сохрани в корень рабочей директории. [Ссылка на сайт]



Сайт для тренировки: <https://zerocoder.ru/course-claude-code>

При первом вызове Playwright агент попросит подтвердить права доступа — разрешаем ему всегда вызывать этот инструмент.

Как агент работает с Playwright ↴

Агент действует последовательно и самостоятельно:

1. Загружает сайт по ссылке через Playwright
2. Находит название страницы и базовые метаданные
3. Делает **скриншот** и сохраняет его в папку `temp`
4. Читает **HTML-код** страницы
5. Скачивает дополнительные файлы (например, `robots.txt`) через встроенную команду `curl`
6. Анализирует все собранные данные и формирует итоговый отчёт



Агент умеет комбинировать инструменты: часть данных вытаскивает через Playwright MCP, часть — через встроенные инструменты вроде `curl`. Это называется комплексная работа с инструментами.

Что даёт Playwright в разработке ↴

Playwright MCP открывает много возможностей за пределами SEO-аудита:

- Агент может **самостоятельно делать скриншоты** сайтов как референсы для разработки
- Может открыть результат своей работы в браузере на **localhost**, сделать скриншот и проверить, всё ли сделано правильно
- Может **сравнить** результат с исходным заданием и исправить ошибки без вашего участия

Такие более комплексные и продолжительные задачи с самопроверкой вы можете давать вашему агенту, когда у вас есть MCP Playwright.

## 5. Подключение Context7 MCP



**Context7** — MCP-сервер, который собирает актуальную документацию по огромному количеству библиотек и фреймворков. Позволяет агенту получать свежую документацию прямо во время работы над проектом.



Ссылка на документацию: <https://context7.com/docs/resources/all-clients>

Подключение Context7 MCP ↴

Установка Context7 отличается от Playwright — **требует API-ключ**:

1. Регистрируемся на [context7.com](https://context7.com)
2. Заходим в аккаунт → переходим в [Дашборд](#) → создаём API-ключ
3. Находим команду установки в [документации](#) (раздел MCP-клиенты)

```
claude mcp add --scope user --header "CONTEXT7_API_KEY: YOUR_API_KEY" --transport http context7
https://mcp.context7.com/mcp
```

4. Вставляем свой API-ключ вместо плейсхолдера в команде
5. Запускаем команду в терминале



Никому не передавайте API-ключи. Если вы случайно опубликовали ключ — удалите его и создайте новый.

Context7 устанавливается **глобально** командой по умолчанию. Если вы хотите ограничить его конкретным проектом — перенесите конфигурацию из глобального `.claude.json` в файл `.mcp.json` внутри папки проекта вручную или попросите это сделать агента.

Как Context7 работает внутри одного MCP-сервера ↴

Проверяем Context7 простым запросом: *используй Context7, найди актуальную документацию по Next.js App Router, коротко объясни, как задавать метаданные для страницы.*

Агент последовательно использует **два инструмента внутри одного MCP-сервера**:

1. `ResolveLibraryID` — определяет идентификатор библиотеки (в данном случае Next.js)
2. `GetLibraryDocs` — запрашивает актуальную документацию по найденному ID и нужной теме

В итоге агент возвращает актуальный фрагмент кода из документации — именно так, как это описано в официальных исходниках.

Контекст и токены: важно знать ↴

Команда `/context` внутри Claude Code показывает, чем занято контекстное окно агента. Это важно понимать при работе с MCP.

Типичное распределение токенов при трёх подключённых MCP-серверах:

- Системный промпт: ~5 900 токенов
- Описание встроенных инструментов: ~15 000 токенов
- **MCP Tools (3 сервера): ~5 800 токенов**
- Сообщения в текущей сессии: ~4 000 токенов

Итого: из 200 000 токенов контекстного окна уже ~36 000 занято до начала реального общения.



Чем больше MCP-серверов подключено, тем меньше контекста остаётся для реальной работы. Подключайте только те серверы, которые действительно нужны в конкретном проекте. Используйте **проектную установку**, чтобы не тратьте контекст в других проектах.



Context7 занимает относительно мало токенов — его можно держать в глобальных настройках без существенного ущерба для контекста.

## 6. Где искать MCP-серверы и как выбирать

Где искать MCP-серверы ↴

Для Notion можно так:



## Основные источники для поиска MCP-серверов:

- [GitHub: modelcontextprotocol/servers](#) — официальный репозиторий, 85 000+ звёзд. Сейчас это скорее агрегатор ссылок на другие источники.
- [mcp-hunt](#) — подборка трендовых MCP-серверов.
- [Smithery](#) — основной рекомендуемый агрегатор. Большой каталог серверов с фильтрами, рейтингами и описаниями.
- [Glama](#) — ещё один агрегатор с возможностью сортировки.

На **Smithery** можно найти MCP-серверы для разных задач:

- поиск: **Brave Search, Exa Search**;
- работа с данными: **Supabase**;
- коммуникации: **Slack, Google Drive**;
- аналитика;
- научные исследования;
- GitHub;
- погода;
- математика;
- **Bitrix24**.

Что проверять перед установкой ☞

Перед тем как установить любой MCP-сервер, проверяем:

- **Кто автор** — официальный сервер от команды сервиса или сделан сторонним разработчиком-энтузиастом
- **Есть ли README** — документация, примеры использования, описание инструментов
- **Как давно обновлялся** — актуален сервер или заброшен
- **Нужны ли права и секреты** — API-ключ, токены, авторизация (как у Context7 и Ahrefs)
- **Можно ли ограничить до read-only** — важно понимать, может ли сервер совершать действия от вашего имени
- **Не дублирует ли уже подключённый инструмент** — не стоит держать два поисковика, если один справляется



Некоторые MCP-серверы могут быть **опасными**: они могут отправить агента на вредоносный сайт, передать промпт, который «взламывает» агента и вытащит ваши данные. Используйте только проверенные, официальные серверы с хорошей репутацией.



Часть MCP-серверов требует **платной подписки** на сам сервис — например, Ahrefs для SEO-аудита. Это не «волшебная таблетка»: MCP даёт доступ к инструменту, но не заменяет подписку.

## 7. Итоги урока

Главные принципы ☞

**MCP-сервер расширяет рабочее пространство агента** за пределы локальной работы на компьютере.

Подключение можно считать хорошим тогда, когда оно решает конкретную задачу внутри проекта. Не MCP ради MCP — а реальный инструмент для реальной проблемы.

Лучше один MCP, чем 10 шумных подключений, которые забивают контекст. Всё равно вы их использовать не будете, а на работоспособность агента это влияет негативно.

Не забываем про **проверку результатов**: всё нужно отсматривать глазами и проверять руками — особенно на начальном этапе работы с новым инструментом. Важно выработать доверие к агенту: понять, что он делает, как делает и почему выбирает те или иные инструменты.



В следующем уроке разберём **Skills** — архитектурное решение, которое часто можно использовать вместо MCP, при этом оставив контекстное окно практически пустым.

## Дополнительные материалы

Ссылки на сервисы из урока ↘

- [Playwright](#) — библиотека для браузерной автоматизации. MCP-сервер позволяет агенту открывать сайты, делать скриншоты и анализировать страницы.
- [Context7](#) — MCP-сервер с актуальной документацией по тысячам библиотек и фреймворков. Требуется регистрация и API-ключ.
- [Smithery](#) — основной агрегатор MCP-серверов с фильтрами, рейтингами и описаниями.
- [Glama](#) — агрегатор MCP-серверов с возможностью сортировки.
- [MCP Hunt](#) — подборка трендовых MCP-серверов.
- [GitHub: modelcontextprotocol/servers](#) — официальный репозиторий MCP-серверов и ссылок на экосистему MCP.
- [Node.js](#) — среда выполнения JavaScript. Необходима для работы большинства MCP-серверов. **NPX** устанавливается вместе с ней.
- [Ahrefs](#) — сервис для SEO-аудита. Имеет собственный MCP-сервер, для работы требуется платная подписка.

Словарь занятия ↘

- **MCP (Model Context Protocol)** — открытый стандарт от Anthropic для подключения внешних инструментов и сервисов к AI-агенту
- **MCP-сервер** — конкретная реализация подключения к внешнему сервису; может содержать несколько инструментов внутри
- **Playwright MCP** — MCP-сервер на базе библиотеки Playwright; позволяет агенту управлять браузером, делать скриншоты и анализировать веб-страницы
- **Context7** — MCP-сервер, предоставляющий актуальную документацию по библиотекам и фреймворкам прямо в контекст агента
- **Проектная установка** — конфигурация MCP-сервера, работающая только в конкретной папке проекта
- **Глобальная установка** — конфигурация MCP-сервера, доступная из всех проектов на компьютере
- **API-ключ** — уникальный токен авторизации, необходимый для доступа к некоторым MCP-серверам
- **Контекстное окно** — ограниченное пространство памяти агента (в Claude Code — до 200 000 токенов); MCP-серверы занимают часть этого пространства
- **read-only** — режим работы MCP-сервера только на чтение данных, без возможности совершать действия
- **NPX** — пакетный менеджер для Node.js, используется для запуска и установки MCP-серверов
- **robots.txt** — файл на сайте, определяющий правила индексации для поисковых роботов; используется при SEO-аудите
- **Skills** — архитектурное решение в Claude Code, альтернатива MCP в ряде сценариев

## Домашнее задание

Предлагаем два варианта заданий. Первый поможет развить ключевое умение по теме занятия, а второй даст свободу для **проектирования алгоритма для упрощения или даже автоматизации одной из твоих рутинных рабочих задач**.



Выбери только один вариант домашнего задания и выполни его сегодня.

Другой вариант, если захочешь, можешь проработать позже самостоятельно для закрепления материала.

Вариант 1 (Базовый — повтори за экспертом) ↘

Подключи **Playwright MCP** к своему рабочему проекту в **Claude Code** и проведи **SEO-аудит любого сайта**: своего, сайта конкурента или любого другого.

### Что нужно сделать:

1. Убедись, что на компьютере установлены **Node.js**, **NPX** и **Claude Code**.
2. `node --version`
3. `npx --version`

`claude --version`

2. Перейди в терминале в корень своего проекта.
3. Найди команду установки на [playwright.dev](https://playwright.dev):

### Installation → Client Setup → Claude Code

Выполни команду установки в терминале.

4. Открой **Claude Code** и проверь подключение командой:

`/mcp`

5. Убедись, что **Playwright** появился в списке подключённых MCP-серверов.
6. Напиши агенту промпт:

*Составь план SEO-аудита, определи, что можно вытащить с сайта [URL] при помощи MCP Playwright, и проведи полноценный аудит. Результат сохрани в корень рабочей директории.*

7. Дождись результата, просмотрю SEO-отчёт и сохранённый скриншот сайта в папке temp.

### Результат:

Файл с SEO-отчётом и скриншот сайта в папке проекта.

### Материалы, которые нужно прикрепить к ДЗ:

- скриншот команды `/mcp` в **Claude Code**, где видно подключённый **Playwright**;
- скриншот промпта, который ты отправил(-а) агенту;
- скриншот результата работы агента или открытого SEO-отчёта;
- сам файл SEO-отчёта, если платформа позволяет прикреплять файлы;
- небольшой текстовый отчёт.

### Что написать в текстовом отчёте:

- какой сайт ты выбрал(-а) для аудита;
- для чего использовался **Playwright MCP**;

- какие данные удалось получить с сайта;
- какие выводы дал SEO-аудит;
- что можно улучшить на сайте по результатам проверки.

Вариант 2 (Продвинутый — адаптируй под свою задачу) ☞

Выбери **один MCP-сервер**, который решает реальную задачу в твоём проекте, подключи его и получи первый практический результат.

**Что нужно сделать:**

1. Определи, какого внешнего контекста не хватает твоему агенту:
  - данные из сервиса;
  - актуальная документация;
  - поиск в интернете;
  - работа с файлами в облаке;
  - аналитика;
  - другой источник данных для твоего проекта.
2. Найди подходящий MCP-сервер на [Smithery](#) или в официальном репозитории [GitHub: modelcontextprotocol/servers](#).
3. Проверь MCP-сервер по чек-листу:
  - кто автор;
  - когда сервер обновлялся;
  - есть ли понятный **README**;
  - какие права и доступы нужны;
  - требуется ли API-ключ;
  - не дублирует ли сервер уже подключённый инструмент.
4. Установи MCP-сервер **проектно** — в папку своего проекта.
5. Проверь подключение через команду:

/mcp

6. Выполни первый реальный запрос к агенту с использованием этого MCP-сервера.
7. Проверь расход контекста командой:

/context

**Результат:**

Подключённый и проверенный MCP-сервер, который решает конкретную задачу проекта, и первый артефакт его работы.

**Материалы, которые нужно прикрепить к ДЗ:**

- скриншот выбранного MCP-сервера на **Smithery** или в GitHub-репозитории;
- скриншот результата работы агента или созданного артефакта;
- скриншот команды /context, где видно, сколько токенов занял MCP-сервер;
- сам файл или ссылка на артефакт, если результат можно приложить отдельно;

- небольшой текстовый отчёт.

#### Что написать в текстовом отчёте:

- какую задачу ты решил(-а) с помощью MCP-сервера;
- какой MCP-сервер выбрал(-а);
- почему выбрал(-а) именно его;
- какие данные, сервисы или функции он добавил агенту;
- какой первый результат удалось получить;
- помог ли MCP-сервер решить задачу проекта и почему.

Анкета обратной связи по уроку

Эл. адрес

Курс\*

Модуль\*

Урок\*

Понравился ли урок?\*

Понравился, все было понятно Неплохо, но не все получилось/не все понятно Не понравился

Обратная связь по уроку (не обязательно)

#### Задание

Предлагаем два варианта заданий. Первый поможет развить ключевое умение по теме занятия, а второй даст свободу для **проектирования алгоритма для упрощения или даже автоматизации одной из твоих рутинных рабочих задач.**



Выбери только один вариант домашнего задания и выполни его сегодня.

Другой вариант, если захочешь, можешь проработать позже самостоятельно для закрепления материала.

Вариант 1 (Базовый — повтори за экспертом) ▾

Подключи **Playwright MCP** к своему рабочему проекту в **Claude Code** и проведи **SEO-аудит любого сайта**: своего, сайта конкурента или любого другого.

#### Что нужно сделать:

1. Убедись, что на компьютере установлены **Node.js**, **NPX** и **Claude Code**.
2. `node --version`
3. `npx --version`

`claude --version`

2. Перейди в терминале в корень своего проекта.
3. Найди команду установки на [playwright.dev](https://playwright.dev):

#### Installation → Client Setup → Claude Code

Выполни команду установки в терминале.

4. Открой **Claude Code** и проверь подключение командой:

`/mcp`

5. Убедись, что **Playwright** появился в списке подключённых MCP-серверов.

6. Напиши агенту промпт:

*Составь план SEO-аудита, определи, что можно вытащить с сайта [URL] при помощи MCP Playwright, и проведи полноценный аудит. Результат сохрани в корень рабочей директории.*

7. Дождись результата, просмотри SEO-отчёт и сохранённый скриншот сайта в папке temp.

### Результат:

Файл с SEO-отчётом и скриншот сайта в папке проекта.

### Материалы, которые нужно прикрепить к ДЗ:

- скриншот команды /mcp в **Claude Code**, где видно подключённый **Playwright**;
- скриншот промпта, который ты отправил(-а) агенту;
- скриншот результата работы агента или открытого SEO-отчёта;
- сам файл SEO-отчёта, если платформа позволяет прикреплять файлы;
- небольшой текстовый отчёт.

### Что написать в текстовом отчёте:

- какой сайт ты выбрал(-а) для аудита;
- для чего использовался **Playwright MCP**;
- какие данные удалось получить с сайта;
- какие выводы дал SEO-аудит;
- что можно улучшить на сайте по результатам проверки.

Вариант 2 (Продвинутый — адаптируй под свою задачу) ☝

Выбери **один MCP-сервер**, который решает реальную задачу в твоём проекте, подключи его и получи первый практический результат.

### Что нужно сделать:

1. Определи, какого внешнего контекста не хватает твоему агенту:
  - данные из сервиса;
  - актуальная документация;
  - поиск в интернете;
  - работа с файлами в облаке;
  - аналитика;
  - другой источник данных для твоего проекта.
2. Найди подходящий MCP-сервер на [Smithery](#) или в официальном репозитории [GitHub: modelcontextprotocol/servers](#).
3. Проверь MCP-сервер по чек-листу:
  - кто автор;
  - когда сервер обновлялся;
  - есть ли понятный **README**;
  - какие права и доступы нужны;

- требуется ли API-ключ;
- не дублирует ли сервер уже подключённый инструмент.

4. Установи MCP-сервер **проектно** — в папку своего проекта.

5. Проверь подключение через команду:

/mcp

6. Выполни первый реальный запрос к агенту с использованием этого MCP-сервера.

7. Проверь расход контекста командой:

/context

### Результат:

Подключённый и проверенный MCP-сервер, который решает конкретную задачу проекта, и первый артефакт его работы.

### Материалы, которые нужно прикрепить к ДЗ:

- скриншот выбранного MCP-сервера на **Smithery** или в GitHub-репозитории;
- скриншот результата работы агента или созданного артефакта;
- скриншот команды /context, где видно, сколько токенов занял MCP-сервер;
- сам файл или ссылка на артефакт, если результат можно приложить отдельно;
- небольшой текстовый отчёт.

### Что написать в текстовом отчёте:

- какую задачу ты решил(-а) с помощью MCP-сервера;
- какой MCP-сервер выбрал(-а);
- почему выбрал(-а) именно его;
- какие данные, сервисы или функции он добавил агенту;
- какой первый результат удалось получить;
- помог ли MCP-сервер решить задачу проекта и почему.

[Список тренингов](#)[Практический курс по вайб-кодингу на Claude Code](#)[Модуль 3: Цепочки задач и прикладные сценарии для бизнеса](#)

CLb01 Локальный модели. Автоматизация рутинных задач

Есть задание

[Следующий урок](#)

Дорогой студент! Интерфейсы современных приложений могут очень быстро меняться. Если вдруг то, что видишь ты, отличается от того, что демонстрирует эксперт, настолько, что тебе непонятно, куда нажимать — напиши об этом в учебный чат, указав номер урока и видео. Наш куратор обязательно поможет тебе разобраться, а мы постараемся как можно скорее актуализировать материал :)



Как проходить уроки эффективно: доступы, оплата и лимиты

**1** Доступ к мировым технологиям. Мы показываем лучшие мировые нейросети, но доступ к некоторым из них из РФ и/или других стран может требовать средств защищенного доступа. Мы действуем в рамках закона и НЕ

консультируем по настройке соединения, но показываем эти сервисы, потому что они — лидеры рынка. Разобраться с доступом к ним — значит получить конкурентное преимущество.

2. Весь курс можно пройти на бесплатных версиях инструментов. Мы показываем платные PRO-функции для ознакомления, чтобы ты мог(ла) оценить их потенциал и осознанно решить, нужны ли они тебе в будущем.
3. Умное управление лимитами. Бесплатные попытки ограничены, поэтому регистрируйся в сервисе только перед выполнением ДЗ. Если лимиты закончились, а попрактиковаться еще хочется — часто помогает регистрация нового аккаунта на другую почту.
4. Сначала смотри, потом делай. Сначала посмотри урок полностью, чтобы понять логику, и только потом трать генерации. Это сэкономит твоё время и попытки.

Так ты освоишь самые мощные инструменты бесплатно и без лишних нервов! 🚀

---

#### Результат дня

- ✓ Поймёшь, что такое локальные LLM, чем они отличаются от облачных и в каких случаях их стоит использовать
- ✓ Установишь Ollama и скачаешь локальную модель (Gemma 4 или аналог) на свой компьютер
- ✓ Научишься настраивать модель: расширять контекстное окно и добавлять поддержку инструментов через Model File
- ✓ Подключишь локальную модель к агентной среде Goose и убедишься, что она умеет вызывать внешние инструменты
- ✓ Напишешь Python-скрипт, который использует локальную LLM через Ollama для автоматического анализа клиентских отзывов из CSV-таблицы

▶ Время чтения и просмотра: ~ 80 минут

🕒 Время выполнения задания: ~60 минут



Под каждым видео в уроке есть текстовая расшифровка, чтобы у тебя была возможность лучше запомнить новые термины, прочитать пояснения и инструкции.

Чтобы прочитать текст, нажимай на него. Текст скрыт в строках подчёркнутых пунктирной линией со значком "⌵" в конце строки.

---

### Теория на сегодня

#### 1. Локальные модели: зачем, когда и как выбирать

Что такое локальная модель и зачем она нужна ⌵

Представьте ситуацию: у вас на руках клиентская база, сотни строк, внутренние задачи или заметки по проекту. Вы хотите, чтобы ИИ помог это рассортировать, выделить главное, составить черновые письма. Но возникает вопрос: можно ли вообще загружать такие данные в интернет?

Здесь появляется логичная мысль: что если модель может работать прямо на моём компьютере, без доступа к интернету, сохраняя всю приватную информацию локально? Это и есть **локальная модель**.

Есть три практичных повода заинтересоваться **локальными LLM**:

- **Приватность.** Внутренние корпоративные списки, отчёты, личные заметки — с локальной моделью всё остаётся на вашем устройстве. Никаких вопросов о том, куда ушли данные и кто ими пользуется.



- **Затраты.** Облачные запросы стоят денег. Если вы каждый день обрабатываете однотипные тексты и активно используете API, локальная модель поможет срезать эти затраты.
- **Автономность.** Там, где слабый интернет или его нет вовсе, локальная модель работает полностью независимо.

Локальная модель — это не бесплатная замена облаку. Это другой инструмент со своими сильными и слабыми сторонами.



Локальная модель нужна там, где важны **приватность, контроль и регулярная рутина**.

Локальные vs облачные модели: сравнение ↘

Разберём ключевые отличия по нескольким параметрам:

- **Где работает:** облачная — на серверах компании через интернет; локальная — прямо на вашем компьютере.
- **Интернет:** облачной он обязателен; локальная работает без сети.
- **Качество ответов:** облачные модели, особенно крупные проприетарные, заметно сильнее маленьких локальных. Качество локальной зависит от мощности вашего железа.
- **Приватность:** локальные модели обеспечивают полную приватность — данные никуда не уходят. В облачных данные передаются на сервер, хотя большинство провайдеров позволяют отключить использование данных для обучения в настройках.
- **Порог входа:** облачная — регистрация за пару минут; локальная — нужно скачать, настроить, подобрать модель под железо.
- **Стоимость:** облачная — месячная подписка или оплата за токены; локальная — ваше железо и электроэнергия.
- **Контроль:** в облачной провайдер всё настраивает сам; в локальной можно тонко настраивать параметры под конкретную задачу.



Никакие приватные данные — API-ключи, пароли, секреты — нельзя отправлять в облачные модели. С локальными моделями в этом плане свободы значительно больше.

Когда выбирать локальную, а когда облачную модель ↘

Разберём на конкретных примерах:

- **Менеджер обрабатывает клиентскую базу перед рассылкой** — данные чувствительные → **локальная модель**.
- **Дизайнер просит придумать необычную идею для баннера** — никаких приватных данных → **облачная модель**.
- **Нет интернета, нужно переписать черновик документа** — ответ очевиден → **локальная модель**.
- **Нужен глубокий юридический анализ или сложная математика** → **облачная модель**: маленькая локальная с этим, скорее всего, не справится.

Самый простой способ запустить локальную модель — приложение **Ollama**. Оно позволяет скачивать и запускать модели в формате **GGUF**, адаптированном для работы на обычном компьютере. Вместо подписки вы платите только железом и электроэнергией.

## 2. Установка Ollama и скачивание первой модели



[Ollama](#) — это программа, которая позволяет запускать нейросетевые модели **локально на компьютере**, без обращения к ChatGPT, Claude или другим облачным сервисам.



Ссылка на модели в Ollama — <https://ollama.com/search>

Установка Ollama ↘

Переходим к практике. **Ollama** — приложение, которое мы будем использовать для запуска локальных моделей.

Установка проходит в несколько шагов:

1. Заходим на сайт [Ollama](#).
2. Скачиваем приложение под вашу операционную систему (macOS, Windows, Linux).
3. Устанавливаем стандартным способом.

После установки Ollama можно открыть как обычное приложение — откроется чат-интерфейс, привычный и понятный. В нём можно работать как с облачными моделями (если оформлена подписка Ollama Cloud), так и с локальными — которые нам ещё нужно скачать.



Второй способ работы с Ollama — через **терминал**. Именно этот способ мы будем использовать дальше, потому что он даёт больше контроля.

Основные команды Ollama в терминале ↘

Для начала убедимся, что Ollama запущена (приложение должно быть открыто), и перейдём в терминал.

Полезные команды:

- ``ollama`` — показывает все доступные опции взаимодействия с приложением.
- ``ollama list`` — выводит список всех **локально установленных** моделей.
- ``ollama run <название модели>`` — запускает модель и открывает чат прямо в терминале.
- ``ollama pull <название модели>`` — только скачивает модель, без немедленного запуска.
- ``ollama ps`` — показывает статистику последнего запуска модели: название, размер, загрузка GPU/CPU, использованный контекст.
- ``ollama show <название модели>`` — показывает дефолтные характеристики модели: параметры, контекстное окно, поддерживаемые возможности.



Перед запуском любой задачи переходите в вашу **рабочую директорию** — это стандартная практика при работе с агентами.

Как выбрать модель для установки ↘



Ссылка на модели в Ollama — <https://ollama.com/search>

В интерфейсе Ollama есть подборка моделей, адаптированных для запуска внутри приложения. При выборе обращайте внимание на теги:

- **Число + B** (например, 4B, 27B) — количество параметров модели в миллиардах.
- Чем больше параметров, тем умнее модель, но тем больше ресурсов она требует.

**Рекомендация для старта:** [Gemma 4 от Google](#) — открытая (open-source), эффективная и достаточно умная модель. Начните с неё.

3. Подбор модели под ваше железо

Требования к железу ↘

Прежде чем скачивать модель, нужно понимать, что потянет ваш компьютер:

- **8 ГБ оперативной / объединённой памяти** → модели на **2–4 миллиарда параметров**.
- **16 ГБ** → можно пробовать модели на **12–16 миллиардов параметров**.
- **24 ГБ и выше** → возможен запуск моделей на **26 миллиардов параметров**, но и это бывает непросто.

Models

[View all →](#)

| Name                           | Size / Usage           | Context | Input       |
|--------------------------------|------------------------|---------|-------------|
| gemma4:latest                  | 9.6GB                  | 128K    | Text, Image |
| gemma4:e2b                     | 7.2GB                  | 128K    | Text, Image |
| gemma4:e4b <span>latest</span> | 9.6GB                  | 128K    | Text, Image |
| gemma4:26b                     | 18GB                   | 256K    | Text, Image |
| gemma4:31b                     | 20GB                   | 256K    | Text, Image |
| gemma4:31b-cloud               | <div><div></div></div> | 256K    | Text, Image |



Если модель не помещается в GPU-память полностью, Ollama перенесёт часть вычислений на CPU. Это сделает работу очень медленной. В команде ollama ps смотрите на параметр GPU — он должен быть **100%**.



**CPU / ЦПУ** — это центральный процессор компьютера.

Он отвечает почти за всё: запуск программ, работу системы, обычные вычисления. В ноутбуке это, например, Intel Core, AMD Ryzen, Apple M-серии.

**GPU / ГПУ** — это графический процессор, то есть видеокарта.

Изначально он нужен для графики, игр, видео, 3D, но ещё он очень хорошо подходит для нейросетей, потому что умеет быстро делать много однотипных вычислений параллельно.

Инструкция: как посмотреть оперативную память ↘

Windows

Способ 1: через параметры

1. Нажмите **Пуск**.
2. Откройте **Параметры**.
3. Перейдите в **Система** → **О системе**.
4. Найдите строку **Установленная оперативная память (RAM)**.

Способ 2: через диспетчер задач

1. Нажмите **Ctrl + Shift + Esc**.
  2. Откройте вкладку **Производительность**.
  3. Выберите **Память**.
  4. В правом верхнем углу будет указан объём оперативной памяти.
- 

## macOS

1. Нажмите на значок **Apple** в левом верхнем углу.
2. Выберите **Об этом Mac**.
3. Найдите строку **Память**.

Там будет указано, например: **8 ГБ, 16 ГБ, 32 ГБ**.

---

## Linux

### Способ 1: через терминал

1. Откройте **Терминал**.
2. Введите команду:

`free -h`

1. Посмотрите строку **Mem** — там будет указан общий объём оперативной памяти.

### Способ 2: через настройки

1. Откройте **Настройки**.
2. Перейдите в раздел **О системе** или **About**.
3. Найдите строку **Memory / Память**.

Как использовать Hugging Face для выбора модели ↴



[Hugging Face](#) — платформа, где собраны все модели для машинного обучения. Здесь можно не только скачать модели, но и проверить их совместимость с вашим железом.

Как это сделать:

1. Заходим в профиль на Hugging Face → *Edit Profile* → **Local Apps and Hardware**.

[+ New](#)

ZeroCoder123123

[Profile](#)

- Inbox (0)
- ⚙ Settings
- \$ Billing
- 🚀 Get PRO

Organizations

## ZeroCoder

ZeroCoder123123

[+ New](#)
[Edit profile](#)
[Settings](#)

Papers

Notifications

Jobs

Local Apps and Hardware

Gated Repositories

Content Preferences

Connected Apps

MCP

Theme

 Following 0

All

Models

Datasets

Spaces

Buckets

Papers

Collections



Spaces agents.md for your

Every Gradio Space now auto-se  
description that AI agents can re  
Code, Codex, or Pi) at it and they

 Hugging Face Changelog

- Добавляем ваше устройство: для MacBook — *Apple Silicon*, для Windows — провайдер и название вашей видеокарты (AMD / Nvidia / Intel / Qualcomm).

## Local Apps and Hardware

Set your preferred local inference apps, and the hardware you own to run them

**My Hardware** Publicly Visible

No items added yet

**Add new Hardware**

GPU CPU Apple Silicon

**GPU Provider**  
NVIDIA

**GPU Model**  
RTX 3060 Mobile

**VRAM** **Number of items**  
6 GB 1

Add item

3. После этого при просмотре моделей Hugging Face будет показывать, подходит ли конкретная версия модели под ваше железо.

Можно выбрать модель: google/gemma-4-E4B-it

gemma

**Models**

google/gemma-4-31B-it

OBLITERATUS/gemma-4-E4B-it-OBLITERATED

google/gemma-4-31B-it-assistant

google/gemma-4-E4B-it

google/gemma-4-26B-A4B-it

dealignai/Gemma-4-31B-JANG\_4M-CRACK

→ See all model results for "gemma"

**Datasets**

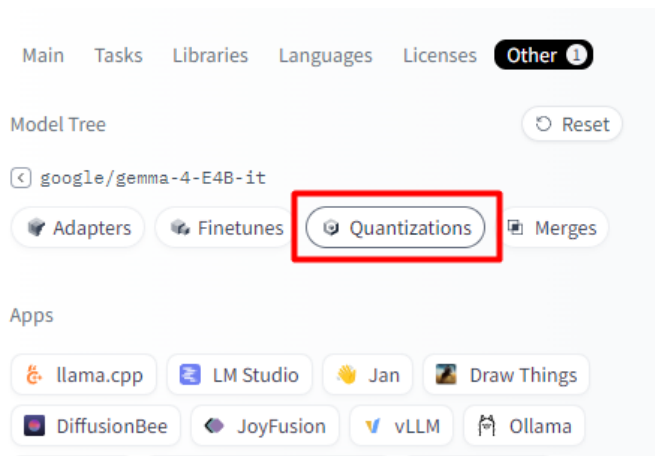


Если не знаете, какая у вас видеокарта — введите название и год выпуска вашего устройства в поисковик и посмотрите технические характеристики.

Квантизация: что это и как выбирать ↘



**Квантизация** — это степень сжатия модели. Чем выше квантизация, тем меньше модель сжата, тем она умнее, но тем больше весит и требует памяти.



Models 188

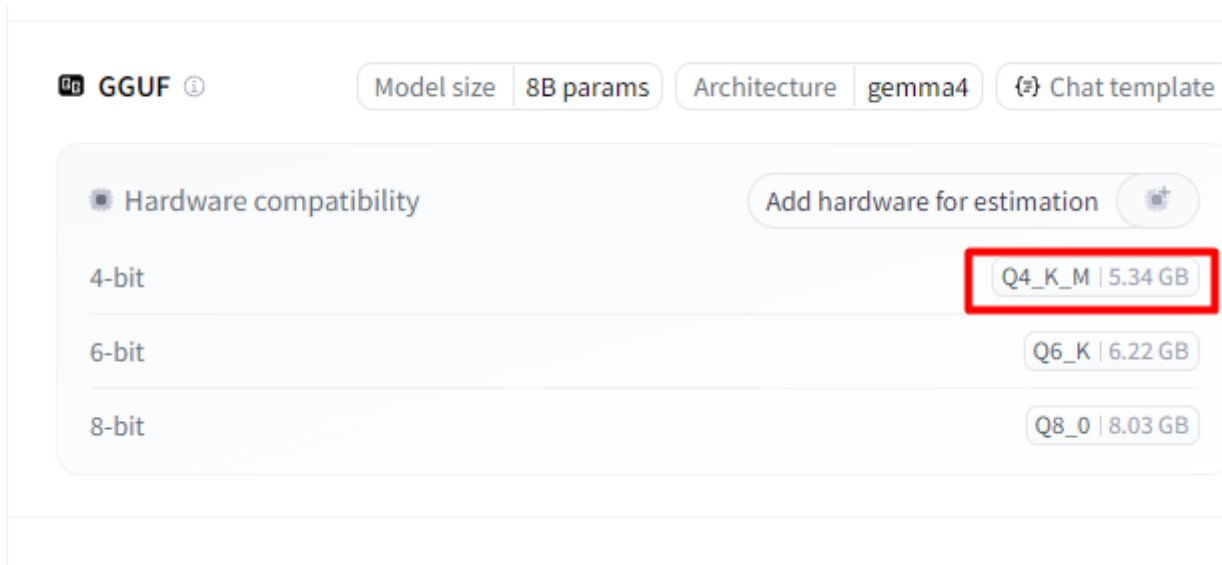
Filter by name

OBLITERATUS/gemma-4-E4B-it-OBLITERATED  
Text Generation • 8B • Updated about 1 month ago • 306k • 630

ggml-org/gemma-4-E4B-it-GGUF  
8B • Updated Apr 13 • 119k • 54

unsloth/gemma-4-E4B-it-MLX-8bit  
Image-Text-to-Text • 3B • Updated Apr 13 • 1.34k • 11

- Для запуска в Ollama нужны модели в формате **GGUF**.
- Рекомендуемая квантизация для большинства задач: **Q4\_K\_M** — это золотая середина между качеством и размером. Такую версию можно найти практически у любой модели.



- Если брать квантизацию ниже Q4 — модели становятся заметно «глупее» и работать с ними сложнее.

Чем выше квантизация — тем больше модель весит на диске. Q4\_K\_M для модели на 4B параметров занимает около 5 ГБ, более высокие квантизации — до 15 ГБ и больше.

**Рекомендация:** работайте с моделью **Gemma 4 на 4 миллиарда параметров, квантизация Q4\_K\_M**. Она запустится на 8 ГБ памяти и выше.

#### 4. Установка модели через Hugging Face и Ollama

Как скачать модель через Hugging Face ↘

Пример выбранной модели: [unsloth/gemma-4-E4B-it-GGUF](https://huggingface.co/unsloth/gemma-4-E4B-it-GGUF)

После того как мы выбрали модель и квантизацию на Hugging Face, скачиваем её в Ollama:

1. На странице модели выбираем провайдера квантизации (например, *unsloth*).
2. Выбираем конкретную квантизацию — **Q4\_K\_M**.

Downloads last month  
**1,211,045**



 **GGUF** ⓘ

Model size

8B params

Architecture

**gemma4**

 Chat template

 Hardware compatibility

Add hardware for estimation 

2-bit

UD-IQ2\_M | 3.55 GB

UD-Q2\_K\_XL | 3.76 GB

3-bit

UD-IQ3\_XXS | 3.72 GB

Q3\_K\_S | 3.86 GB

Q3\_K\_M | 4.06 GB

UD-Q3\_K\_XL | 4.59 GB

4-bit

IQ4\_XS | 4.72 GB

Q4\_K\_S | 4.84 GB

IQ4\_NL | 4.84 GB

Q4\_0 | 4.84 GB

Q4\_1 | 5.07 GB

**Q4\_K\_M | 4.98 GB**

UD-Q4\_K\_XL | 5.13 GB

5-bit

Q5\_K\_S | 5.4 GB

Q5\_K\_M | 5.48 GB

UD-Q5\_K\_XL | 6.66 GB

6-bit

Q6\_K | 7.07 GB

UD-Q6\_K\_XL | 7.46 GB

8-bit

Q8\_0 | 8.19 GB

UD-Q8\_K\_XL | 8.71 GB

16-bit

BF16 | 15.1 GB

3. Нажимаем **Use this model** → выбираем приложение **Ollama**.



File

gemma-4-E4B-it-Q4\_K\_M.gguf 4.98 GB

Download Use this model

| Metadata                          | Value               |
|-----------------------------------|---------------------|
| version                           | 3                   |
| tensor_count                      | 720                 |
| kv_count                          | 56                  |
| general.architecture              | gemma4              |
| general.type                      | model               |
| general.sampling.top_k            | 64                  |
| general.sampling.top_p            | 0.949999988079071   |
| general.sampling.temp             | 1                   |
| general.name                      | Gemma-4-E4B-It      |
| general.basename                  | Gemma-4-E4B-It      |
| general.quantized_by              | Unsloth             |
| general.size_label                | 7.5B                |
| general.license                   | apache-2.0          |
| general.license.link              | https://ai.google.d |
| general.repo_url                  | https://huggingface |
| general.base_model.count          | 1                   |
| general.base_model.0.name         | Gemma 4 E4B It      |
| general.base_model.0.organization | Google              |

Libraries

- llama-cpp-python

Notebooks

- Google Colab
- Kaggle

Local Apps

- llama.cpp
- LM Studio
- Jan
- vLLM
- Ollama**
- Unsloth Studio new
- Pi new
- Hermes Agent new

View all local apps

4. Hugging Face генерирует команду для терминала.
5. Копируем команду, вставляем в терминал → модель начинает загружаться.

Пример команды: `ollama run hf.co/unsloth/gemma-4-E4B-it-GGUF:Q4_K_M`

```
Microsoft Windows [Version 10.0.26200.7922]
(c) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

C:\Users\kvzay>ollama run hf.co/unsloth/gemma-4-E4B-it-GGUF:Q4_K_M
pulling manifest
pulling 519b9793ed6c: 35%
```

Когда вы вводите команду `ollama run <модель>`, после загрузки автоматически откроется чат с моделью. Если хотите только скачать модель без немедленного запуска — замените `run` на `pull`:

`ollama pull <ссылка на модель>`

```
C:\Users\kvzay>ollama run gemma4:e4b
pulling manifest
pulling 4c27e0f5b5ad: 100%
pulling 7339fa418c9a: 100%
pulling 56380ca2ab89: 100%
pulling f0988ff50a24: 100%
verifying sha256 digest
writing manifest
success
>>> send a message (/? for help)
```

9.6 GB  
11 KB  
42 B  
473 B



Проверьте свободное место на диске перед загрузкой. Модель на 4B параметров весит около 5 ГБ. Не забивайте диск под самый край — компьютеру нужно место для операций.



Если появляется ошибка, например, 500 Internal Server Error: unable to load model:

C:\Users\kvzay\.ollama\models\blobs\sha256-519b9793ed6ce0ff530f1b7c96e848e08e49e7af4d57bb97f76215963a54146d,  
то:

- Удалите модель: `ollama rm hf.co/unsloth/gemma-4-E4B-it-GGUF:Q4_K_M`
- Установи командой: `ollama run gemma4:e4b`

Или попробуйте подобрать другую модель, которая лучше подходит для вашего компьютера. Это можно сделать с помощью нейросети или Claude Code: они помогут выбрать локальную модель под ваши задачи и характеристики устройства.

Проверка установки ↴

После загрузки проверяем результат командой `ollama list` — модель должна появиться в списке установленных.

Чтобы остановить загрузку или прервать любую операцию в терминале — **Ctrl+C**.

## 5. Запуск модели и основные команды Ollama

Запуск модели в терминале ↴

`ollama run <название модели>`

После запуска сразу можно начать общаться с моделью прямо в терминале. Это полноценный чат с историей диалога: каждый ваш запрос и ответ модели попадают в **контекстное окно**, и модель помнит предыдущие сообщения до исчерпания контекста.

Команды для мониторинга модели ↴

После запуска модели откроем новое окно терминала и разберём две важнейшие команды:

`ollama ps` — статистика последнего запуска:

- Название и ID модели
- Размер модели
- Загрузка **GPU** или **CPU** — должно быть **100% GPU**. Если часть ушла на CPU — модель слишком тяжёлая для вашего железа, лучше её закрыть.
- Использованный **контекст** — по умолчанию у всех моделей в Ollama это **4096 токенов**. Это очень мало для агентной работы.

`ollama show <название модели>` — дефолтные характеристики модели:

- Количество параметров
- Контекстное окно
- Поддерживаемые возможности: **Tools, Thinking, Completion**

По умолчанию у всех моделей в Ollama контекстное окно — 4096 токенов. Для сравнения: у облачных моделей — от 120 тысяч до миллиона. 4096 токенов катастрофически мало для агентной работы — несколько запросов плюс обрабатываемый текст заполнят контекст полностью.



Работа с моделью, у которой дефолтный контекст 4096 токенов, внутри агентной среды практически невозможна. Этот параметр нужно увеличить вручную.

## 6. Создание Model File и создание новой модели

Что такое Model File и зачем он нужен ↴

**Model File** — это текстовый файл с настройками модели. Через него можно:

- Увеличить **контекстное окно**.
- Добавить поддержку **инструментов (Tools)**.
- Задать системное сообщение и другие параметры.

Нам нужно решить две задачи:

1. Увеличить контекст с **4096 до 32 000 токенов**.
2. Добавить параметры, которые сообщат Ollama о поддержке инструментов.

Чтобы посмотреть Model File для модели вводим:

```
ollama show <название модели> --modelfile
```

Как создать Model File ↴

Введите команду для создания Model File:

```
ollama show <название_модели> --modelfile > /<путь_к_папке>/<имя_файла>
```

Пример команды:

```
ollama show hf.co/NidAll/supergemma4-e4b-abliterated-Q4_K_M-GGUF:Q4_K_M --modelfile > /Users/kvzay/doc/llm-tools-32k
```

- Вместо <название модели> — название установленной модели (из ollama list).
- Вместо /путь/к/директории — путь к рабочей папке (можно узнать командой pwd).
- Вместо название-новой-модели — любое понятное имя, например llm-tools-32k.



Называйте новую модель так, чтобы сразу было понятно, что в ней изменено: например, supergemma-tools-32k.

Чтобы узнать путь к текущей папке, введите команду:

**Mac:**

```
pwd
```

**Windows CMD:**

```
cd
```

**Windows PowerShell:**

```
pwd
```

**Linux:**

```
pwd
```

Команда покажет путь к папке, в которой вы сейчас находитесь. Например:

```
C:\Users\kvzay\OneDrive\Документы\Ollamalocalmodel
```

Общая логика процесса:

1. Вводим nano и путь к этому файлу:

Для Mac:

% nano "C:\Users\kvzay\OneDrive\Документы\Ollamalocalmodel\llm-tools-32k"

Для Windows:

notepad "C:\Users\kvzay\OneDrive\Документы\Ollamalocalmodel\llm-tools-32k"

2. Открывается файл в блокноте (или можно открыть файл через Zed).

Редактирование Model File ↘

После создания файла открываем его в текстовом редакторе (VS Code или любом другом) и добавляем два параметра.

PARAMETER num\_ctx 32768

Это значение соответствует **32 000 токенов**. Для других размеров окна (64K, 128K) значения нужно искать в документации — важно использовать корректные числа, иначе настройка не сработает.

**Параметр 2 — поддержка инструментов.** Добавляем две строки, которые сообщат Ollama, что модель поддерживает Tool Calling:

TEMPLATE gemma4

PARAMETER render gemma4



Эти строки нужны только в том случае, если ollama show не показывает Tools для вашей модели. Если Tools уже есть — достаточно добавить только параметр контекста.

Создание новой модели из Model File ↘

Сохраняем файл и создаём новую модель командой:

ollama create <название новой модели> -f /путь/к/modelfile

Пример команды:



ollama create llm-tools-32k -f /Users/kvzay/llm-tools-32k

После успешного выполнения проверяем результат:

- ollama list — новая модель должна появиться в списке.
- ollama show <название новой модели> — в выводе должны появиться **Tools** и **Thinking**.

Если всё сделано правильно, вы увидите в `ollama show`: Thinking, Tools, Completion. Модель готова к работе внутри агентных сред.

## 7. Подключение локальной модели к агенту Goose



**Goose** — это open-source ИИ-агент, который запускается на компьютере и помогает выполнять задачи через нейросеть: писать код, работать с файлами, автоматизировать действия, анализировать данные, искать информацию и подключаться к внешним инструментам.

Проще говоря:

**Goose = локальный ИИ-помощник/агент для компьютера.**

Выбор агентной среды для локальной модели ↘

Для запуска локальных моделей внутри агентной среды можно использовать несколько инструментов: **Claude Code**, **OpenCode**, **Goose** и другие.

Мы работаем с [Goose](#) — он проще в настройке для локальных моделей, чем Claude Code или OpenCode, и удобнее при минимальном контексте.

Важное требование: у модели должен быть **Tool Use** или **Function Calling**. Без этого запустить модель внутри агентной среды не получится.

Установка и настройка Goose ↘

1. Устанавливаем [Goose](#) через CLI-команду (инструкция на сайте [goose.ai](#)).

- macOS

```
curl -fsSL https://github.com/aaif-goose/goose/releases/download/stable/download_cli.sh | bash
```

- Linux

```
curl -fsSL https://github.com/aaif-goose/goose/releases/download/stable/download_cli.sh | bash
```

- Windows

```
curl -fsSL https://github.com/aaif-goose/goose/releases/download/stable/download_cli.sh | bash
```

2. Вводим команды: `goose configure`

3. При первом запуске появляется **Setup Wizard** — мастер настройки.

4. Нажимаем *Configure Providers* → выбираем **Ollama**.

5. Оставляем дефолтные значения (localhost) → Enter.

6. Продвинутые настройки не нужны → выбираем No.

7. Из списка установленных моделей выбираем нашу новую модель (``llm-tools-32k``).

После настройки запускаем агента командой:

```
goose session
```

Тестирование агента с локальной моделью ↘

Запускаем Goose в нашей рабочей директории и проверяем работу:

- Агент автоматически читает системные инструкции из файла ``AGENTS.md`` (или `CLAUDE.md` — форматы совместимы).
- Модель умеет вызывать инструменты — например, **BraveSearch MCP**, настроенный ранее.

Тест: просим агента найти три последних видео на YouTube-канале и сохранить их в JSON-файл.

**Результат:** модель вызвала BraveSearch, нашла видео, но не додумалась самостоятельно создать файл — понадобился дополнительный запрос с явным указанием использовать инструмент записи.

Это показательный пример: маленькая локальная модель может запускать инструменты, но для выполнения задач из нескольких шагов ей нужны очень чёткие инструкции. Облачная модель выполнила бы это за один запрос.



Контекстное окно в 32 000 токенов — реальный лимит. Goose показывает использованный контекст. Следите за тем, сколько контекста осталось, особенно при сложных задачах.

## 8. Практическое применение: анализ клиентских отзывов

Практическое применение: анализ клиентских отзывов ↘

Такой сценарий пригодится, когда нужно автоматизировать рутинную работу с текстовыми данными:

- анализ и обработка данных;
- перевод;

- суммаризация и подготовка контента;
- лёгкие код-задачи;
- классификация отзывов, комментариев и обращений.

Маленькие локальные модели хорошо подходят для узких задач, где есть понятная структура входных данных и чёткие правила обработки.

Например, если у нас есть таблица с клиентскими отзывами, модель может пройти по каждой строке и определить:

- отзыв позитивный или негативный;
- нужно ли сохранить отзыв в базу;
- нужно ли позвонить клиенту;
- нужно ли отправить отзыв на дополнительный анализ;
- есть ли запрос на возврат.

Демонстрация: анализ тональности отзывов ↘

В примере используется CSV-таблица с клиентами онлайн-курса.

В таблице есть данные:

- имя клиента;
- ID;
- дата активации;
- источник;
- дата последнего контакта;
- есть ли контакт пользователя;
- комментарий к курсу.

Главная задача — обработать колонку с комментарием и автоматически добавить новые значения:

- sentiment — тональность отзыва;
- action — действие менеджера.

Пример логики:

- если отзыв позитивный — positive;
- если отзыв негативный — negative;
- если в отзыве есть запрос на возврат — возврат;
- если отзыв позитивный и есть контакт пользователя — звонок;
- если отзыв позитивный — сохранить;
- если отзыв негативный, но возврат не нужен — анализ.

Подготовка промпта для модели ↘

Для задачи создаём отдельный файл с промптом, например:

`prompts/sentiment-prompt.md`

В промпте описываем роль модели, входные данные, правила классификации и формат ответа.

Пример структуры промпта:

Ты — анализатор клиентских отзывов. На вход ты получаешь одну строку CSV с колонками: контакт у пользователя yes/no и comment — текст отзыва.

Твоя задача:

1. Проанализируй sentiment отзыва: positive или negative.
2. Определи действие менеджера по правилам ниже.
3. Верни на выходе только значения для колонок, без пояснений, заголовков и форматирования.

Правила определения действия:

- `возврат` — если в комментарии есть запрос на возврат средств. Это приоритетное действие.
- `звонок` — если отзыв позитивный и контакт = yes. Менеджер звонит для допродажи новых курсов.
- `сохранить` — если отзыв позитивный. Отзыв добавляется в базу.
- `анализ` — если отзыв негативный и нет запроса на возврат.

Формат выходных данных:

Верни одну строку в формате CSV:

`sentiment,action`

Где:

- `sentiment` — `positive` или `negative`;
- `action` — одно или несколько значений: `сохранить`, `звонок`, `анализ`, `возврат`.

Если действий несколько, разделяй их точкой с запятой.

Примеры входа и выхода:

Вход: yes, "Отличный курс, всё понравилось!"

Выход: positive,звонок;сохранить

Вход: no, "Классный материал, спасибо"

Выход: positive,сохранить

Вход: yes, "Ужасное качество, хочу вернуть деньги"

Выход: negative, возврат

Вход: no, "Плохо объяснили тему, разочарован"

Выход: negative, анализ

Правило приоритета: `возврат` всегда перекрывает другие действия, даже если в отзыве есть позитивные слова.

Теперь обработай следующую строку и верни только результат:

Такой промпт помогает локальной модели работать предсказуемо: она получает не абстрактную просьбу "проанализируй отзывы", а строгие правила и примеры.

Скрипт для обработки таблицы через Ollama ↴

Дальше создаём Python-скрипт, который будет построчно читать CSV-файл, отправлять каждую строку в локальную модель через Ollama и записывать результат обратно в таблицу.

В примере используется файл:

2026-05-01-clients-sample.csv

Скрипт лежит в папке:

scripts/analyze\_client\_sentiment.py

Внутри скрипта задаются основные параметры:

```
DEFAULT_MODEL = "hf.co/NidAll/supergemma4-e4b-abliterated-Q4_K_M-GGUF:Q4_K_M"
```

```
DEFAULT_OLLAMA_URL = "http://localhost:11434/api/generate"
```

```
DEFAULT_CONTACT_COLUMN = "Есть контакт"
```

```
DEFAULT_COMMENT_COLUMN = "Комментарий к курсу"
```

```
DEFAULT_SENTIMENT_COLUMN = "sentiment"
```

```
DEFAULT_ACTION_COLUMN = "action"
```

Что делает скрипт:

1. Открывает CSV-файл.
2. Берёт нужные колонки: контакт пользователя и комментарий.
3. Подставляет строку в промпт.
4. Отправляет запрос в локальную модель через Ollama.
5. Получает ответ модели.
6. Добавляет в таблицу новые колонки sentiment и action.
7. Сохраняет новый CSV-файл с результатами.



Проверка работы скрипта ↴

Для теста запускаем обработку первых четырёх строк:

```
python3 scripts/analyze_client_sentiment.py --limit 4
```

После запуска в терминале появляется результат:

Строка 1: positive,сохранить

Строка 2: positive,звонок;сохранить

Строка 3: negative,анализ

Строка 4: negative,анализ

Готово: 2026-05-01-clients-sample-analyzed.csv

Это значит, что локальная модель успешно обработала строки и записала результат в новый CSV-файл.

Что важно проверить после обработки ↴

После запуска нужно открыть итоговый файл и проверить:

- появились ли новые колонки sentiment и action;
- корректно ли определена тональность отзывов;
- правильно ли сработало правило про возврат;
- не путает ли модель позитивные и негативные комментарии;
- не добавляет ли модель лишние пояснения вместо короткого результата.

Если модель ошибается, нужно доработать промпт:

- добавить больше примеров;
- уточнить правила;
- явно указать формат ответа;
- добавить приоритеты действий;
- отдельно прописать спорные случаи.

Главные выводы ↴

Маленькие локальные модели не всегда хорошо справляются со сложными рассуждениями, но отлично подходят для простых повторяющихся задач.

Их можно использовать, когда нужно:

- обработать таблицу с отзывами;
- разложить комментарии по категориям;
- определить тональность;
- подготовить черновую классификацию;
- не отправлять клиентские данные в облако.

## 9. О минусах

Основные минусы ↴

### 1. Качество – может галлюцинировать, выдумывать факты на сложных задачах

2. Скорость – на слабом железе минуты вместо секунд
3. Контекст – большие документы могут «выпадать» из памяти
4. Железо – без видеокарты или мощного процессора не запустится
5. Стабильность – один и тот же запрос может дать разные ответы
6. Проверка – финальное решение всё равно за человеком

## 10. Итоги

Итоги ☑

- Узнали зачем и в каких случаях
- Узнали о минусах локальных LLM
- Научились скачивать и устанавливать через Ollama
- Поняли какое выбирать железо для работы с локальными LLM
- Обернули локальные LLM в агентов
- Написали скрипт для анализа сентимента

## Дополнительные материалы

Ссылки на сервисы из урока ☑

- [Ollama](#) -- приложение для запуска локальных нейросетевых моделей на компьютере.
- [Ollama: поиск моделей](#) -- каталог моделей, которые можно скачать и запустить через Ollama.
- [Hugging Face](#) -- платформа с моделями машинного обучения, где можно искать локальные модели, GGUF-версии и квантизации.
- [Gemma 4](#) -- семейство открытых моделей Google, которое можно использовать для локального запуска.
- [Goose](#) -- open-source ИИ-агент, который можно подключить к локальной модели через Ollama.
- [GitHub Goose Releases](#) -- страница с установочными файлами и CLI-скриптом Goose.
- [Python](#) -- язык программирования, который можно использовать для автоматизации обработки данных через локальную LLM.
- [Brave Search MCP](#) -- пример MCP-сервера, который можно подключить к агенту для поиска информации через инструменты.

Словарь занятия ☑

- ✓ **Локальная модель** -- нейросетевая модель, которая запускается прямо на компьютере пользователя и может работать без передачи данных в облачные сервисы.
- ✓ **Облачная модель** -- модель, которая работает на серверах провайдера и требует подключения к интернету.
- ✓ **LLM** -- большая языковая модель, которая умеет понимать и генерировать текст.
- ✓ **Ollama** -- приложение для скачивания, запуска и настройки локальных моделей на компьютере.
- ✓ **GGUF** -- формат моделей, адаптированный для локального запуска, в том числе через Ollama.
- ✓ **Hugging Face** -- платформа, где можно искать, сравнивать и скачивать модели машинного обучения.
- ✓ **Квантизация** -- степень сжатия модели. Чем сильнее сжатие, тем меньше модель весит, но тем ниже может быть качество ответов.
- ✓ **Q4\_K\_M** -- популярный вариант квантизации, который даёт баланс между размером модели и качеством работы.

- ✓ **Параметры модели / B** -- количество параметров модели в миллиардах. Например, **4B** означает около 4 миллиардов параметров.
- ✓ **CPU / ЦПУ** -- центральный процессор компьютера. Может запускать модель, но обычно медленнее GPU.
- ✓ **GPU / ГПУ** -- графический процессор, или видеокарта. Ускоряет работу нейросетей и помогает быстрее выполнять вычисления.
- ✓ **Оперативная память** -- память компьютера, которая используется для текущих задач и влияет на то, какие модели можно запускать локально.
- ✓ **GPU-память / VRAM** -- память видеокарты. Если модель помещается в неё полностью, она обычно работает быстрее.
- ✓ **Контекстное окно** -- объём текста, который модель может учитывать в рамках одной сессии.
- ✓ **Токены** -- части текста, на которые модель разбивает запросы и ответы. Контекстное окно измеряется в токенах.
- ✓ **Model File / Modelfile** -- текстовый файл с настройками модели в Ollama. Через него можно задать контекстное окно, системное сообщение и другие параметры.
- ✓ `num_ctx` -- параметр в Model File, который отвечает за размер контекстного окна модели.
- ✓ **Tool Calling / Function Calling** -- способность модели вызывать внешние инструменты, команды или функции.
- ✓ **Goose** -- локальный ИИ-агент, который может работать с файлами, кодом, инструментами и локальными моделями.
- ✓ **localhost** -- локальный адрес компьютера, через который программы могут обращаться к сервисам, запущенным на этом же устройстве.
- ✓ **CSV-файл** -- табличный файл с данными, который часто используют для выгрузок, списков клиентов, отзывов и других структурированных данных.

## Домашнее задание

Предлагаем два варианта заданий. Первый поможет развить ключевое умение по теме занятия, а второй даст свободу для **проектирования алгоритма для упрощения или даже автоматизации одной из твоих рутинных рабочих задач.**



Выбери только один вариант домашнего задания и выполни его сегодня.

Другой вариант, если захочешь, можешь проработать позже самостоятельно для закрепления материала.

Вариант 1. Базовый - повтори за экспертом ☞

Установи **Ollama**, скачай локальную модель и проверь, что она запускается на твоём компьютере.

### Что нужно сделать:

1. Установи **Ollama** на свой компьютер.
2. Открой терминал.
3. Проверь, что Ollama доступна:

ollama

1. Найди подходящую модель в каталоге **Ollama** или на **Hugging Face**.
2. Скачай и запусти модель командой:

ollama run <название\_модели>

3. Проверь список установленных моделей:

ollama list

4. Проверь параметры запущенной модели:

ollama ps

5. Задай модели 2–3 простых вопроса в терминале и оцени скорость ответа.

### Результат:

Локальная модель установлена, запускается через **Ollama** и отвечает на запросы в терминале.

### Материалы, которые нужно прикрепить к ДЗ:

- скриншот команды ollama list, где видно установленную модель;
- скриншот диалога с моделью в терминале;
- небольшой текстовый отчёт.

### Что написать в текстовом отчёте:

- какую модель ты выбрал(-а);
- почему выбрал(-а) именно её;
- сколько оперативной памяти на твоём компьютере;
- насколько быстро модель отвечала;
- подходит ли эта модель для твоих задач и почему.

Вариант 2. Продвинутый - адаптируй под свою задачу 📄

Настрой локальную модель для агентной работы и протестируй её на простой задаче автоматизации.

### Что нужно сделать:

1. Выбери локальную модель, которая подходит под твоё устройство.
2. Проверь её характеристики командой:

ollama show <название\_модели>

3. Создай **Model File**:

ollama show <название\_модели> --modelfile > <имя\_файла>

4. Открой файл в редакторе и добавь параметр контекстного окна:

PARAMETER num\_ctx 32768

5. Создай новую модель из Model File:

ollama create <название\_новой\_модели> -f <путь\_к\_файлу>

6. Проверь, что новая модель появилась в списке:

ollama list

7. Подключи новую модель к **Goose**.

8. Запусти сессию агента:

goose session

9. Дай агенту простую рабочую задачу, например:

- проанализировать 5–10 клиентских отзывов;

- разложить комментарии по тональности;
- выделить негативные отзывы;
- предложить действие менеджера для каждого отзыва.

### Результат:

Настроенная локальная модель с увеличенным контекстным окном и первый практический результат работы через агентную среду.

### Материалы, которые нужно прикрепить к ДЗ:

- скриншот команды ollama show <название\_новой\_модели>;
- скриншот команды ollama list, где видно новую модель;
- скриншот настройки или запуска **Goose**;
- скриншот результата работы агента;
- файл или текстовый фрагмент с результатом анализа;
- небольшой текстовый отчёт.

### Что написать в текстовом отчёте:

- какую модель ты использовал(-а);
- какие параметры изменил(-а) через **Model File**;
- удалось ли увеличить контекстное окно;
- подключилась ли модель к **Goose**;
- какую задачу ты дал(-а) агенту;
- насколько хорошо локальная модель справилась с задачей;
- где локальная модель может быть полезна в твоей работе.

Анкета обратной связи по уроку

Эл. адрес

Курс\*

Модуль\*

Урок\*

Понравился ли урок?\*

Понравился, все было понятно Неплохо, но не все получилось/не все понятно Не понравился

Обратная связь по уроку (не обязательно)

### Задание

Предлагаем два варианта заданий. Первый поможет развить ключевое умение по теме занятия, а второй даст свободу для **проектирования алгоритма для упрощения или даже автоматизации одной из твоих рутинных рабочих задач.**



Выбери только один вариант домашнего задания и выполни его сегодня.

Другой вариант, если захочешь, можешь проработать позже самостоятельно для закрепления материала.

Вариант 1. Базовый - повтори за экспертом ☞

Установи **Ollama**, скачай локальную модель и проверь, что она запускается на твоём компьютере.

#### Что нужно сделать:

1. Установи **Ollama** на свой компьютер.
2. Открой терминал.
3. Проверь, что Ollama доступна:

ollama

1. Найди подходящую модель в каталоге **Ollama** или на **Hugging Face**.
2. Скачай и запусти модель командой:

ollama run <название\_модели>

3. Проверь список установленных моделей:

ollama list

4. Проверь параметры запущенной модели:

ollama ps

5. Задай модели 2–3 простых вопроса в терминале и оцени скорость ответа.

#### Результат:

Локальная модель установлена, запускается через **Ollama** и отвечает на запросы в терминале.

#### Материалы, которые нужно прикрепить к ДЗ:

- скриншот команды ollama list, где видно установленную модель;
- скриншот диалога с моделью в терминале;
- небольшой текстовый отчёт.

#### Что написать в текстовом отчёте:

- какую модель ты выбрал(-а);
- почему выбрал(-а) именно её;
- сколько оперативной памяти на твоём компьютере;
- насколько быстро модель отвечала;
- подходит ли эта модель для твоих задач и почему.

Вариант 2. Продвинутый - адаптируй под свою задачу 📄

Настрой локальную модель для агентной работы и протестируй её на простой задаче автоматизации.

#### Что нужно сделать:

1. Выбери локальную модель, которая подходит под твоё устройство.
2. Проверь её характеристики командой:

ollama show <название\_модели>

3. Создай **Model File**:

ollama show <название\_модели> --modelfile > <имя\_файла>

4. Открой файл в редакторе и добавь параметр контекстного окна:

PARAMETER num\_ctx 32768

5. Создай новую модель из Model File:

```
ollama create <название_новой_модели> -f <путь_к_файлу>
```

6. Проверь, что новая модель появилась в списке:

```
ollama list
```

7. Подключи новую модель к **Goose**.

8. Запусти сессию агента:

```
goose session
```

9. Дай агенту простую рабочую задачу, например:

- проанализировать 5–10 клиентских отзывов;
- разложить комментарии по тональности;
- выделить негативные отзывы;
- предложить действие менеджера для каждого отзыва.

### Результат:

Настроенная локальная модель с увеличенным контекстным окном и первый практический результат работы через агентную среду.

### Материалы, которые нужно прикрепить к ДЗ:

- скриншот команды `ollama show <название_новой_модели>`;
- скриншот команды `ollama list`, где видно новую модель;
- скриншот настройки или запуска **Goose**;
- скриншот результата работы агента;
- файл или текстовый фрагмент с результатом анализа;
- небольшой текстовый отчёт.

### Что написать в текстовом отчёте:

- какую модель ты использовал(-а);
- какие параметры изменил(-а) через **Model File**;
- удалось ли увеличить контекстное окно;
- подключилась ли модель к **Goose**;
- какую задачу ты дал(-а) агенту;
- насколько хорошо локальная модель справилась с задачей;
- где локальная модель может быть полезна в твоей работе.

[Список тренингов](#)[Практический курс по вайб-кодингу на Claude Code](#)[Модуль 3: Цепочки задач и прикладные сценарии для бизнеса](#)

CLb02 Контент-завод для фриланса и бизнеса: реальные цепочки процессов

[Предыдущий урок](#)

Есть задание

[Следующий урок](#)

Дорогой студент! Интерфейсы современных приложений могут очень быстро меняться. Если вдруг то, что видишь ты, отличается от того, что демонстрирует эксперт, настолько, что тебе непонятно, куда нажимать — напиши об этом в учебный чат, указав номер урока и видео. Наш куратор обязательно поможет тебе разобраться, а мы постараемся как можно скорее актуализировать материал :)



Как проходить уроки эффективно: доступы, оплата и лимиты

**1** Доступ к мировым технологиям. Мы показываем лучшие мировые нейросети, но доступ к некоторым из них из РФ и/или других стран может требовать средств защищенного доступа. Мы действуем в рамках закона и НЕ консультируем по настройке соединения, но показываем эти сервисы, потому что они — лидеры рынка. Разобраться с доступом к ним — значит получить конкурентное преимущество.

**2** Весь курс можно пройти на бесплатных версиях инструментов. Мы показываем платные PRO-функции для ознакомления, чтобы ты мог(ла) оценить их потенциал и осознанно решить, нужны ли они тебе в будущем.

**3** Умное управление лимитами. Бесплатные попытки ограничены, поэтому регистрируйся в сервисе только перед выполнением ДЗ. Если лимиты закончились, а попрактиковаться еще хочется — часто помогает регистрация нового аккаунта на другую почту.

**4** Сначала смотри, потом делай. Сначала посмотри урок полностью, чтобы понять логику, и только потом трать генерации. Это сэкономит твоё время и попытки.

Так ты освоишь самые мощные инструменты бесплатно и без лишних нервов! 🚀

---

Результат дня

✅ Поймёшь, как выстроить контент-завод из одного источника в разнотипный контент для нескольких площадок

✅ Научишься создавать и адаптировать **скилл** в Claude Code для автогенерации постов под разные соцсети

✅ Узнаешь, как подключить **MCP-сервер fal.ai** для генерации изображений прямо из агента

✅ Настроишь автопубликацию в соцсети через сервис Postiz.

✅ Поймёшь, как масштабировать архитектуру и применять её для личного бренда, фриланса и клиентских задач

▶ Время чтения и просмотра: ~60 минут

🕒 Время выполнения задания: ~80 минут



Под каждым видео в уроке есть текстовая расшифровка, чтобы у тебя была возможность лучше запомнить новые термины, прочитать пояснения и инструкции.

Чтобы прочитать текст, нажимай на него. Текст скрыт в строках подчёркнутых пунктирной линией со значком "⌵" в конце строки.

---

*Упоминаемая в данном уроке социально сети Instagram является продуктом компании Meta, которая 21 марта 2022 года была признана экстремистской организацией на территории Российской Федерации. Деятельность Meta по реализации продуктов Instagram и Facebook в России запрещена.*

*Данный урок носит исключительно информационный и образовательный характер. Мы не призываем к использованию указанных ресурсов и не занимаемся их продвижением.*

**Теория на сегодня**

## **1. Контент-завод: архитектура и логика работы**

Что такое контент-завод и зачем он нужен ⌵



Сегодня мы рассмотрим реальный пример использования архитектуры **Claude Code** для решения конкретной задачи: из одного источника создать большой набор разнотипного контента, который можно автоматически публиковать на разные площадки.

Задача пустого экрана знакома каждому: нужно что-то написать, создать, суммаризировать или переписать. Агенты, и в частности **Claude Code**, могут значительно ускорить и улучшить этот процесс.

На этом уроке мы разберём:

- зачем выстраивать контент-завод
- как его выстраивать внутри Claude Code на основе архитектуры **скиллов**
- какие инструменты для этого нужны
- какую архитектуру Claude Code выбрать

Как работает цепочка контент-завода ↘

Верхнеуровневая логика такова: есть **один источник**, который мы хотим переработать в посты и визуальный контент для **Telegram**, соцсетей и других площадок.

Шаги:

1. Найти источник
2. Объяснить агенту, как его обработать и какие форматы контента нужны
3. Создать файлы на локальном устройстве
4. Опубликовать

**Источником** может быть абсолютно что угодно: ссылка на статью, ролик, ваш текст, набросок, идея, пост в одной из соцсетей.



Важно заранее определить **рабочие директории и пути файлов** — куда они сохраняются. Это обеспечивает стабильность всей системы.

Архитектура цепочки внутри Claude Code ↘

Цепочка контент-завода состоит из следующих звеньев:

- **Источник** — внешние данные (статья, ссылка, текст)
- **План обработки** — описывается через **скиллы**, которые мы уже умеем создавать
- **Апрув\* и публикация** — здесь остаётся ручной этап проверки, потому что мы хотим убедиться, что агент правильно выполнил работу. После этого используем инструмент **Postiz** для публикации.

*\*Апрув — это одобрение или согласование. От английского **approve** — «одобрять».*

Полной автоматизации без участия человека здесь нет намеренно — мы хотим проверять качество результата перед публикацией.

## 2. Скилл для генерации контента: структура и настройка



Эксперт использует те социальные сети, с которыми постоянно работает для продвижения своего канала. Ты можешь выбрать другие комфортные для себя площадки, например **ВКонтакте**, и адаптировать навык под свои задачи.

Файл со скиллом:

[SKILL.md](#)

Структура скилла AI-News to Social ↘

Мы открываем визуальный редактор и проектные настройки Claude Code. В папке **скиллы** находим скилл **AI-News to Social** — он создан и немного адаптирован под задачу контент-завода.

Структура скилла:

1. **Роль агента** — SMM-редактор курса по агентам и автоматизации. Работает на русском языке, объясняет новости простыми словами, показывает практическую пользу для новичков, не разгоняет хайп.
2. **Индут (вход)** — один источник: статья, новость, пост, фрагмент рассылки, ссылка плюс цитата, короткое описание. Если источников несколько, агент просит выбрать один или предлагает обработать по очереди.
3. **Порядок работы** — прочитай источник целиком, создай **JSON-файл** в заданной директории (папка **posts**).
4. **Формат результата** — агент создаёт **JSON строго по схеме**: каждая площадка — объект с обязательным строковым полем ``content``. Опционально на верхнем уровне — метадата об источнике.



Мы не пишем промпты скиллов вручную — описываем агенту логику, что нужно произвести, и просим его написать промпт для самого себя. Это сильно ускоряет процесс.

Примеры и стили для каждой площадки ↘

Чтобы агент генерировал текст в нужном стиле, ему задаётся **базовый стиль проекта** — применяется ко всем платформам: голос, принципы письма, признаки хорошего текста.

Далее по каждой площадке указывается:

- структура поста (объём, эмодзи, хештеги)
- **конкретный пример поста**, на который агент опирается при генерации

Площадки в нашем скилле:

- **Telegram** — развёрнутый пост, эмодзи, хештеги, структура + пример
- **Instagram\*** — своя структура + пример
- **X (Twitter)** — до 280 символов + пример
- **TikTok** — caption (описание к видео) + пример



Вы можете добавлять другие площадки, менять примеры и адаптировать скилл под свои задачи — структура гибкая.

*\*Продукт компании Meta, признанной в РФ экстремистской организацией.*

JSON-схема и Image Prompt ↘

Агент заполняет **JSON-схему** строго по заданным ключам: платформа → контент → содержание. Использование **JSON-формата** важно, потому что:

- он машиночитаемый
- его можно передавать в скрипты и другие инструменты
- агенты хорошо умеют его заполнять
- это позволяет масштабировать архитектуру в будущем

Дополнительно агент генерирует **Image Prompt** для модели генерации изображений. Промпт описывает конкретную сцену, узнаваемый мем-формат или визуальный образ по новости. Результат записывается в поле ``image_prompt`` на верхнем уровне JSON.

Так в одном JSON-файле оказывается весь контент под все площадки плюс промпт для генерации картинки — всё готово для дальнейшей передачи.

Запуск скилла и результат ↴

Запускаем агента в терминале, указываем скилл через слэш (/AI-News to Social) и передаём ссылку на источник.

После выполнения задачи агент создаёт файл в папке `posts`. В нём:

- **метадата** об источнике
- **Image Prompt** для генерации картинки
- готовый контент для каждой площадки в нужном формате

Например, для **X** — один твит до 280 символов, для **Telegram** — развёрнутый пост, для **TikTok** — описание к видео.

### 3. Генерация картинок через MCP-сервер fal.ai

Подключение MCP-сервера fal.ai ↴



К сожалению, нейросетей, которые бесплатно генерируют изображения, немного, и их качество не всегда соответствует желаемому результату. Именно поэтому в уроке для генерации картинок мы используем платный сервис. Оплата в нём возможна только зарубежной картой или *через посредников\**.

**Вы можете оставить «контент-завод» на уровне текстовых постов, чтобы разобраться в самой логике его создания, либо использовать другие доступные варианты.**

Например, можно поискать другие **MCP-серверы** для генерации изображений или использовать **ProxyAPI**. Его можно пополнить российской картой, а внутри есть модели, которые поддерживают генерацию картинок.

Единственный полностью бесплатный вариант — локально развернуть на компьютере нейросеть для генерации изображений, например Stable Diffusion, и затем подключить её к проекту. В этом случае генерация не будет требовать оплаты внешнего сервиса, но для стабильной работы понадобится достаточно мощный компьютер, желательно с хорошей видеокартой.

*\*рекомендация от наших студентов*

Что такое Proxy API и как зарегистрироваться ↴

**Proxy API** — сервис, который позволяет подключать API различных нейросетей, в том числе тех, которые недоступны напрямую с территории России. Регистрация через email.

Для пополнения баланса доступны:

- **банковская карта** (российская);
- **СБП**.

Минимальная сумма пополнения — **200 рублей**. Для тестового проекта этого будет достаточно надолго.

В разделе **«Ключи API»** создаём новый ключ (максимум 3 ключа на аккаунте). Ключ нужно сохранить — он понадобится для подключения к проекту.

В разделе **«Цены»** можно посмотреть все доступные модели и стоимость за миллион токенов: **GPT-4, GPT-4 mini, Claude Opus, Gemini Flash** и другие. Чем новее модель — тем дороже. Для учебных проектов подходят более доступные варианты.



Никогда не публикуй свои API-ключи в открытом доступе и не отправляй их посторонним. В учебных целях мы добавляем ключ в диалог с ассистентом для экономии времени, но в реальных проектах ключ нужно хранить в защищённом файле конфигурации.

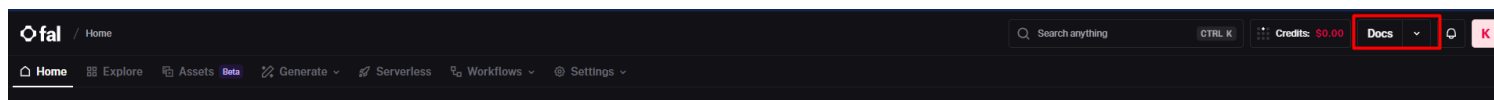
Для генерации картинок мы используем **MCP-сервер** сервиса **fal.ai**.



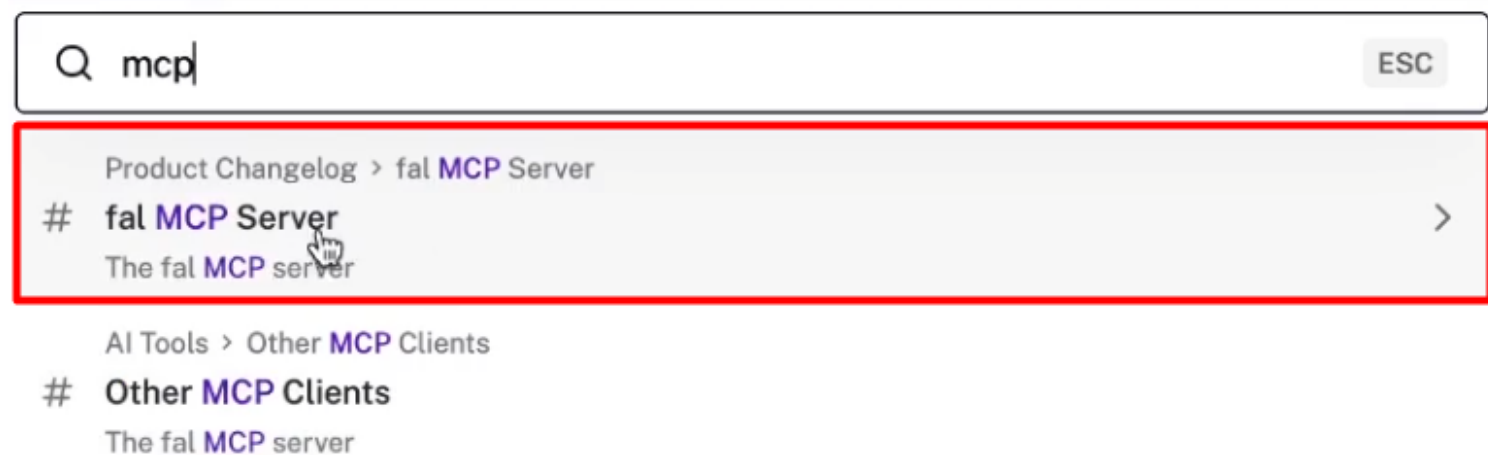
[fal.ai](https://fal.ai) — платный сервис. Нужно зарегистрироваться и пополнить баланс для отправки запросов.

Порядок подключения:

1. Зарегистрироваться на **fal.ai**
2. Перейти в раздел **Docs**.



3. В поиске найти MCP → открыть страницу **fal MCP Server**



4. Скопировать конфигурацию MCP-сервера

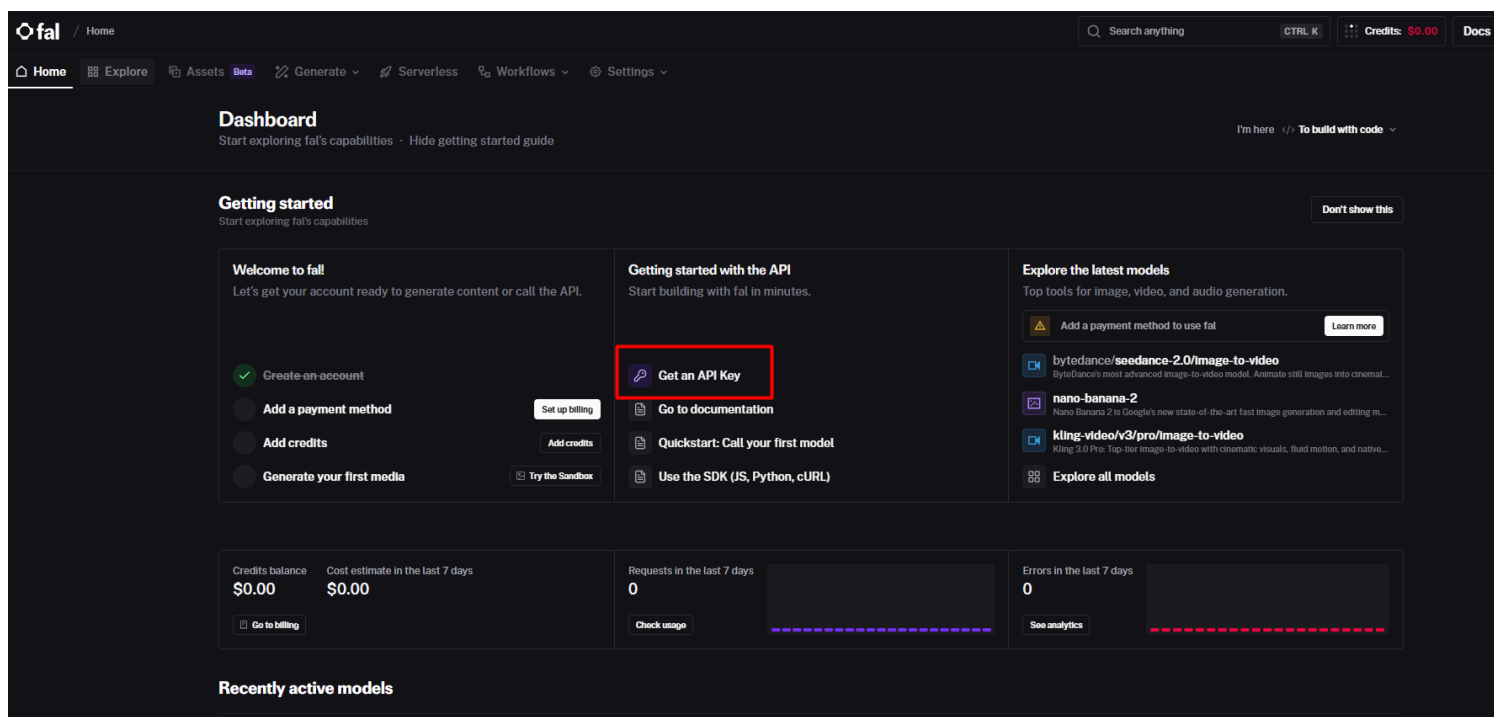
5. `claude mcp add --transport http fal-ai \`

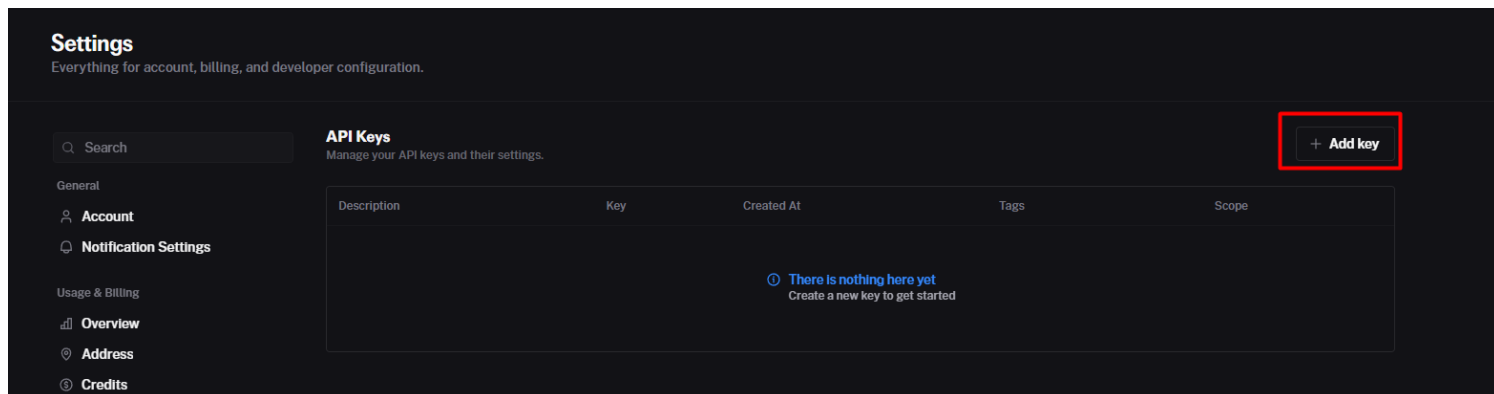
6. `https://mcp.fal.ai/mcp \`

`--header "Authorization: Bearer $FAL_KEY"`

5. Вставить её в нужную папку проекта, заменив переменную на ваш **API-ключ**

6. API-ключ находится в разделе [Профиль](#) → **Dashboard** → **Get an API Keys** → **Add key**





После вставки конфигурации файл `~/claude.json` обновится автоматически — MCP-сервер добавлен.



API-ключи никому не показывайте и удаляйте их после демонстраций.

Генерация изображения через агента ↘

После подключения MCP-сервера мы просим агента сгенерировать картинку на основе **Image Prompt**, созданного на предыдущем шаге.

Используй **MCP-сервер Fal AI**, чтобы сгенерировать изображение по промпту, который ты получил на предыдущем шаге. Используй `gpt-image-2`.

В запросе важно явно указать:

- использовать **MCP-сервер fal.ai**
- модель — **GPT Image 2** (или другую нужную)
- конкретные инструменты внутри MCP (это повышает стабильность работы)



Чёткий промпт, сгенерированный агентом на предыдущем шаге, повышает стабильность генерации изображения — мы уже знаем, что произойдёт, потому что заранее описали формат.

Агент отправляет промпт в MCP, опрашивает статус генерации и, как только изображение готово, сохраняет его в папку `posts`. URL изображения также добавляется в **JSON-схему**.

Самоисправление агента и финальный JSON ↘

В процессе работы агент может столкнуться с ошибками (например, некорректный JSON-файл). В этом и состоит сила агентных решений: агент **самостоятельно перепроверяет** свою работу, находит обходной путь и исправляет ошибку без нашего участия.

По итогу в папке `posts` собирается **полный JSON-файл** со всеми ассетами:

- тексты постов для каждой площадки
- URL сгенерированного изображения
- метадата источника

Этот файл готов к передаче на следующий этап — публикацию.

Что получилось в результате ↘

После выполнения всех шагов мы имеем:

- **сгенерированный контент** для всех площадок (Telegram, X, TikTok и др.)
- **сгенерированное изображение**, сохранённое в нужной папке
- **JSON-файл** с обновлённым полем ``image_url``

Всё это собрано в одном машиночитаемом файле и готово к автоматической публикации. Агент самостоятельно добавил URL картинки в схему после её генерации.

Мы просто общаемся с агентом в терминале — а он собирает весь пакет контента за нас.

#### 4.1 Публикация через Postiz (бесплатный период, нужна зарубежная карта)



[Postiz](#) — сервис для автопостинга в соцсети. Он упрощает публикацию, потому что настройка аутентификации вручную для каждой соцсети — сложный и долгий процесс.

Что такое Postiz и как его подключить ↘



Postiz — платный сервис. Есть **бесплатный пробный период на 7 дней**. Для регистрации потребуется иностранная карта.

Как подключить соцсети:

1. Войти в Postiz
2. Нажать **«Добавить канал»**
3. Выбрать нужную соцсеть (X, Telegram, YouTube и др.) и пройти аутентификацию

Для **X** — одна кнопка, автоматическая авторизация через залогиненный аккаунт.

Для **Telegram**:

1. Нажать **Connect Telegram**
2. Добавить бота **Postiz News Bot** в вашу группу или канал
3. Отправить в чат команду из инструкции Postiz
4.  Обязательно выдать боту **права администратора** в канале — без этого публикация не пройдёт

Установка скилла Postiz в Claude Code ↘

У Postiz есть два варианта интеграции с агентом: **МСП** и **скилл**. Оба работают, но скилл показал себя чуть стабильнее.



**CLI-инструмент** (Command Line Interface) можно упаковать в скилл — внутри него прописано то же самое, что и в МСП. Поэтому они взаимозаменяемы.

Установка скилла Postiz:

```
npm install -g postiz
```

```
postiz auth login
```

```
npx skills
```

При запуске npx skills:

- отметить **Claude Code** (и другие ваши агенты при необходимости)
- выбрать **проектную** установку
- выбрать **симлинк**

После установки скилл **Postiz** появится в проектных настройках Claude Code. В нём подробно описано, как агенту работать с Postiz и совершать публикации.



После установки нового скилла нужно **заккрыть и заново открыть Claude Code**, чтобы скилл применился.

Публикация контента через агента ☞

Просим агента опубликовать контент:

- Указываем скилл Postiz через слэш
- Указываем целевые площадки (Telegram, X и др.)
- Указываем путь к JSON-файлу с контентом
- Указываем, что нужно опубликовать **с картинкой** (агент сам определит, передавать URL или бинарный файл)

Агент последовательно:

1. Читает JSON-файл с контентом
2. Загружает скилл Postiz
3. Проверяет аутентификацию
4. Загружает картинку на сервер Postiz
5. Совершает публикацию с нужными ID каналов

Результат публикации можно проверить в разделе **Календарь** внутри Postiz — там отображаются все опубликованные посты.

Результат публикации можно проверить в разделе **Календарь** внутри Postiz— там отображаются все опубликованные посты.



Если Telegram не принял публикацию — проверьте, выданы ли боту права администратора в канале.

#### 4.2 Контент-завод внутри Claude Code: автопостинг во ВКонтакте

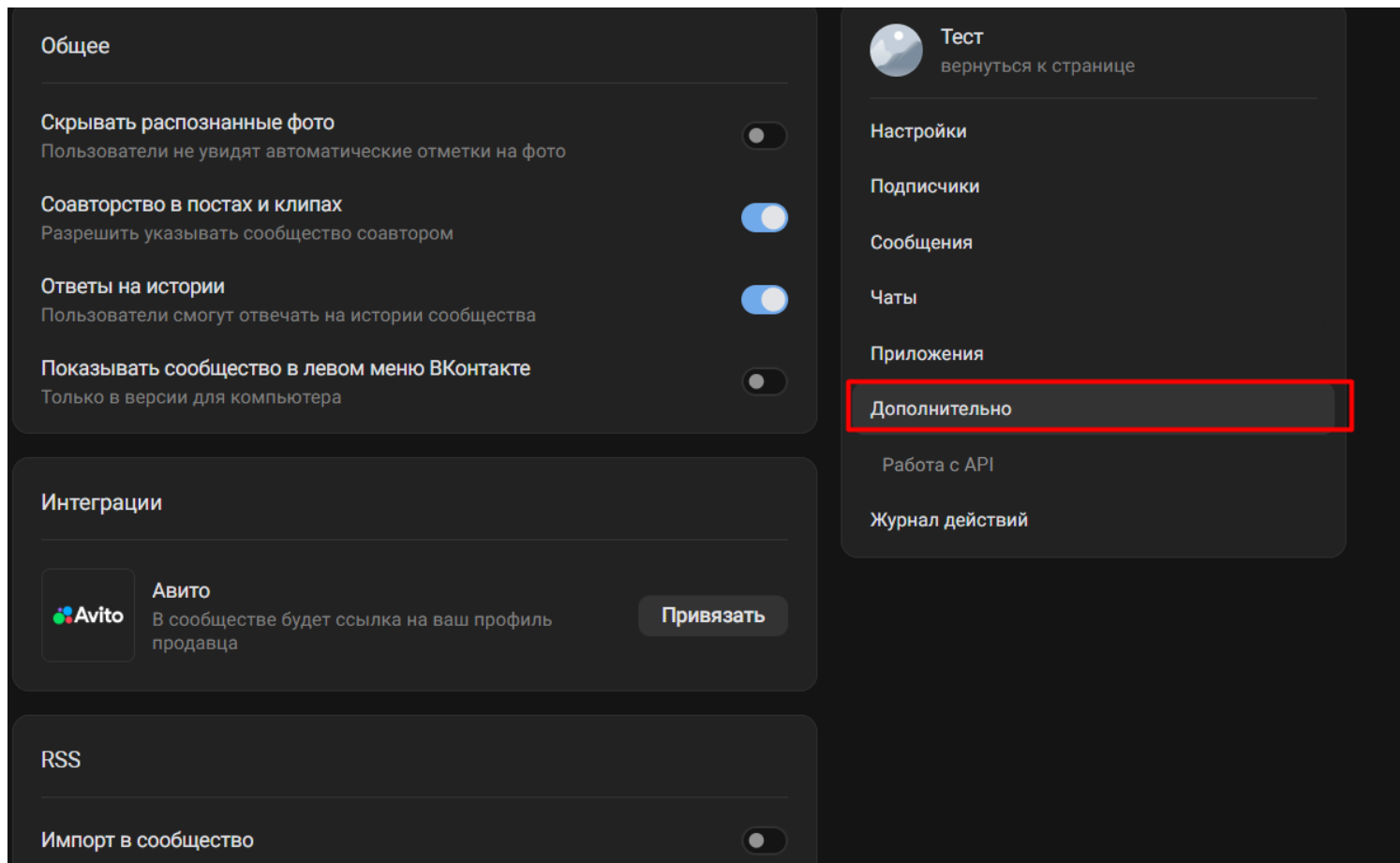
Подготовка: создание сообщества ВКонтакте и получение ключей ☞

Прежде чем запускать проект, нам нужно подготовить все необходимые данные на стороне **ВКонтакте**.

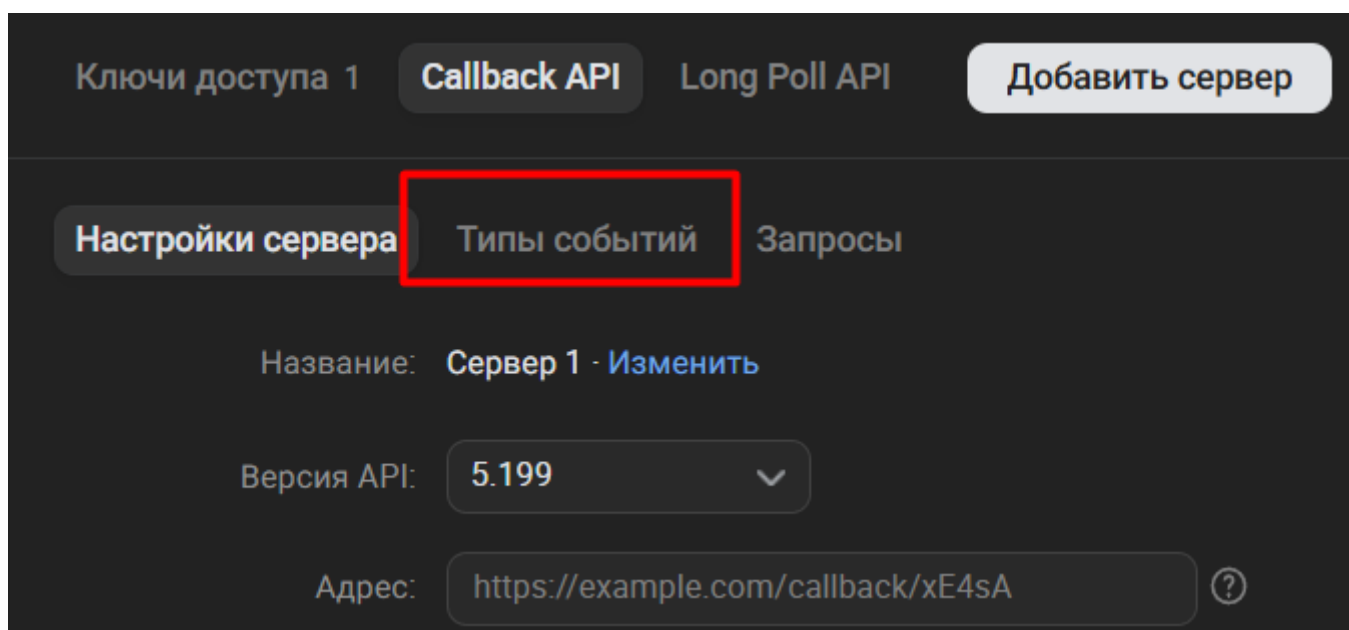
Первый шаг — создать сообщество. Переходим во вкладку **«Сообщества»** и нажимаем **«Создать сообщество»**. Заполняем базовую информацию: название, тематика. На этапе тестирования можно не углубляться — главное, чтобы сообщество было создано.

Далее нам нужно настроить **API-доступ**:

1. Переходим в **«Управление» → «Дополнительно» → «Работа с API»**



2. Открываем вкладку **«Callback»** — здесь хранится всё, что связано с внешним взаимодействием: публикация записей, добавление фотографий и т.д.
3. Переходим в **«Типы событий»** и расставляем галочки. Обязательно включаем **записи на стенах**, можно добавить фотографии и комментарии. В тестовом сообществе можно поставить все галочки — это не навредит.



В разделе API также доступны обсуждения, товары, работа с пользователями, аудиозаписи — это задел на будущие проекты.

Затем переходим в **«Ключи доступа»** и нажимаем **«Создать ключ»**. Даём все разрешения, подтверждаем действие через телефон. Появившийся ключ сразу копируем и сохраняем — повторно его не покажут.



После этого переходим в «**Настройки**» сообщества и копируем **ID сообщества**. Он нам понадобится при настройке проекта.



Для публикации постов с фотографиями нужен дополнительный токен — он отличается от основного ключа API. Ссылка для его получения будет в материалах урока. Если планируешь публиковать только текст — этот токен не нужен.

Подключение Proxy API для генерации картинок ↴

Если мы хотим, чтобы к каждому посту автоматически генерировалась картинка, подключаем [Proxy API](#) — платный сервис, который можно оплатить с российской карты.

Генерация картинок — платное удовольствие. Минимальный порог пополнения — около 200 рублей.

Порядок действий:

1. Регистрируемся в сервисе (ссылка будет в материалах урока)
2. В разделе с ценами выбираем модель генерации изображений — берём самую простую и копируем её название: оно понадобится в промпте
3. Переходим в раздел «**Ключи API**», нажимаем «**Создать новый ключ**», даём ему название (например, *автопостинг*), нажимаем «**Создать**»
4. Копируем появившийся ключ и сохраняем



API-ключ — конфиденциальная информация. Никому его не передавай и не демонстрируй: если ключ попадёт к посторонним, они смогут потратить весь баланс на твоём аккаунте.



Если генерация картинок пока не нужна — просто убери из промпта все строки, связанные с Proxy API. Для текстового автопостинга достаточно только **VK API-ключа** и **ID сообщества**.

Составление промпта и запуск проекта в Claude Code ↴

Когда все ключи собраны, переходим в **Claude Code** и отправляем подготовленный промпт.

Задача для Claude Code звучит так:

Сделай мне инструмент для автопостинга в ВКонтакте.

Я присылаю тебе ссылку на страницу с новостями. Ты выбираешь 3 разные статьи, читаешь каждую и с помощью скилла ai-news-to-social пишешь по ней пост для ВК — с крючком, личным мнением и хэштегами. К каждому посту скилл придумывает описание для генерации картинки. Текст должен генерироваться внутри Claude Code по скиллу, не подключай для этого дополнительные нейросети.

Потом всё это публикуется в мою группу ВКонтакте автоматически — с уникальной картинкой к каждому посту.

Расписание выхода постов:

Картинки генерируются через ProxyAPI (gpt-image-1-mini). Публикация через VK API. Всё хранится в .env файле.

Данные:

- PROXY\_API\_KEY модель:-mini ключ:
- VK\_ACCESS\_TOKEN
- VK\_GROUP\_ID
- oauth-токен

Перейдите по

ссылке: [https://oauth.vk.com/authorize?client\\_id=CLIENT\\_ID&display=page&redirect\\_uri=https://oauth.vk.com/blank.html&scope=wall,groups,photos,offline&response\\_type=token&v=5.199](https://oauth.vk.com/authorize?client_id=CLIENT_ID&display=page&redirect_uri=https://oauth.vk.com/blank.html&scope=wall,groups,photos,offline&response_type=token&v=5.199)

В промпт включаем:

- **Скилл** для генерации текста поста (указываем его название)
- **Proxy API** — для генерации картинок (модель, ключ)
- **VK API** — для публикации (ключ, ID сообщества, токен для фото)
- **Расписание** публикаций



Claude Code уточнит расписание, если ты забудешь его указать. Для теста удобно задать: *3 поста с паузой в одну минуту* — так можно быстро проверить, что всё работает.

Для продакшена можно настроить расписание вида «каждый вторник», «3 поста в неделю» и т.д. — Claude Code поддерживает любые формулировки.

Важный момент по архитектуре проекта: **текст поста генерируется внутри Claude Code по скиллу**, а не через внешнюю модель по API. Картинка генерируется отдельно через Proxy API. Всё это затем передаётся на публикацию во ВКонтакте.



Claude Code может попытаться самостоятельно подключить текстовую модель через Proxy API для генерации постов. Если это произошло — останавливаем выполнение (Ctrl+C) и явно указываем в следующем сообщении: *«Не используй дополнительную модель для генерации текста. Генерация текста поста должна быть по скиллу [название скилла] внутри Claude Code»*.

Работа проекта и результат ↴

После отправки ссылки на сайт с новостями Claude Code запускает полный цикл:

1. Выбирает три разные статьи
2. Читает каждую статью с помощью скилла
3. Формирует пост по требованиям скилла: нужное количество символов, хэштеги, вопрос в конце для вовлечённости
4. Генерирует картинку через Proxy API
5. Публикует пост во ВКонтакте по расписанию

В итоге за несколько минут в группе появляются три полноценных поста с картинками и текстом — без какого-либо ручного участия.



Когда Claude Code уходит в «фоновый режим» — это нормально. Он продолжает работать и публиковать посты, даже если внешне кажется неактивным. Следи за тем, чтобы командная строка оставалась открытой.

Дальнейшие направления развития проекта:

- Адаптировать под **другие социальные сети** (Telegram и др.)
- Превратить в **готовый скилл** с информацией про автопостинг
- Оформить как **полноценное приложение** с интерфейсом и публикацией на сервере — тогда оно будет работать постоянно, а не только пока открыт Claude Code

## 5. Где применять контент-завод

## Сценарии использования ↴

**Личный бренд.** Если вы развиваете личный бренд в нескольких соцсетях, postiz позволяет публиковать контент сразу на все площадки — даже если вы пишете тексты сами. Просто просите агента опубликовать пост туда, куда подключен postiz.

**Клиентский контент.** Если клиент просит генерировать много контента для разных площадок — идеально. Вписываете нужные площадки, форматы и примеры постов в скилл. Шаблон скилла уже есть — берите и адаптируйте (или просите агента адаптировать его за вас).

**SEO-оптимизация статей.** Если нужно создать 5, 10 или 20 вариаций одной статьи под разные ключевые слова и заголовки — архитектура контент-завода подходит и для этого.

**Описания на маркетплейсы.** Быстрая генерация разных описаний для разных товаров плюс картинки для карточек.

**Массовый постинг.** На разных маркетплейсах, сайтах или в соцсетях — генерация большого количества постов в едином процессе.

В следующих уроках мы разберём, как подключить субагентов и сделать этот процесс ещё более автоматизированным — с минимальным участием человека.

## 6. Домашнее задание

### Дополнительные материалы

Ссылки на сервисы из урока ↴

- **fal.ai** — платформа для генерации изображений и видео через API и MCP: [fal.ai](https://fal.ai)
- Postiz — сервис для автопостинга в соцсети (X, Telegram, YouTube и др.): поиск по запросу postiz в Google

### Словарь занятия ↴

- **Контент-завод** — система, которая из одного источника автоматически генерирует разнотипный контент для нескольких площадок
- **Скилл (Skill)** — инструкция-файл для агента, описывающая его роль, логику работы и формат результата; аналог системного промпта, оформленного как переиспользуемый модуль
- **JSON** — машиночитаемый формат данных; используется для хранения и передачи структурированного контента между агентом, скриптами и внешними сервисами
- **JSON-схема** — заранее заданная структура JSON-файла с конкретными ключами, которую агент обязан соблюдать при заполнении
- **Image Prompt** — текстовое описание изображения, которое агент генерирует для передачи в модель генерации картинок
- **MCP (Model Context Protocol)** — протокол, позволяющий подключать внешние сервисы и инструменты к агенту
- **CLI-инструмент** — программа, управляемая через командную строку (терминал); может быть упакована в скилл
- **fal.ai** — платный сервис для генерации изображений и видео с поддержкой MCP
- Postiz — платный сервис-агрегатор для автоматической публикации контента в соцсети; имеет MCP и скилл для Claude Code
- **Автопостинг** — автоматическая публикация контента в соцсети без ручного участия
- **Апрув** — ручной этап проверки результата агента перед публикацией
- **Workaround** — обходной путь; способ решить задачу альтернативным методом при возникновении ошибки

### Домашнее задание

Предлагаем два варианта заданий. Первый поможет развить ключевое умение по теме занятия, а второй даст свободу для **проектирования алгоритма для упрощения или даже автоматизации одной из твоих рутинных рабочих задач**.



Выбери только один вариант домашнего задания и выполни его сегодня.

Другой вариант, если захочешь, можешь проработать позже самостоятельно для закрепления материала.

Вариант 1 (Базовый — повтори за экспертом) ☞

Возьми шаблон скилла **AI-News to Social** из урока и адаптируй его под себя:

1. Измени **роль агента** — опиши свою тематику и аудиторию
2. Укажи **свои площадки** (например, Telegram, VK или другие)
3. Добавь **примеры своих постов** в скилл для каждой площадки
4. Найди любую актуальную статью или новость по вашей теме и запусти скилл через Claude Code
5. Проверь, что агент создал корректный **JSON-файл** с контентом для каждой платформы в папке post`

**Результат:** готовый JSON-файл с адаптированным контентом для минимум двух площадок.

Скрины, которые присылаем, как выполнено домашнее задание.

- Адаптированный скилл AI-News to Social: роль агента, тематика, аудитория и площадки.
- Запуск скилла в Claude Code с выбранной новостью или статьёй.
- Папка post и содержимое JSON-файла с контентом минимум для двух площадок.

Вариант 2 (Продвинутый — адаптируй под свою задачу) ☞

Выстрой полную цепочку контент-завода под реальную задачу:

1. Адаптируй скилл под свои площадки и стиль (как в Варианте 1)
2. Подключи **MCP-сервер fal.ai** (или другого провайдера — Replicate и др.) и добавь генерацию картинки к публикации
3. Зарегистрируйся в Postiz , подключи хотя бы одну соцсеть и установи скилл Postiz в Claude Code. Или разрабатывай ассистента наоборот.
4. Запусти полную цепочку: источник → JSON с контентом → картинка → публикация через Postiz
5. Проверь публикацию в Postiz (раздел Календарь) и в самой соцсети или внутри Clode Code.

**Результат:** опубликованный пост в реальной соцсети, сгенерированный и опубликованный агентом из одного источника.

Анкета обратной связи по уроку

Эл. адрес

Курс\*

Модуль\*

Урок\*

Понравился ли урок?\*

Понравился, все было понятноНеплохо, но не все получилось/не все понятноНе понравился

Обратная связь по уроку (не обязательно)

## Задание

Предлагаем два варианта заданий. Первый поможет развить ключевое умение по теме занятия, а второй даст свободу для **проектирования алгоритма для упрощения или даже автоматизации одной из твоих рутинных рабочих задач**.



Выбери только один вариант домашнего задания и выполни его сегодня.

Другой вариант, если захочешь, можешь проработать позже самостоятельно для закрепления материала.

Вариант 1 (Базовый — повтори за экспертом) ▾

Возьми шаблон скилла **AI-News to Social** из урока и адаптируй его под себя:

1. Измени **роль агента** — опиши свою тематику и аудиторию
2. Укажи **свои площадки** (например, Telegram, VK или другие)
3. Добавь **примеры своих постов** в скилл для каждой площадки
4. Найди любую актуальную статью или новость по вашей теме и запусти скилл через Claude Code
5. Проверь, что агент создал корректный **JSON-файл** с контентом для каждой платформы в папке `post``

**Результат:** готовый JSON-файл с адаптированным контентом для минимум двух площадок.

Скрины, которые присылаем, как выполнено домашнее задание.

- Адаптированный скилл AI-News to Social: роль агента, тематика, аудитория и площадки.
- Запуск скилла в Claude Code с выбранной новостью или статьёй.
- Папка `post` и содержимое JSON-файла с контентом минимум для двух площадок.

Вариант 2 (Продвинутый — адаптируй под свою задачу) ▾

Выстрой полную цепочку контент-завода под реальную задачу:

1. Адаптируй скилл под свои площадки и стиль (как в Варианте 1)
2. Подключи **MCP-сервер fal.ai** (или другого провайдера — Replicate и др.) и добавь генерацию картинки к публикации
3. Зарегистрируйся в Postiz , подключи хотя бы одну соцсеть и установи скилл Postiz в Claude Code. Или разрабатывай ассистента наоборот.
4. Запусти полную цепочку: источник → JSON с контентом → картинка → публикация через Postiz
5. Проверь публикацию в Postiz (раздел Календарь) и в самой соцсети или внутри Clode Code.

**Результат:** опубликованный пост в реальной соцсети, сгенерированный и опубликованный агентом из одного источника.